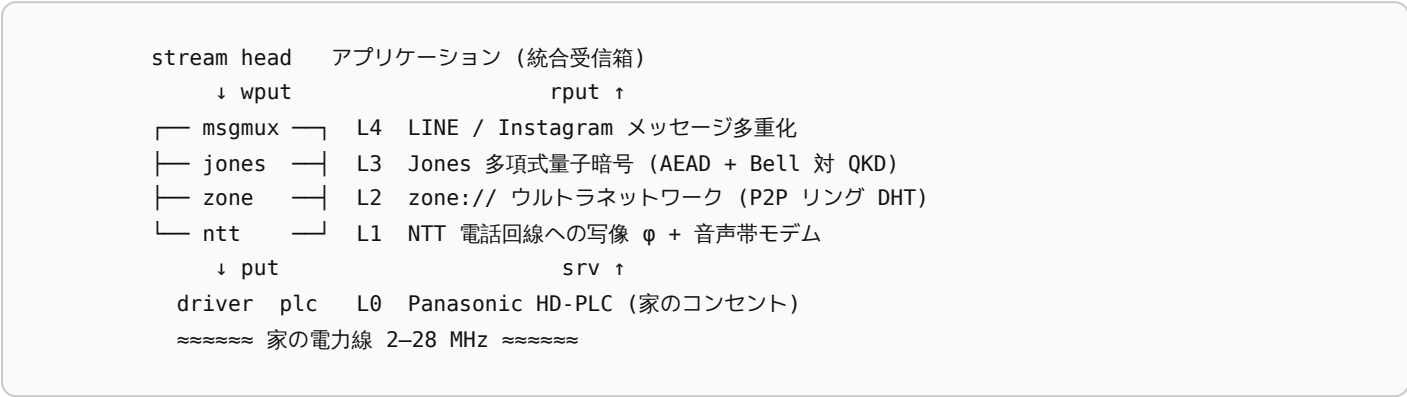


Bada UltraNetwork — 全ソースコード

HD-PLC → NTT → zone://url.or.jp → LINE / Instagram

Panasonic HD-PLC の技術で家の電力線を物理層にして PC をインターネットにつなぎ、そのフレームを NTT の電話回線へ写像し、そこに zone:// のウルトラネットワークを通し、LINE と Instagram のメッセージ機能を取り込む通信システム。

全層を量子プログラミング言語 **Bada** で記述し、層の組み方は **AT&T** ベル研究所の **STREAMS**(Dennis Ritchie の Streams I/O)に倣う。



12	4,722	159	107	512
ファイル	行	KB	自己診断項目 (全合格)	サブキャリア

目次

	ファイル	内容	行
01	bada/01_core.bada	基本演算	432
02	bada/02_sha256.bada	SHA-256 / HMAC-SHA256	198
03	bada/03_jones.bada	L3 Jones 多項式量子暗号	206
04	bada/04_zone.bada	L2 zone:// ウルトラネットワーク	267
05	bada/05_ntt.bada	L1 NTT 電話回線への写像	362
06	bada/06_plc.bada	L0 Panasonic HD-PLC	518
07	bada/07_json.bada	JSON の解析と生成	321
08	bada/08_msgmux.bada	L4 LINE / Instagram 多重化	308
09	bada/09_streams.bada	AT&T ベル研究所式 STREAMS	364
10	bada/10_main.bada	デモ本体	188
11	bada/11_selftest.bada	自己診断	387
12	ultra.bada	単一ファイル版リファレンス	1171
合計			4,722

Bada にモジュール読み込みの仕組みは無いので、実行するときは各モジュールを順に連結して 1 本にする。

node ultra_network/tools/build-bada.js が dist/ultra-full.bada(デモ本体)と dist/ultra-selftest.bada(自己診断)を作り、どちらも実際に走らせて出力を検査してから書き出す。

連結した全層プログラムは Bada の既定ステップ上限(2000 万)を超えるため、node ultra_network/tools/run-bada.js <file.bada> で上限を上げて走らせる。SHA-256 と HMAC をソフトウェアのビット演算で回すぶんが重い。

12 番の ultra.bada は既定の上限で走る縮小版(64 サブキャリア)で、node bada_gui_ide/cli/bada-cli.js run ultra_network/ultra.bada でそのまま動く。

01 bada/01_core.bada

基本演算 — 床関数・三角関数・ニブル表によるビット演算・16 進 / Base64 / UTF-8・32 ビット乗算・擬似乱数

432 行

```

1 # =====
2 # 01_core.bada -- BADA ULTRA NETWORK / 基本演算
3 #
4 # Bada には床関数・三角関数・ビット演算子が無い。この層でそれを
5 # 全部こしらえる。以降のすべてのモジュールがここに乗る。
6 #
7 # u_trunc / u_div 切り捨て・整数除算
8 # ipow / POW2 / POW16 / POW10 べき乗表
9 # u_sin / u_cos Taylor 級数 (範囲縮約つき)
10 # u_log2
11 # u_and / u_or / u_xor / u_not32 ニブル表による論理演算
12 # u_shr / u_shl32 / u_rotr32 シフトと巡回シフト
13 # hex / Base64 / UTF-8 の相互変換
14 #
15 # ビット演算は 1 ビットずつ回すと遅いので、4 ビット (ニブル) の
16 # 表を最初に作り、32 ビット語を 8 ニブルに割って引く。
17 # =====
18
19 # ---- 切り捨て (非負のみ。負には u_fix を使う) -----
20 def u_trunc(x) { return x - (x % 1) }
21
22 def u_fix(x) {
23     if (x < 0) { return 0 - ((0 - x) - ((0 - x) % 1)) }
24     return x - (x % 1)
25 }
26
27 def u_div(a, b) { return u_trunc(a / b) }
28
29 def u_min(a, b) { if (a < b) { return a } return b }
30 def u_max(a, b) { if (a > b) { return a } return b }
31
32 # ---- べき乗 -----
33 def ipow(base, e) {
34     if (e < 0) {
35         r := 1.0
36         k := 0
37         m := 0 - e
38         while (k < m) { r := r * base; k := k + 1 }
39         return 1.0 / r
40     }
41     r := 1.0
42     k := 0
43     while (k < e) { r := r * base; k := k + 1 }
44     return r
45 }
46
47 POW2 := []
48 p := 1
49 i := 0
50 while (i <= 32) { POW2 <- p; p := p * 2; i := i + 1 }
51
52 POW16 := []
53 p := 1
54 i := 0
55 while (i < 8) { POW16 <- p; p := p * 16; i := i + 1 }
56
57 POW10 := []
58 p := 1
59 i := 0
60 while (i <= 10) { POW10 <- p; p := p * 10; i := i + 1 }
61
62 TW032 := 4294967296
63 MASK32 := 4294967295
64
65 # ---- 三角関数 (Taylor 級数 + 範囲縮約) -----
66 TW0_PI := 6.283185307179586
67 PI := 3.141592653589793
68 HALF_PI := 1.5707963267948966

```

```

69
70 def u_sin(x) {
71   t := x % TWO_PI
72   if (t > PI) { t := t - TWO_PI }
73   if (t < 0 - PI) { t := t + TWO_PI }
74   term := t
75   sum := t
76   k := 1
77   while (k < 9) {
78     term := 0 - term * t * t / ((2 * k) * (2 * k + 1))
79     sum := sum + term
80     k := k + 1
81   }
82   return sum
83 }
84
85 def u_cos(x) { return u_sin(x + HALF_PI) }
86
87 LN2 := 0.6931471805599453
88 LN10 := 2.302585092994046
89 def u_log2(x) { return log(x) / LN2 }
90 def u_log10(x) { return log(x) / LN10 }
91
92 # dB <-> 線形 (10^(d/10) を exp で)
93 def db_to_lin(d) { return exp(d * LN10 / 10) }
94 def lin_to_db(v) { return 10 * log(v) / LN10 }
95
96 # ----- ニブル表による論理演算 -----
97 # XOR4[a * 16 + b] のような平坦配列を最初に作っておく。
98 XOR4 := []
99 AND4 := []
100 OR4 := []
101 a := 0
102 while (a < 16) {
103   b := 0
104   while (b < 16) {
105     av := a
106     bv := b
107     xr := 0
108     an := 0
109     orv := 0
110     pw := 1
111     n := 0
112     while (n < 4) {
113       ab := av % 2
114       bb := bv % 2
115       if (ab != bb) { xr := xr + pw }
116       if (ab == 1 && bb == 1) { an := an + pw }
117       if (ab == 1) { orv := orv + pw }
118       else { if (bb == 1) { orv := orv + pw } }
119       av := u_trunc(av / 2)
120       bv := u_trunc(bv / 2)
121       pw := pw * 2
122       n := n + 1
123     }
124     XOR4 <- xr
125     AND4 <- an
126     OR4 <- orv
127     b := b + 1
128   }
129   a := a + 1
130 }
131
132 def u_xor(x, y) {
133   r := 0
134   i := 0
135   while (i < 8) {
136     pw := POW16[i]
137     xn := u_trunc(x / pw) % 16
138     yn := u_trunc(y / pw) % 16
139     r := r + XOR4[xn * 16 + yn] * pw
140     i := i + 1
141   }
142   return r

```

```

143 }
144
145 def u_and(x, y) {
146   r := 0
147   i := 0
148   while (i < 8) {
149     pw := POW16[i]
150     xn := u_trunc(x / pw) % 16
151     yn := u_trunc(y / pw) % 16
152     r := r + AND4[xn * 16 + yn] * pw
153     i := i + 1
154   }
155   return r
156 }
157
158 def u_or(x, y) {
159   r := 0
160   i := 0
161   while (i < 8) {
162     pw := POW16[i]
163     xn := u_trunc(x / pw) % 16
164     yn := u_trunc(y / pw) % 16
165     r := r + OR4[xn * 16 + yn] * pw
166     i := i + 1
167   }
168   return r
169 }
170
171 # 8 ビット / 16 ビットだけ要するときの近道 (ニブルを 2 枚・4 枚しか引かない)。
172 # CRC の内側ループのように何千回も回る場所で効く。
173 def u_xor8(x, y) {
174   return XOR4[(u_trunc(x / 16) % 16) * 16 + (u_trunc(y / 16) % 16)] * 16 +
175     XOR4[(x % 16) * 16 + (y % 16)]
176 }
177
178 def u_and8(x, y) {
179   return AND4[(u_trunc(x / 16) % 16) * 16 + (u_trunc(y / 16) % 16)] * 16 +
180     AND4[(x % 16) * 16 + (y % 16)]
181 }
182
183 # 3 項 XOR を 1 パスで。SHA-256 の sigma 群はこの形が多く、
184 # 2 回に分けるより通り道が半分で済む。
185 def u_xor3(x, y, z) {
186   r := 0
187   i := 0
188   while (i < 8) {
189     pw := POW16[i]
190     xn := u_trunc(x / pw) % 16
191     yn := u_trunc(y / pw) % 16
192     zn := u_trunc(z / pw) % 16
193     r := r + XOR4[XOR4[xn * 16 + yn] * 16 + zn] * pw
194     i := i + 1
195   }
196   return r
197 }
198
199 def u_not32(x) { return MASK32 - x }
200
201 def u_shr(x, n) { return u_trunc(x / POW2[n]) }
202 def u_shl32(x, n) { return (x * POW2[n]) % TW032 }
203
204 # 巡回右シフト。2 つの断片はビットが重ならないので加算でよい。
205 def u_rotr32(x, n) { return u_shr(x, n) + u_shl32(x, 32 - n) }
206
207 def u_add32(x, y) { return (x + y) % TW032 }
208
209 # 32 ビット同士の乗算 (mod 2^32)。
210 # そのまま掛けると積が 2^53 を超えて倍精度の正確整数範囲から外れ、
211 # 下位ビットが失われる。16 ビットずつに割れば各部分積が 2^32 未満に
212 # 収まるので厳密に計算できる (JS の Math.imul と同じ結果になる)。
213 def u_mul32(a, b) {
214   ah := u_trunc(a / 65536) % 65536
215   al := a % 65536
216   bh := u_trunc(b / 65536) % 65536

```

```

217     bl := b % 65536
218     lo := al * bl
219     mid := (ah * bl + al * bh) % 65536
220     return (lo + mid * 65536) % TW032
221 }
222
223 # ---- 16 進 -----
224 HEXD := "0123456789abcdef"
225
226 def hex_byte(v) {
227     return substr(HEXD, u_trunc(v / 16) % 16, 1) + substr(HEXD, v % 16, 1)
228 }
229
230 def hex_of_bytes(bytes) {
231     s := ""
232     i := 0
233     while (i < len(bytes)) { s := s + hex_byte(bytes[i]); i := i + 1 }
234     return s
235 }
236
237 def hex_of_word32(w) {
238     s := ""
239     i := 3
240     while (i >= 0) { s := s + hex_byte(u_trunc(w / POW2[8 * i]) % 256); i := i - 1 }
241     return s
242 }
243
244 # ---- Base64 -----
245 B64AL := "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
246
247 def base64_of_bytes(bytes) {
248     s := ""
249     i := 0
250     n := len(bytes)
251     while (i < n) {
252         b0 := bytes[i]
253         b1 := 0
254         b2 := 0
255         if (i + 1 < n) { b1 := bytes[i + 1] }
256         if (i + 2 < n) { b2 := bytes[i + 2] }
257         c := b0 * 65536 + b1 * 256 + b2
258         s := s + substr(B64AL, u_trunc(c / 262144) % 64, 1)
259         s := s + substr(B64AL, u_trunc(c / 4096) % 64, 1)
260         if (i + 1 < n) { s := s + substr(B64AL, u_trunc(c / 64) % 64, 1) }
261         else { s := s + "=" }
262         if (i + 2 < n) { s := s + substr(B64AL, c % 64, 1) }
263         else { s := s + "=" }
264         i := i + 3
265     }
266     return s
267 }
268
269 # ---- UTF-8 -----
270 # Bada の文字列は 16 ビット符号単位。サロゲート対も正しく置く。
271 def utf8_of_str(s) {
272     out := []
273     i := 0
274     n := len(s)
275     while (i < n) {
276         c := ord(s, i)
277         if (c >= 55296 && c <= 56319 && i + 1 < n) {
278             lo := ord(s, i + 1)
279             if (lo >= 56320 && lo <= 57343) {
280                 c := 65536 + (c - 55296) * 1024 + (lo - 56320)
281                 i := i + 1
282             }
283         }
284         if (c < 128) { out <- c }
285         else {
286             if (c < 2048) {
287                 out <- (192 + u_trunc(c / 64))
288                 out <- (128 + c % 64)
289             } else {
290                 if (c < 65536) {

```

```

291         out <- (224 + u_trunc(c / 4096))
292         out <- (128 + u_trunc(c / 64) % 64)
293         out <- (128 + c % 64)
294     } else {
295         out <- (240 + u_trunc(c / 262144))
296         out <- (128 + u_trunc(c / 4096) % 64)
297         out <- (128 + u_trunc(c / 64) % 64)
298         out <- (128 + c % 64)
299     }
300 }
301 }
302 i := i + 1
303 }
304 return out
305 }
306
307 # 壊れた並びは U+FFFD に替えて 1 バイト進む (回線から来る列は
308 # 攻撃者に触られうるので、復号器は決して落ちてはならない)。
309 def str_of_utf8(bytes) {
310     s := ""
311     i := 0
312     n := len(bytes)
313     while (i < n) {
314         c := bytes[i] % 256
315         if (c < 128) { s := s + chr(c); i := i + 1 }
316         else {
317             need := 0
318             acc := 0
319             if (c >= 192 && c < 224) { need := 1; acc := c - 192 }
320             else { if (c >= 224 && c < 240) { need := 2; acc := c - 224 }
321                     else { if (c >= 240 && c < 248) { need := 3; acc := c - 240 }
322                             else { need := 0 - 1 } } }
323             if (need < 0) { s := s + chr(65533); i := i + 1 }
324             else {
325                 if (i + need >= n) { s := s + chr(65533); i := i + 1 }
326                 else {
327                     ok := true
328                     k := 1
329                     while (k <= need) {
330                         cb := bytes[i + k] % 256
331                         if (cb < 128) { ok := false }
332                         if (cb >= 192) { ok := false }
333                         acc := acc * 64 + (cb % 64)
334                         k := k + 1
335                     }
336                     bad := false
337                     if (!ok) { bad := true }
338                     if (need == 1 && acc < 128) { bad := true }
339                     if (need == 2 && acc < 2048) { bad := true }
340                     if (need == 3 && acc < 65536) { bad := true }
341                     if (acc >= 55296 && acc <= 57343) { bad := true }
342                     if (acc > 1114111) { bad := true }
343                     if (bad) { s := s + chr(65533); i := i + 1 }
344                     else {
345                         if (acc < 65536) { s := s + chr(acc) }
346                         else {
347                             v := acc - 65536
348                             s := s + chr(55296 + u_trunc(v / 1024)) + chr(56320 + v % 1024)
349                         }
350                         i := i + need + 1
351                     }
352                 }
353             }
354         }
355     }
356     return s
357 }
358
359 # ---- バイト列 <-> ビット列 -----
360 def bytes_to_bits(bytes) {
361     bits := []
362     i := 0
363     while (i < len(bytes)) {
364         v := bytes[i]

```

```

365         k := 7
366         while (k >= 0) {
367             pw := POW2[k]
368             if (v >= pw) { bits <- 1; v := v - pw } else { bits <- 0 }
369             k := k - 1
370         }
371         i := i + 1
372     }
373     return bits
374 }
375
376 def bits_to_bytes(bits) {
377     out := []
378     i := 0
379     while (i + 7 < len(bits)) {
380         v := 0
381         k := 0
382         while (k < 8) { v := v * 2 + bits[i + k]; k := k + 1 }
383         out <- v
384         i := i + 8
385     }
386     return out
387 }
388
389 # ---- 文字列ハッシュ (zone.bada と完全に同一。相互運用のため不変) ----
390 def str_hash(s) {
391     h := 7
392     i := 0
393     n := len(s)
394     while (i < n) {
395         h := (h * 31 + ord(s, i)) % 999983
396         i := i + 1
397     }
398     return h
399 }
400
401 # 32 ビット混合ハッシュ (番号写像・疑似乱数の種に使う)。
402 # FNV-1a に最終攪拌を足したもの。索引 index.html の JS 実装 mix32 と
403 # 定数まで同一にしてあるので、両者は同じ乱数列 = 同じ回線特性を出す。
404 def mix32(s) {
405     h := 2166136261
406     i := 0
407     while (i < len(s)) {
408         h := u_xor(h, ord(s, i))
409         h := u_mul32(h, 16777619)      # FNV 素数
410         i := i + 1
411     }
412     h := u_xor(h, u_shr(h, 15))
413     h := u_mul32(h, 625341585)      # 0x2545F491
414     h := u_xor(h, u_shr(h, 13))
415     return h
416 }
417
418 # 再現性のある疑似乱数 (同じ種なら常に同じ列)
419 def rng_new(seed) {
420     s := mix32(seed)
421     if (s == 0) { s := 1 }
422     return [s]
423 }
424
425 def rng_next(st) {
426     s := st[0]
427     s := u_xor(s, u_shl32(s, 13))
428     s := u_xor(s, u_shr(s, 17))
429     s := u_xor(s, u_shl32(s, 5))
430     st[0] = s
431     return s / TW032
432 }

```

02 bada/02_sha256.bada

SHA-256 / HMAC-SHA256 — RFC 6234 と RFC 2104。Webhook 署名の検証に使う。ビット演算子が無いので 01 の表引きの上に組む

198 行

```

1 # =====
2 # 02_sha256.bada -- BADA ULTRA NETWORK / SHA-256 と HMAC-SHA256
3 #
4 # LINE の X-Line-Signature (HMAC-SHA256 -> Base64) と Instagram の
5 # X-Hub-Signature-256 (HMAC-SHA256 -> hex) を、本物の計算で検証する
6 # ために要る。RFC 6234 (SHA-2) と RFC 2104 (HMAC) のとおり。
7 #
8 # Bada にビット演算子は無いので、01_core.bada のニブル表による
9 # u_xor / u_and / u_not32 / u_rotr32 の上に組む。1 ブロックあたり
10 # およそ 9 万ステップで回る。
11 # =====
12
13 SHA_K := [
14     1116352408, 1899447441, 3049323471, 3921009573, 961987163, 1508970993,
15     2453635748, 2870763221, 3624381080, 310598401, 607225278, 1426881987,
16     1925078388, 2162078206, 2614888103, 3248222580, 3835390401, 4022224774,
17     264347078, 604807628, 770255983, 1249150122, 1555081692, 1996064986,
18     2554220882, 2821834349, 2952996808, 3210313671, 3336571891, 3584528711,
19     113926993, 338241895, 666307205, 773529912, 1294757372, 1396182291,
20     1695183700, 1986661051, 2177026350, 2456956037, 2730485921, 2820302411,
21     3259730800, 3345764771, 3516065817, 3600352804, 4094571909, 275423344,
22     430227734, 506948616, 659060556, 883997877, 958139571, 1322822218,
23     1537002063, 1747873779, 1955562222, 2024104815, 2227730452, 2361852424,
24     2428436474, 2756734187, 3204031479, 3329325298
25 ]
26
27 SHA_H0 := [1779033703, 3144134277, 1013904242, 2773480762,
28             1359893119, 2600822924, 528734635, 1541459225]
29
30 # 小文字 sigma (メッセージスケジュール用)
31 def sha_s0(x) { return u_xor3(u_rotr32(x, 7), u_rotr32(x, 18), u_shr(x, 3)) }
32 def sha_s1(x) { return u_xor3(u_rotr32(x, 17), u_rotr32(x, 19), u_shr(x, 10)) }
33
34 # 大文字 Sigma (圧縮関数用)
35 def sha_S0(x) { return u_xor3(u_rotr32(x, 2), u_rotr32(x, 13), u_rotr32(x, 22)) }
36 def sha_S1(x) { return u_xor3(u_rotr32(x, 6), u_rotr32(x, 11), u_rotr32(x, 25)) }
37
38 # ch と maj はニブル 1 パスに融合する。and/not/xor を別々に呼ぶと
39 # 32 ビット語を 3 度も 5 度も舐め直すことになり、そこが一番効く。
40 # 4 ビットの NOT は 15 - n。
41 def sha_ch(e, f, g) {
42     r := 0
43     i := 0
44     while (i < 8) {
45         pw := POW16[i]
46         en := u_trunc(e / pw) % 16
47         fn := u_trunc(f / pw) % 16
48         gn := u_trunc(g / pw) % 16
49         r := r + XOR4[AND4[en * 16 + fn] * 16 + AND4[(15 - en) * 16 + gn]] * pw
50         i := i + 1
51     }
52     return r
53 }
54
55 def sha_maj(a, b, c) {
56     r := 0
57     i := 0
58     while (i < 8) {
59         pw := POW16[i]
60         an := u_trunc(a / pw) % 16
61         bn := u_trunc(b / pw) % 16
62         cn := u_trunc(c / pw) % 16
63         r := r + XOR4[XOR4[AND4[an * 16 + bn] * 16 + AND4[an * 16 + cn]] * 16 +
64                     AND4[bn * 16 + cn]] * pw
65         i := i + 1
66     }
67     return r

```

```

68 }
69
70 # バイト列 -> 32 バイトのダイジェスト (バイト列)
71 def sha256(msg) {
72   n := len(msg)
73   m := []
74   i := 0
75   while (i < n) { m <- msg[i]; i := i + 1 }
76   m <- 128
77   while (len(m) % 64 != 56) { m <- 0 }
78   # ビット長を 64 ビット ビッグエンディアンで
79   bits_hi := u_trunc(n / 536870912)
80   bits_lo := (n * 8) % TW032
81   k := 3
82   while (k >= 0) { m <- (u_trunc(bits_hi / POW2[8 * k]) % 256); k := k - 1 }
83   k := 3
84   while (k >= 0) { m <- (u_trunc(bits_lo / POW2[8 * k]) % 256); k := k - 1 }
85
86   h := []
87   i := 0
88   while (i < 8) { h <- SHA_H0[i]; i := i + 1 }
89
90   w := []
91   i := 0
92   while (i < 64) { w <- 0; i := i + 1 }
93
94   blk := 0
95   nblk := u_trunc(len(m) / 64)
96   while (blk < nblk) {
97     base := blk * 64
98     t := 0
99     while (t < 16) {
100       w[t] := m[base + t * 4] * 16777216 + m[base + t * 4 + 1] * 65536 +
101         m[base + t * 4 + 2] * 256 + m[base + t * 4 + 3]
102       t := t + 1
103     }
104     t := 16
105     while (t < 64) {
106       w[t] := (sha_s1(w[t - 2]) + w[t - 7] + sha_s0(w[t - 15]) + w[t - 16]) % TW032
107       t := t + 1
108     }
109     av := h[0]
110     bv := h[1]
111     cv := h[2]
112     dv := h[3]
113     ev := h[4]
114     fv := h[5]
115     gv := h[6]
116     hv := h[7]
117     t := 0
118     while (t < 64) {
119       t1 := (hv + sha_s1(ev) + sha_ch(ev, fv, gv) + SHA_K[t] + w[t]) % TW032
120       t2 := (sha_s0(av) + sha_maj(av, bv, cv)) % TW032
121       hv := gv
122       gv := fv
123       fv := ev
124       ev := (dv + t1) % TW032
125       dv := cv
126       cv := bv
127       bv := av
128       av := (t1 + t2) % TW032
129       t := t + 1
130     }
131     h[0] := (h[0] + av) % TW032
132     h[1] := (h[1] + bv) % TW032
133     h[2] := (h[2] + cv) % TW032
134     h[3] := (h[3] + dv) % TW032
135     h[4] := (h[4] + ev) % TW032
136     h[5] := (h[5] + fv) % TW032
137     h[6] := (h[6] + gv) % TW032
138     h[7] := (h[7] + hv) % TW032
139     blk := blk + 1
140   }
141 }

```

```

142     out := []
143     i := 0
144     while (i < 8) {
145         k := 3
146         while (k >= 0) { out <- (u_trunc(h[i] / POW2[8 * k]) % 256); k := k - 1 }
147         i := i + 1
148     }
149     return out
150 }
151
152 def sha256_hex(msg) { return hex_of_bytes(sha256(msg)) }
153
154 # ---- HMAC (RFC 2104) -----
155 def hmac_sha256(key, msg) {
156     k := []
157     i := 0
158     if (len(key) > 64) {
159         kk := sha256(key)
160         i := 0
161         while (i < len(kk)) { k <- kk[i]; i := i + 1 }
162     } else {
163         i := 0
164         while (i < len(key)) { k <- key[i]; i := i + 1 }
165     }
166     while (len(k) < 64) { k <- 0 }
167
168     inner := []
169     outer := []
170     i := 0
171     while (i < 64) {
172         inner <- u_xor(k[i], 54)
173         outer <- u_xor(k[i], 92)
174         i := i + 1
175     }
176     i := 0
177     while (i < len(msg)) { inner <- msg[i]; i := i + 1 }
178     ih := sha256(inner)
179     i := 0
180     while (i < len(ih)) { outer <- ih[i]; i := i + 1 }
181     return sha256(outer)
182 }
183
184 def hmac_sha256_hex(key, msg) { return hex_of_bytes(hmac_sha256(key, msg)) }
185 def hmac_sha256_b64(key, msg) { return base64_of_bytes(hmac_sha256(key, msg)) }
186
187 # 署名照合はタイミング差を漏らさない
188 def const_time_eq(a, b) {
189     if (len(a) != len(b)) { return false }
190     d := 0
191     i := 0
192     while (i < len(a)) { d := u_or(d, u_xor(a[i], b[i])); i := i + 1 }
193     return d == 0
194 }
195
196 def const_time_eq_str(a, b) {
197     return const_time_eq(utf8_of_str(a), utf8_of_str(b))
198 }

```

03 bada/03_jones.bada

L3 Jones 多項式量子暗号 — Kauffman ブラケットから導く結び目鍵・Bell 対 QKD・鍵ストリーム AEAD

206 行

```

1 # =====
2 # 03_jones.bada -- BADA ULTRA NETWORK / L3 Jones 多項式量子暗号
3 #
4 # ゾーンの秘密は「結び目」そのもの。
5 #
6 # 1. 鍵 結び目図の Kauffman ブラケット
7 # <D>(A) = Σ (A/B 平滑化について) A^(a-b) · d^(loops-1),
8 # d = -A² - 1/A² を複数の A で標準化してハッシュしたものが
9 # そのゾーンの長期鍵になる。結び目が秘密で、多項式が鍵スケジュール。
10 # 2. QKD セッションソルトは Bell 対 (H + CNOT + 測定) が合意する。
11 # 禁止状態 |01>, |10> の確率が厳密に 0 であることが盗聴されて
12 # いない証拠。複製不可能定理により盗聴は相関を壊す。
13 # 3. AEAD (結び目鍵, ソルト) を種とする鍵ストリームで暗号化し、
14 # 鍵付きタグで封緘する。改竄・鍵違いは 409 zone-guard-reject。
15 #
16 # 算法は bada_gui_ide/examples/zone.bada と完全に同一。よって
17 # ここで作ったレコードは ZoneBrowser がそのまま開ける。
18 # =====
19
20 def uf_find(parent, x) {
21     r := x
22     while (parent[r] >= 0) { r := parent[r] }
23     return r
24 }
25
26 # 交点は [id, e0, e1, e2, e3, sign]。ブラケットは sign を使わない。
27 def kauffman(cross, A) {
28     n := len(cross)
29     if (n == 0) { return 1.0 }
30     maxlbl := 0
31     i := 0
32     while (i < n) {
33         k := 1
34         while (k <= 4) {
35             if (cross[i][k] > maxlbl) { maxlbl := cross[i][k] }
36             k := k + 1
37         }
38         i := i + 1
39     }
40     U := maxlbl + 1
41     d := 0 - A * A - 1.0 / (A * A)
42     states := POW2[n]
43     sum := 0.0
44     bits := []
45     q := 0
46     while (q < n) { bits <- 0; q := q + 1 }
47
48     st := 0
49     while (st < states) {
50         parent := []
51         q := 0
52         while (q < U) { parent <- (0 - 1); q := q + 1 }
53         a_cnt := 0
54         b_cnt := 0
55         i := 0
56         while (i < n) {
57             e0 := cross[i][1]
58             e1 := cross[i][2]
59             e2 := cross[i][3]
60             e3 := cross[i][4]
61             # A 平滑化は (e0,e1)(e2,e3)、B 平滑化は (e0,e3)(e1,e2)
62             if (bits[i] == 0) {
63                 a_cnt := a_cnt + 1
64                 p := e0; qq := e1; r := e2; s := e3
65             } else {
66                 b_cnt := b_cnt + 1
67                 p := e0; qq := e3; r := e1; s := e2
68             }

```

```

69     ra := uf_find(parent, p)
70     rb := uf_find(parent, qq)
71     if (ra != rb) { parent[rb] = ra }
72     rc := uf_find(parent, r)
73     rd := uf_find(parent, s)
74     if (rc != rd) { parent[rd] = rc }
75     i := i + 1
76 }
77 # 閉ループの本数を数える
78 seen := []
79 q := 0
80 while (q < U) { seen <- 0; q := q + 1 }
81 loops := 0
82 lbl := 0
83 while (lbl < U) {
84     r2 := uf_find(parent, lbl)
85     if (seen[r2] == 0) { seen[r2] = 1; loops := loops + 1 }
86     lbl := lbl + 1
87 }
88 sum := sum + ipow(A, a_cnt - b_cnt) * ipow(d, loops - 1)
89 # 次の状態へ (2 進の繰り上がり)
90 c := 0
91 carry := 1
92 while (c < n && carry == 1) {
93     if (bits[c] == 0) { bits[c] = 1; carry := 0 }
94     else { bits[c] = 0 }
95     c := c + 1
96 }
97 st := st + 1
98 }
99 return sum
100 }
101
102 JONES_A := [0.8, 1.0, 1.2, 1.5, 2.0]
103
104 def jones_key(diagram) {
105     acc := "jones|"
106     i := 0
107     while (i < len(JONES_A)) {
108         acc := acc + f5(kauffman(diagram, JONES_A[i])) + "|"
109         i := i + 1
110     }
111     return str_hash(acc)
112 }
113
114 # 各ゾーンの正体である結び目 (= その秘密)。zone.bada と同一。
115 # url.or.jp 三葉結び目 (交点 3)
116 # bada.or.jp 8 の字結び目 (交点 4)
117 def zone_diagram(host) {
118     if (host == "bada.or.jp") {
119         return [ [0,1,2,3,4,1], [1,3,4,5,6,1], [2,5,6,7,8,1], [3,7,8,1,2,1] ]
120     }
121     return [ [0,1,2,3,4,1], [1,3,4,5,6,1], [2,5,6,1,2,1] ]
122 }
123
124 # ---- 鍵ストリーム AEAD -----
125 def lcg(state) { return (state * 48271) % 2147483647 }
126
127 def key_seed(key, salt) {
128     state := (key + salt * 2654435761) % 2147483647
129     if (state <= 0) { state := state + 2147483646 }
130     return state
131 }
132
133 # 16 ビット符号単位で動くので CJK も絵文字も往復する
134 def zone_encrypt(plain, key, salt) {
135     state := key_seed(key, salt)
136     out := []
137     i := 0
138     while (i < len(plain)) {
139         state := lcg(state)
140         out <- (ord(plain, i) + (state % 65536)) % 65536
141         i := i + 1
142     }

```

```

143     return out
144 }
145
146 def zone_decrypt(ct, key, salt) {
147     state := key_seed(key, salt)
148     out := ""
149     i := 0
150     while (i < len(ct)) {
151         state := lcg(state)
152         out := out + chr((ct[i] - (state % 65536) + 65536) % 65536)
153         i := i + 1
154     }
155     return out
156 }
157
158 # 暗号文にかけける鍵付き認証タグ
159 def zone_tag(ct, key, salt) {
160     h := (key % 999983) + salt + 1
161     i := 0
162     while (i < len(ct)) {
163         h := (h * 31 + ct[i] + 7) % 2147483647
164         i := i + 1
165     }
166     return h
167 }
168
169 # 封緘 -> [ciphertext, salt, tag]
170 def zone_seal(plain, key, salt) {
171     ct := zone_encrypt(plain, key, salt)
172     return [ct, salt, zone_tag(ct, key, salt)]
173 }
174
175 # 開封 -> [plaintext]、[] なら 409 zone-guard-reject
176 def zone_open(sealed, key) {
177     ct := sealed[0]
178     salt := sealed[1]
179     tag := sealed[2]
180     if (zone_tag(ct, key, salt) != tag) { return [] }
181     return [zone_decrypt(ct, key, salt)]
182 }
183
184 # ---- Bell 対 QKD -----
185 #  $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$  では  $|01\rangle$  と  $|10\rangle$  の確率が厳密に 0。
186 # intercept-resend の盗聴はこの相関を壊すので、そこで検出できる。
187 def qkd_session() {
188     bell := qubit(2)
189     bell := H(bell, 0)
190     bell := CNOT(bell, 0, 1)
191     outcome := Measure(bell)
192     forbidden := outcome[1] + outcome[2]
193     ok := (outcome[1] == 0) && (outcome[2] == 0)
194     salt := (len(tuplespace) * 131 + 17) % 4096
195     return [ok, salt, outcome, forbidden]
196 }
197
198 # 盗聴された量子路の模擬 (片側を計算基底で測って再送した後の分布)。
199 # 禁止状態が 0 でなくなるので qkd_check が弾く。
200 def qkd_session_tapped() {
201     dist := [0.5, 0.1225, 0.1225, 0.32]
202     salt := (len(tuplespace) * 131 + 17) % 4096
203     return [false, salt, dist, dist[1] + dist[2]]
204 }
205
206 def qkd_check(q) { return q[0] }

```

04 bada/04_zone.bada

L2 zone:// ウルトラネットワーク — URL 文法・P2P リング DHT・ピアの保管庫・アカシック台帳・@@ ブロック・封筒

267 行

```

1 # =====
2 # 04_zone.bada -- BADA ULTRA NETWORK / L2 zone:// ウルトラネットワーク WWW
3 #
4 # https: / http: に代わるスキーム。中央サーバも DNS ルートも無い。
5 # アドレスは「ピア自身のハッシュだけ」から作ったリング DHT が解決し、
6 # 全レコードは追記専用の tuplespace (アカシック台帳) に載る。
7 #
8 # zone://<host>[/<path>]
9 #
10 # 出力する "@@" ブロックは既存の ZoneBrowser
11 # (bada_gui_id/dist/zone-browser.html) が読む形式と同じ。
12 # =====
13
14 RING := 4096      # リングの位数 (zone.bada と同一)
15
16 def zone_id(s) { return str_hash(s) % RING }
17
18 # zone://host/path -> [host, path, labels] ([] なら zone URL ではない)
19 def zone_parse(url) {
20     if (str_find(url, "zone://") != 0) { return [] }
21     rest := substr(url, 7, len(url) - 7)
22     cut := str_find(rest, "/")
23     if (cut < 0) { return [rest, "/", split(rest, ".")] }
24     host := substr(rest, 0, cut)
25     path := substr(rest, cut, len(rest) - cut)
26     return [host, path, split(host, ".")]
27 }
28
29 # リング上の巡回距離
30 def ring_dist(a, b) {
31     d := a - b
32     if (d < 0) { d := 0 - d }
33     e := RING - d
34     if (e < d) { return e }
35     return d
36 }
37
38 # ピアは [name, node-id, ring-index, store_urls, store_recs]
39 def peer_new(name) { return [name, zone_id("peer:" + name), 0, [], []] }
40
41 ZONE_PEERS := ["tokyo.zone.jp", "osaka.zone.jp", "nagoya.zone.jp",
42               "sapporo.zone.jp", "fukuoka.zone.jp", "naha.zone.jp"]
43
44 def zone_boot() {
45     net := []
46     i := 0
47     while (i < len(ZONE_PEERS)) { net <- peer_new(ZONE_PEERS[i]); i := i + 1 }
48     # node-id の昇順へ挿入ソート
49     sorted := []
50     i := 0
51     while (i < len(net)) {
52         p := net[i]
53         pos := len(sorted)
54         found := false
55         j := 0
56         while (j < len(sorted)) {
57             if (!found) {
58                 if (p[1] < sorted[j][1]) { pos := j; found := true }
59             }
60             j := j + 1
61         }
62         out := []
63         k := 0
64         while (k < len(sorted)) {
65             if (k == pos) { out <- p }
66             out <- sorted[k]
67             k := k + 1
68         }

```

```

69     if (pos == len(sorted)) { out <- p }
70     sorted := out
71     i := i + 1
72 }
73 m := 0
74 while (m < len(sorted)) { sorted[m][2] = m; m := m + 1 }
75 return sorted
76 }
77
78 # 鍵を担当するピアの添字
79 def dht_nearest(net, key) {
80     best := 0
81     bd := ring_dist(net[0][1], key)
82     i := 1
83     while (i < len(net)) {
84         d := ring_dist(net[i][1], key)
85         if (d < bd) { bd := d; best := i }
86         i := i + 1
87     }
88     return best
89 }
90
91 # 貪欲ルーティング。隣接 (+1 / -1 / +2 のフィンガ) だけを見て進む。
92 # -> [担当ピアの添字, ホップした名前の並び]
93 def zone_route(net, key) {
94     n := len(net)
95     target := dht_nearest(net, key)
96     cur := 0
97     hops := [net[0][0]]
98     guard := 0
99     while (cur != target && guard < 2 * n) {
100         nb := [(cur + 1) % n, (cur + n - 1) % n, (cur + 2) % n]
101         best := cur
102         bd := ring_dist(net[cur][1], key)
103         i := 0
104         while (i < len(nb)) {
105             d := ring_dist(net[nb[i]][1], key)
106             if (d < bd) { bd := d; best := nb[i] }
107             i := i + 1
108         }
109         if (best == cur) { cur := target } else { cur := best }
110         hops <- net[cur][0]
111         guard := guard + 1
112     }
113     return [target, hops]
114 }
115
116 # ----- ピアの保管庫 -----
117 def peer_store_put(peer, url, sealed) {
118     i := 0
119     while (i < len(peer[3])) {
120         if (peer[3][i] == url) { peer[4][i] = sealed; return i }
121         i := i + 1
122     }
123     peer[3] <- url
124     peer[4] <- sealed
125     return len(peer[3]) - 1
126 }
127
128 # -> [sealed]、[] なら 404 zone-not-found
129 def peer_store_get(peer, url) {
130     i := 0
131     while (i < len(peer[3])) {
132         if (peer[3][i] == url) { return [peer[4][i]] }
133         i := i + 1
134     }
135     return []
136 }
137
138 # ----- 状態符号 -----
139 def zone_status_text(code) {
140     if (code == 200) { return "zone-delivered" }
141     if (code == 400) { return "bad-zone" }
142     if (code == 404) { return "zone-not-found" }

```

```

143     if (code == 409) { return "zone-guard-reject" }
144     if (code == 495) { return "quantum-channel-compromised" }
145     if (code == 503) { return "plc-link-failure" }
146     if (code == 401) { return "webhook-signature-mismatch" }
147     return "unknown"
148 }
149
150 # ---- 公開と取得 -----
151 # NET を渡して url を封緘・保管し、台帳に追記する -> [status, ownerId, hops]
152 def zone_publish(net, url, content) {
153     z := zone_parse(url)
154     if (len(z) == 0) { return [400, 0 - 1, []] }
155     q := qkd_session()
156     if (!qkd_check(q)) { return [495, 0 - 1, []] }
157     key := jones_key(zone_diagram(z[0]))
158     sealed := zone_seal(content, key, q[1])
159     r := zone_route(net, zone_id(url))
160     peer_store_put(net[r[0]], url, sealed)
161     fact := ["zone-record", url, net[r[0]][0], zone_id(url), sealed[2]]
162     fact >> tuplespace
163     return [200, r[0], r[1]]
164 }
165
166 # -> [status, 本文, ownerId, hops]
167 def zone_get(net, url) {
168     z := zone_parse(url)
169     if (len(z) == 0) { return [400, "", 0 - 1, []] }
170     r := zone_route(net, zone_id(url))
171     hit := peer_store_get(net[r[0]], url)
172     if (len(hit) == 0) { return [404, "", r[0], r[1]] }
173     key := jones_key(zone_diagram(z[0]))
174     opened := zone_open(hit[0], key)
175     fact := ["zone-fetch", url, net[r[0]][0]]
176     fact >> tuplespace
177     if (len(opened) == 0) { return [409, "", r[0], r[1]] }
178     return [200, opened[0], r[0], r[1]]
179 }
180
181 # ---- "@@" ブロック (ZoneBrowser が読む形式) -----
182 def zone_serve_block(net, url, status, body, sealed, key, qkd_ok) {
183     z := zone_parse(url)
184     host := ""
185     path := ""
186     if (len(z) > 0) { host := z[0]; path := z[1] }
187     r := zone_route(net, zone_id(url))
188     hops := ""
189     i := 0
190     while (i < len(r[1])) {
191         if (i > 0) { hops := hops + " -> " }
192         hops := hops + r[1][i]
193         i := i + 1
194     }
195     print("@@HOST " + host)
196     print("@@PATH " + path)
197     print("@@KEY " + zone_id(url))
198     print("@@ROUTE " + hops)
199     print("@@NODE " + net[r[0]][0])
200     if (qkd_ok) { print("@@QKD ok") } else { print("@@QKD compromised") }
201     if (len(sealed) == 3) {
202         ct := sealed[0]
203         c0 := 0
204         c1 := 0
205         c2 := 0
206         if (len(ct) > 0) { c0 := ct[0] }
207         if (len(ct) > 1) { c1 := ct[1] }
208         if (len(ct) > 2) { c2 := ct[2] }
209         print("@@CIPHER " + c0 + " " + c1 + " " + c2 + " " + len(ct))
210         print("@@TAG " + sealed[2])
211     }
212     print("@@JONESKEY " + key)
213     print("@@STATUS " + status)
214     print("@@BODY_BEGIN")
215     print(body)
216     print("@@BODY_END")

```

```

217     return status
218 }
219
220 # ---- 封筒 (回線に流す実バイト列) -----
221 # "Z" "N" | ver | urlLen | url(utf8) | salt(2) | tag(4) | ctLen(2) | ct(2 each)
222 def envelope_encode(url, sealed) {
223     ub := utf8_of_str(url)
224     out := [90, 78, 1]
225     out <- u_trunc(len(ub) / 256)
226     out <- len(ub) % 256
227     i := 0
228     while (i < len(ub)) { out <- ub[i]; i := i + 1 }
229     out <- u_trunc(sealed[1] / 256)
230     out <- sealed[1] % 256
231     t := sealed[2]
232     k := 3
233     while (k >= 0) { out <- (u_trunc(t / POW2[8 * k]) % 256); k := k - 1 }
234     ct := sealed[0]
235     out <- u_trunc(len(ct) / 256)
236     out <- len(ct) % 256
237     i := 0
238     while (i < len(ct)) {
239         out <- u_trunc(ct[i] / 256)
240         out <- ct[i] % 256
241         i := i + 1
242     }
243     return out
244 }
245
246 # -> [status, url, sealed]
247 def envelope_decode(bytes) {
248     if (len(bytes) < 13) { return [400, "", []] }
249     if (bytes[0] != 90) { return [400, "", []] }
250     if (bytes[1] != 78) { return [400, "", []] }
251     ulen := bytes[3] * 256 + bytes[4]
252     if (len(bytes) < 5 + ulen + 8) { return [400, "", []] }
253     ub := []
254     i := 0
255     while (i < ulen) { ub <- bytes[5 + i]; i := i + 1 }
256     url := str_of_utf8(ub)
257     p := 5 + ulen
258     salt := bytes[p] * 256 + bytes[p + 1]
259     tag := bytes[p + 2] * 16777216 + bytes[p + 3] * 65536 + bytes[p + 4] * 256 + bytes[p + 5]
260     clen := bytes[p + 6] * 256 + bytes[p + 7]
261     p := p + 8
262     if (len(bytes) < p + 2 * clen) { return [400, "", []] }
263     ct := []
264     i := 0
265     while (i < clen) { ct <- (bytes[p + 2 * i] * 256 + bytes[p + 2 * i + 1]); i := i + 1 }
266     return [200, url, [ct, salt, tag]]
267 }

```

05 bada/05_ntt.bada

L1 NTT 電話回線への写像 — 写像 ϕ と割当表・音声帯 16-QAM モデム・G.711 μ -law・RTP・ISDN・呼制御

362 行

```

1 # =====
2 # 05_ntt.bada -- BADA ULTRA NETWORK / L1 NTT 電話回線への写像
3 #
4 # 写像  $\phi$  : zone:// アドレス |-> NTT 番号
5 #   リング上の位置 zone_id が市外局番を決める (近い zone は近い収容局
6 #   になる)。加入者番号は URL のハッシュから採り、衝突は局内の空き
7 #   番号を線形探索して解消する。番号ポータビリティと同じ「割当表」を
8 #   持つので  $\phi^{-1}$  は厳密な逆写像になる。
9 #
10 # ベアラ (実際に加入者線を通す部分) は 3 種類:
11 #   メタル      300-3400 Hz の音声帯 16-QAM ソフトモデム
12 #               fs 8 kHz / 搬送波 2 kHz / 2000 baud / 8000 bps
13 #   INS ネット64 B チャンネル 64 kbps のデジタル透過 (HDLC)
14 #   ひかり電話  G.711  $\mu$ -law に符号化して RTP (PT=0 PCMU / 20 ms)
15 #
16 #   搬送波が標準化周波数のちょうど 1/4 なので cos/sin は
17 #   {1,0,-1,0} と {0,1,0,-1} という厳密な直交系になり、相関復調に
18 #   近似が入らない。だから音声帯を通して誤差なしで往復する。
19 # =====
20
21 AREA_CODE := ["011", "022", "03", "045", "052", "06", "075", "082", "092", "098"]
22 AREA_CITY := ["札幌", "仙台", "東京", "横浜", "名古屋", "大阪", "京都", "広島", "福岡", "那覇"]
23 AREA_SUB  := [ 7, 7, 8, 7, 7, 8, 7, 7, 7, 7]
24
25 # 割当表 (追記専用の平行配列)
26 NTT_REG_URL := []
27 NTT_REG_NUM := []
28 NTT_REG_KIND := []
29 NTT_REG_SEC := []
30
31 def ntt_pad(v, width) {
32   s := "" + (v % POW10[width])
33   while (len(s) < width) { s := "0" + s }
34   return s
35 }
36
37 def ntt_find_num(number) {
38   i := 0
39   while (i < len(NTT_REG_NUM)) {
40     if (NTT_REG_NUM[i] == number) { return i }
41     i := i + 1
42   }
43   return 0 - 1
44 }
45
46 def ntt_find_url(url, kind) {
47   i := 0
48   while (i < len(NTT_REG_URL)) {
49     if (NTT_REG_URL[i] == url && NTT_REG_KIND[i] == kind) { return i }
50     i := i + 1
51   }
52   return 0 - 1
53 }
54
55 #  $\phi$ . kind は "geo" (0AB-J) / "ip" (050) / "mobile" (090)
56 # -> [番号, 区画添字, 探索回数]
57 def ntt_map(url, kind) {
58   hit := ntt_find_url(url, kind)
59   if (hit >= 0) { return [NTT_REG_NUM[hit], NTT_REG_SEC[hit], 0] }
60   area := "050"
61   width := 8
62   sector := 0 - 1
63   if (kind == "mobile") { area := "090"; width := 8 }
64   else {
65     if (kind != "ip") {
66       z := zone_id(url)
67       sector := u_trunc(z * len(AREA_CODE) / RING)
68       if (sector >= len(AREA_CODE)) { sector := len(AREA_CODE) - 1 }

```

```

69     area := AREA_CODE[sector]
70     width := AREA_SUB[sector]
71 }
72 }
73 base := mix32("ntt:" + kind + ":" + url) % POW10[width]
74 probe := 0
75 number := area + ntt_pad(base, width)
76 while (ntt_find_num(number) >= 0 && probe < 100000) {
77     probe := probe + 1
78     number := area + ntt_pad(base + probe, width)
79 }
80 NTT_REG_URL <- url
81 NTT_REG_KIND <- kind
82 NTT_REG_SEC <- sector
83 NTT_REG_NUM <- number
84 fact := ["ntt-assign", url, number, kind]
85 fact >> tuplespace
86 return [number, sector, probe]
87 }
88
89 # φ-1 -- 割当表の厳密な逆引き。未割当なら ""
90 def ntt_unmap(number) {
91     digits := ""
92     i := 0
93     while (i < len(number)) {
94         c := ord(number, i)
95         if (c >= 48 && c <= 57) { digits := digits + chr(c) }
96         i := i + 1
97     }
98     hit := ntt_find_num(digits)
99     if (hit < 0) { return "" }
100    return NTT_REG_URL[hit]
101 }
102
103 # 表示整形
104 #   10桁 0AB-J : 03-1234-5678 (市外 2 桁) / 011-234-5678 (市外 3 桁)
105 #   11桁       : 050-1234-5678 / 090-1234-5678 (IP 電話・携帯)
106 def ntt_format(number) {
107     n := len(number)
108     if (n == 11) {
109         return substr(number, 0, 3) + "-" + substr(number, 3, 4) + "-" + substr(number, 7, 4)
110     }
111     if (n != 10) { return number }
112     head := 3
113     d1 := substr(number, 1, 1)
114     if (d1 == "3") { head := 2 }
115     if (d1 == "6") { head := 2 }
116     mid := 10 - head - 4
117     return substr(number, 0, head) + "-" + substr(number, head, mid) + "-" +
118         substr(number, head + mid, 4)
119 }
120
121 def ntt_e164(number) { return "+81" + substr(number, 1, len(number) - 1) }
122
123 def ntt_city(sector) {
124     if (sector < 0) { return "IP / 携帯" }
125     return AREA_CITY[sector]
126 }
127
128 # ---- 回線種別 -----
129 def ntt_line_name(id) {
130     if (id == "isdn") { return "INS ネット 64 (ISDN)" }
131     if (id == "hikari") { return "ひかり電話 (IP)" }
132     return "アナログ加入電話 (メタル)"
133 }
134
135 def ntt_line_bps(id) {
136     if (id == "metal") { return 8000 }
137     return 64000
138 }
139
140 # ---- 音声帯 16-QAM ソフトモデム -----
141 NTT_FS := 8000
142 NTT_FC := 2000

```

```

143 NTT_SPS := 4
144 NTT_AMP := 2600
145 # Gray 符号の 2 ビット -> 振幅。00,01,10,11 -> -3,-1,+3,+1
146 QAM_GRAY := [0 - 3, 0 - 1, 3, 1]
147
148 def qam_level(two) { return QAM_GRAY[two % 4] }
149
150 def qam_bits(level) {
151     best := 0
152     bd := 99999
153     i := 0
154     while (i < 4) {
155         d := abs(QAM_GRAY[i] - level)
156         if (d < bd) { bd := d; best := i }
157         i := i + 1
158     }
159     return best
160 }
161
162 # バイト列 -> 音声帯サンプル列
163 def ntt_modulate(bytes) {
164     bits := bytes_to_bits(bytes)
165     while (len(bits) % 4 != 0) { bits <- 0 }
166     out := []
167     i := 0
168     while (i < len(bits)) {
169         I := qam_level(bits[i] * 2 + bits[i + 1])
170         Q := qam_level(bits[i + 2] * 2 + bits[i + 3])
171         # n=0..3 で cos(πn/2)={1,0,-1,0}, sin(πn/2)={0,1,0,-1}
172         out <- NTT_AMP * I
173         out <- 0 - NTT_AMP * Q
174         out <- 0 - NTT_AMP * I
175         out <- NTT_AMP * Q
176         i := i + 4
177     }
178     return out
179 }
180
181 # 音声帯サンプル列 -> バイト列 (相関復調 + 硬判定)
182 def ntt_demodulate(samples, bitlen) {
183     nsym := u_trunc(len(samples) / NTT_SPS)
184     bits := []
185     k := 0
186     while (k < nsym) {
187         s0 := samples[k * 4]
188         s1 := samples[k * 4 + 1]
189         s2 := samples[k * 4 + 2]
190         s3 := samples[k * 4 + 3]
191         I := 0.5 * (s0 - s2) / NTT_AMP
192         Q := 0 - 0.5 * (s1 - s3) / NTT_AMP
193         bi := qam_bits(I)
194         bq := qam_bits(Q)
195         bits <- u_trunc(bi / 2)
196         bits <- (bi % 2)
197         bits <- u_trunc(bq / 2)
198         bits <- (bq % 2)
199         k := k + 1
200     }
201     trimmed := []
202     i := 0
203     while (i < bitlen && i < len(bits)) { trimmed <- bits[i]; i := i + 1 }
204     return bits_to_bytes(trimmed)
205 }
206
207 # 画面表示・雑音評価のためのコンステレーション点
208 def ntt_constellation(samples, limit) {
209     nsym := u_min(u_trunc(len(samples) / NTT_SPS), limit)
210     pts := []
211     k := 0
212     while (k < nsym) {
213         s0 := samples[k * 4]
214         s1 := samples[k * 4 + 1]
215         s2 := samples[k * 4 + 2]
216         s3 := samples[k * 4 + 3]

```

```

217     pts <- [0.5 * (s0 - s2) / NTT_AMP, 0 - 0.5 * (s1 - s3) / NTT_AMP]
218     k := k + 1
219 }
220 return pts
221 }
222
223 # ---- G.711 μ-law (ひかり電話のコーデック) -----
224 MULAW_BIAS := 132
225 MULAW_CLIP := 32635
226
227 def mulaw_encode(sample) {
228     sign := 0
229     s := sample
230     if (s < 0) { s := 0 - s; sign := 128 }
231     if (s > MULAW_CLIP) { s := MULAW_CLIP }
232     s := s + MULAW_BIAS
233     exponent := 7
234     mask := 16384
235     while (u_and(s, mask) == 0 && exponent > 0) {
236         exponent := exponent - 1
237         mask := u_trunc(mask / 2)
238     }
239     mantissa := u_and(u_trunc(s / POW2[exponent + 3]), 15)
240     return 255 - u_or(u_or(sign, exponent * 16), mantissa)
241 }
242
243 def mulaw_decode(ulaw) {
244     u := 255 - ulaw
245     sign := u_and(u, 128)
246     exponent := u_and(u_trunc(u / 16), 7)
247     mantissa := u_and(u, 15)
248     s := (mantissa * 8 + MULAW_BIAS) * POW2[exponent]
249     s := s - MULAW_BIAS
250     if (sign != 0) { return 0 - s }
251     return s
252 }
253
254 # ---- RTP (ひかり電話が音声を選ぶ形式) -----
255 RTP_PT_PCMU := 0
256 RTP_PTIME := 160 # 20 ms @ 8 kHz
257
258 # -> パケットの並び。各要素は [seq, ts, header, payload]
259 def rtp_packetize(ulaw, ssrc, seq0, ts0) {
260     pkts := []
261     i := 0
262     seq := seq0
263     ts := ts0
264     while (i < len(ulaw)) {
265         pay := []
266         k := 0
267         while (k < RTP_PTIME && i + k < len(ulaw)) { pay <- ulaw[i + k]; k := k + 1 }
268         hdr := [128, RTP_PT_PCMU, u_trunc(seq / 256) % 256, seq % 256]
269         j := 3
270         while (j >= 0) { hdr <- (u_trunc(ts / POW2[8 * j]) % 256); j := j - 1 }
271         j := 3
272         while (j >= 0) { hdr <- (u_trunc(ssrc / POW2[8 * j]) % 256); j := j - 1 }
273         pkts <- [seq, ts, hdr, pay]
274         seq := (seq + 1) % 65536
275         ts := (ts + len(pay)) % TW032
276         i := i + RTP_PTIME
277     }
278     return pkts
279 }
280
281 # 順序が乱れていても seq で並べ直してから連結する
282 def rtp_depaketize(pkts) {
283     n := len(pkts)
284     order := []
285     i := 0
286     while (i < n) { order <- i; i := i + 1 }
287     i := 1
288     while (i < n) {
289         j := i
290         while (j > 0 && pkts[order[j - 1]][0] > pkts[order[j]][0]) {

```

```

291         t := order[j - 1]
292         order[j - 1] = order[j]
293         order[j] = t
294         j := j - 1
295     }
296     i := i + 1
297 }
298 out := []
299 i := 0
300 while (i < n) {
301     pay := pkts[order[i]][3]
302     k := 0
303     while (k < len(pay)) { out <- pay[k]; k := k + 1 }
304     i := i + 1
305 }
306 return out
307 }
308
309 # ----- ベアラ -----
310 # バイト列 -> 回線上の信号
311 # [kind, line, bitlen, データ]。kind は "digital" / "voiceband" / "rtp"
312 def ntt_bearer_send(bytes, line, ssrc) {
313     if (line == "isdn") {
314         oct := [126]
315         i := 0
316         while (i < len(bytes)) { oct <- bytes[i]; i := i + 1 }
317         oct <- 126
318         return ["digital", line, len(bytes) * 8, oct]
319     }
320     vb := ntt_modulate(bytes)
321     if (line == "metal") { return ["voiceband", line, len(bytes) * 8, vb] }
322     ulaw := []
323     i := 0
324     while (i < len(vb)) { ulaw <- mulaw_encode(vb[i]); i := i + 1 }
325     return ["rtp", line, len(bytes) * 8, rtp_packetize(ulaw, ssrc, 1000, 0)]
326 }
327
328 # 回線上の信号 -> バイト列
329 def ntt_bearer_recv(sig) {
330     kind := sig[0]
331     if (kind == "digital") {
332         oct := sig[3]
333         out := []
334         i := 1
335         while (i < len(oct) - 1) { out <- oct[i]; i := i + 1 }
336         return out
337     }
338     if (kind == "rtp") {
339         ulaw := rtp_depacketize(sig[3])
340         vb := []
341         i := 0
342         while (i < len(ulaw)) { vb <- mulaw_decode(ulaw[i]); i := i + 1 }
343         return ntt_demodulate(vb, sig[2])
344     }
345     return ntt_demodulate(sig[3], sig[2])
346 }
347
348 # ----- 呼制御 (発呼 -> 接続 -> ベアラ確立) -----
349 def ntt_call(from_url, to_url, line) {
350     kind := "geo"
351     if (line == "hikari") { kind := "ip" }
352     a := ntt_map(from_url, kind)
353     b := ntt_map(to_url, kind)
354     st := []
355     st <- ["on-hook", "待機"]
356     st <- ["off-hook", "発呼 (" + ntt_format(a[0]) + ")"]
357     st <- ["dial", "選択信号 " + ntt_format(b[0]) + " (PB 信号)"]
358     st <- ["ringing", "呼出中 - 相手交換機へ IAM"]
359     st <- ["connected", ntt_line_name(line) + " 接続 ・ " + ntt_line_bps(line) + " bps"]
360     st <- ["data", "ベアラ確立"]
361     return [a, b, st]
362 }

```

06 bada/06_plc.bada

L0 Panasonic HD-PLC — 多重経路伝達関数・適応ビットローディング・ウェーブレット OFDM・CRC-32/24・IEEE 1901 フレーム・CSMA/CA

518 行

```

1 # =====
2 # 06_plc.bada -- BADA ULTRA NETWORK / L0 Panasonic HD-PLC
3 #
4 # 家の電力線（コンセント）を物理層にして PC をインターネットへつなぐ層。
5 #
6 # ・帯域 2-28 MHz を 512 本のサブキャリアに分ける ( $\Delta f \approx 50.78$  kHz)
7 # ・変調は QAM ではなく PAM + ウェーブレット変換 (DWT)。
8 #   ガードインターバルが要らないのが HD-PLC 方式の利点。ここでは
9 #   正規化 Haar の多重解像度フィルタバンクがその役を務める
10 #   (解析と合成は厳密な逆変換の対)
11 # ・電力線の伝達関数は Zimmermann-Dostert の多重経路モデル
12 #    $H(f) = \sum g_i \cdot \exp(-(a_0 + a_1 f^k) d_i) \cdot \exp(-j2\pi f d_i / v)$ 
13 # ・分岐点のインピーダンス不整合損は周波数とともに増える
14 # ・雑音は色付き背景雑音  $N(f)$  [dBm/Hz] =  $a + b \cdot f_{\text{MHz}}^c$ 
15 # ・SNR から適応ビットローディング  $b = \text{floor}(\log_2(1 + \text{SNR}/\Gamma))$ 
16 # ・アマチュア無線帯にはノッチを入れて送信しない
17 # ・IEEE 1901 のフレーム (プリアンプル + FC/CRC-24 + PB/CRC-32)
18 # ・CSMA/CA の優先度解決 (PRS0 / PRS1 + 競合窓バックオフ)
19 # =====
20
21 PLC_FLO      := 2000000
22 PLC_FHI      := 28000000
23 PLC_NCARR     := 512
24 PLC_TXPSD    := 0 - 50      # 送信電力密度 dBm/Hz (規制値相当)
25 PLC_GAP_DB    := 9.8        # SNR ギャップ  $\Gamma$  (BER  $1e-7$  相当)
26 PLC_MAXBITS  := 5           # 32-PAM まで
27 PLC_VP       := 150000000   # 電力線の位相速度 m/s
28
29 PLC_DF       := (PLC_FHI - PLC_FLO) / PLC_NCARR
30 PLC_TSYM     := 1.0 / PLC_DF
31
32 # アマチュア無線帯 (MHz)。HD-PLC はここを送信しない。
33 HAM_LO := [1.8, 3.5, 7.0, 10.1, 14.0, 18.068, 21.0, 24.89, 28.0]
34 HAM_HI := [2.0, 3.8, 7.2, 10.15, 14.35, 18.168, 21.45, 24.99, 29.7]
35
36 def in_ham_band(f_hz) {
37   f := f_hz / 1000000
38   i := 0
39   while (i < len(HAM_LO)) {
40     if (f >= HAM_LO[i] && f <= HAM_HI[i]) { return true }
41     i := i + 1
42   }
43   return false
44 }
45
46 # ---- 屋内配線のトポロジ -----
47 # コンセントは [id, 表示名, 分電盤からの距離 m, 分岐数, 相]
48 OUTLETS := [
49   ["living", "居間 コンセント",          12, 3, "L1"],
50   ["study", "書斎 コンセント (PC)",       21, 4, "L1"],
51   ["kitchen", "台所 コンセント",          17, 5, "L2"],
52   ["bedroom", "寝室 コンセント",          28, 4, "L2"],
53   ["router", "ONU 脇 コンセント (親機)",   5, 2, "L1"]
54 ]
55
56 def outlet_of(id) {
57   i := 0
58   while (i < len(OUTLETS)) {
59     if (OUTLETS[i][0] == id) { return OUTLETS[i] }
60     i := i + 1
61   }
62   return OUTLETS[0]
63 }
64
65 # 多重経路。経路 i は反射係数  $g_i$  と行程長  $d_i$  を持つ。-> [[g, d], ...]
66 def plc_paths(a, b) {
67   oa := outlet_of(a)

```

```

68     ob := outlet_of(b)
69     rnd := rng_new("plc:" + a + ">" + b)
70     base := oa[2] + ob[2]
71     nb := u_max(oa[3], ob[3])
72     npath := 3 + nb - 1
73     if (npath > 6) { npath := 6 }
74     paths := []
75     i := 0
76     while (i < npath) {
77         g := 1.0
78         d := base
79         if (i > 0) {
80             # 反射波は分岐で減衰し、行程が伸びる
81             g := (0.75 - 0.12 * i) * (0.6 + 0.4 * rng_next(rnd))
82             d := base + i * (6 + 14 * rng_next(rnd))
83         }
84         if (i % 2 == 1) { g := 0 - g }      # 反射で位相反転
85         paths <- [g, d]
86         i := i + 1
87     }
88     # 異相 (L1 <-> L2) はカプラ越えで一段落ちる
89     if (oa[4] != ob[4]) {
90         i := 0
91         while (i < len(paths)) { paths[i][0] = paths[i][0] * 0.45; i := i + 1 }
92     }
93     return paths
94 }
95
96 # |H(f)| - 減衰 exp(-a1 f d) と経路差による楕形フェージング
97 def plc_transfer(paths, f_hz) {
98     a1 := 0.00000000078
99     re := 0.0
100     im := 0.0
101     i := 0
102     while (i < len(paths)) {
103         g := paths[i][0]
104         d := paths[i][1]
105         att := exp(0 - a1 * f_hz * d)
106         ph := 0 - TWO_PI * f_hz * d / PLC_VP
107         re := re + g * att * u_cos(ph)
108         im := im + g * att * u_sin(ph)
109         i := i + 1
110     }
111     return sqrt(re * re + im * im)
112 }
113
114 # 分岐のインピーダンス不整合損 [dB]。無効成分なので周波数とともに増える。
115 def plc_coupling_db(a, b, f_hz) {
116     oa := outlet_of(a)
117     ob := outlet_of(b)
118     j := (oa[3] - 1) + (ob[3] - 1)
119     db := 3.0 * j + 5.0
120     if (oa[4] != ob[4]) { db := db + 9.0 }
121     t := (f_hz - PLC_FLO) / (PLC_FHI - PLC_FLO)
122     return db + (14 + 2.6 * j) * t
123 }
124
125 # 色付き背景雑音 [dBm/Hz]
126 def plc_noise_db(f_hz) {
127     fm := f_hz / 1000000
128     return 0 - 145 + 53.23 * exp(0 - 0.337 * log(fm))
129 }
130
131 # コンセント間の回線特性 -> [freqs, snrDb, paths]
132 def plc_channel(a, b) {
133     paths := plc_paths(a, b)
134     freqs := []
135     snrs := []
136     n := 0
137     while (n < PLC_NCARR) {
138         f := PLC_FLO + (n + 0.5) * PLC_DF
139         h := plc_transfer(paths, f)
140         if (h < 0.000000000001) { h := 0.000000000001 }
141         hdb := 20 * log(h) / LN10 - plc_coupling_db(a, b, f)

```

```

142     freqs <- f
143     snrs <- (PLC_TXPSD + hdb - plc_noise_db(f))
144     n := n + 1
145 }
146 return [freqs, snrs, paths]
147 }
148
149 # 適応ビットローディング b = floor(log2(1 + SNR/Γ))、ノッチは 0
150 # -> [bits, bitsPerSymbol, active, notched, phyBps, snrs]
151 def plc_tonemap(ch) {
152     gap := db_to_lin(PLC_GAP_DB)
153     freqs := ch[0]
154     snrs := ch[1]
155     bits := []
156     total := 0
157     active := 0
158     notched := 0
159     n := 0
160     while (n < PLC_NCARR) {
161         if (in_ham_band(freqs[n])) {
162             bits <- 0
163             notched := notched + 1
164         } else {
165             lin := db_to_lin(snrs[n])
166             bq := u_trunc(u_log2(1 + lin / gap))
167             if (bq < 1) { bq := 0 }
168             if (bq > PLC_MAXBITS) { bq := PLC_MAXBITS }
169             bits <- bq
170             if (bq > 0) { active := active + 1; total := total + bq }
171         }
172         n := n + 1
173     }
174     return [bits, total, active, notched, total / PLC_TSYM, snrs]
175 }
176
177 # ---- Gray 符号つき PAM -----
178 # PAM は 5 ビット (32-PAM) までなので、0..31 の対応表を先に作る。
179 # OFDM の内側では 1 シンボルあたり 512 回引かれるので、ここを
180 # 逐次 XOR のままにすると全体の costs の大半を食う。
181 GRAY_ENC := []
182 i := 0
183 while (i < 32) { GRAY_ENC <- u_xor(i, u_trunc(i / 2)); i := i + 1 }
184 GRAY_DEC := []
185 i := 0
186 while (i < 32) { GRAY_DEC <- 0; i := i + 1 }
187 i := 0
188 while (i < 32) { GRAY_DEC[GRAY_ENC[i]] = i; i := i + 1 }
189
190 def gray_encode(v) { return GRAY_ENC[v] }
191 def gray_decode(g) { return GRAY_DEC[g] }
192
193 # b ビット -> 2^b-PAM の振幅 (... , -3, -1, +1, +3, ...)
194 def pam_level(idx, b) { return 2 * idx - (POW2[b] - 1) }
195
196 def pam_index(level, b) {
197     M := POW2[b]
198     i := u_fix((level + (M - 1) + 1) / 2)
199     if (i < 0) { i := 0 }
200     if (i > M - 1) { i := M - 1 }
201     return i
202 }
203
204 # ---- ウェーブレット変換 (正規化 Haar、厳密な逆変換の対) -----
205 SQ1_2 := 0.7071067811865476
206
207 # 時間波形 -> ウェーブレット係数 (サブバンド = サブキャリア)
208 def dwt_analyze(x) {
209     a := []
210     i := 0
211     while (i < len(x)) { a <- x[i]; i := i + 1 }
212     n := len(a)
213     while (n > 1) {
214         half := u_trunc(n / 2)
215         out := []

```

```

216     i := 0
217     while (i < half) { out <- (a[2 * i] + a[2 * i + 1]) * SQ1_2; i := i + 1 }
218     i := 0
219     while (i < half) { out <- (a[2 * i] - a[2 * i + 1]) * SQ1_2; i := i + 1 }
220     i := 0
221     while (i < n) { a[i] = out[i]; i := i + 1 }
222     n := half
223 }
224 return a
225 }
226
227 # ウェーブレット係数 -> 時間波形 (dwt_analyze の逆)
228 def dwt_synthesize(c) {
229     a := []
230     i := 0
231     while (i < len(c)) { a <- c[i]; i := i + 1 }
232     N := len(a)
233     n := 1
234     while (n < N) {
235         half := n
236         n := n * 2
237         out := []
238         i := 0
239         while (i < n) { out <- 0; i := i + 1 }
240         i := 0
241         while (i < half) {
242             out[2 * i] = (a[i] + a[half + i]) * SQ1_2
243             out[2 * i + 1] = (a[i] - a[half + i]) * SQ1_2
244             i := i + 1
245         }
246         i := 0
247         while (i < n) { a[i] = out[i]; i := i + 1 }
248     }
249     return a
250 }
251
252 # ---- ウェーブレット OFDM -----
253 # -> [samples, nSymbols, bitLen]
254 def plc_modulate(bytes, tm) {
255     tone := tm[0]
256     per := tm[1]
257     bits := bytes_to_bits(bytes)
258     reallen := len(bits)
259     nsym := u_trunc((reallen + per - 1) / per)
260     pad := nsym * per - reallen
261     i := 0
262     while (i < pad) { bits <- 0; i := i + 1 }
263
264     samples := []
265     p := 0
266     s := 0
267     while (s < nsym) {
268         coeffs := []
269         n := 0
270         while (n < PLC_NCARR) { coeffs <- 0; n := n + 1 }
271         n := 0
272         while (n < PLC_NCARR) {
273             bq := tone[n]
274             if (bq > 0) {
275                 v := 0
276                 k := 0
277                 while (k < bq) { v := v * 2 + bits[p]; p := p + 1; k := k + 1 }
278                 coeffs[n] = pam_level(gray_decode(v), bq)
279             }
280             n := n + 1
281         }
282         wave := dwt_synthesize(coeffs)
283         i := 0
284         while (i < PLC_NCARR) { samples <- wave[i]; i := i + 1 }
285         s := s + 1
286     }
287     return [samples, nsym, reallen]
288 }
289

```

```

290 def plc_demodulate(samples, tm, bitlen) {
291     tone := tm[0]
292     nsym := u_trunc(len(samples) / PLC_NCARR)
293     bits := []
294     s := 0
295     while (s < nsym) {
296         chunk := []
297         i := 0
298         while (i < PLC_NCARR) { chunk <- samples[s * PLC_NCARR + i]; i := i + 1 }
299         coeffs := dwt_analyze(chunk)
300         n := 0
301         while (n < PLC_NCARR) {
302             bq := tone[n]
303             if (bq > 0) {
304                 g := gray_encode(pam_index(coeffs[n], bq))
305                 k := bq - 1
306                 while (k >= 0) {
307                     pw := POW2[k]
308                     if (g >= pw) { bits <- 1; g := g - pw } else { bits <- 0 }
309                     k := k - 1
310                 }
311             }
312             n := n + 1
313         }
314         s := s + 1
315     }
316     trimmed := []
317     i := 0
318     while (i < bitlen && i < len(bits)) { trimmed <- bits[i]; i := i + 1 }
319     return bits_to_bytes(trimmed)
320 }
321
322 # 実回線を通す: サブバンドごとに雑音を載せ、受信側で等化する。
323 # ビットローディングの余裕が足りないサブキャリアだけが誤る。
324 def plc_through_channel(samples, tm, margin_db, seed) {
325     rnd := rng_new("awgn:" + seed)
326     tone := tm[0]
327     snrs := tm[5]
328     nsym := u_trunc(len(samples) / PLC_NCARR)
329     out := []
330     s := 0
331     while (s < nsym) {
332         chunk := []
333         i := 0
334         while (i < PLC_NCARR) { chunk <- samples[s * PLC_NCARR + i]; i := i + 1 }
335         coeffs := dwt_analyze(chunk)
336         n := 0
337         while (n < PLC_NCARR) {
338             if (tone[n] == 0) { coeffs[n] = 0 }
339             else {
340                 snr := db_to_lin(snrs[n] - margin_db)
341                 sigma := sqrt((POW2[2 * tone[n]] - 1) / 3 / snr)
342                 u1 := rng_next(rnd)
343                 if (u1 < 0.000000000001) { u1 := 0.000000000001 }
344                 u2 := rng_next(rnd)
345                 gauss := sqrt(0 - 2 * log(u1)) * u_cos(TWO_PI * u2)
346                 coeffs[n] = coeffs[n] + sigma * gauss
347             }
348             n := n + 1
349         }
350         wave := dwt_synthesize(coeffs)
351         i := 0
352         while (i < PLC_NCARR) { out <- wave[i]; i := i + 1 }
353         s := s + 1
354     }
355     return out
356 }
357
358 # ---- CRC (IEEE 1901 のフレーム検査) -----
359 CRC32_TAB := []
360 n := 0
361 while (n < 256) {
362     c := n
363     k := 0

```

```

364     while (k < 8) {
365         if (c % 2 == 1) { c := u_xor(3988292384, u_trunc(c / 2)) }
366         else { c := u_trunc(c / 2) }
367         k := k + 1
368     }
369     CRC32_TAB <- c
370     n := n + 1
371 }
372
373 def crc32(bytes) {
374     c := MASK32
375     i := 0
376     while (i < len(bytes)) {
377         c := u_xor(CRC32_TAB[u_xor8(c % 256, bytes[i])], u_trunc(c / 256))
378         i := i + 1
379     }
380     return MASK32 - c
381 }
382
383 # CRC-24 (多項式 0x5D6DCB) - フレーム制御の検査
384 def crc24(bytes) {
385     c := 11994830
386     i := 0
387     while (i < len(bytes)) {
388         c := u_xor(c, bytes[i] * 65536)
389         k := 0
390         while (k < 8) {
391             c := c * 2
392             if (c >= 16777216) { c := u_xor(c, 6122955) }
393             c := c % 16777216
394             k := k + 1
395         }
396         i := i + 1
397     }
398     return c
399 }
400
401 # ---- フレーム (プリアンブル + FC + PB ブロック) -----
402 PLC_PB_BYTES := 520
403 PLC_PREAMBLE := [26, 207, 252, 29, 26, 207, 252, 29]
404
405 def plc_frame(payload, src, dst, cap) {
406     n := len(payload)
407     nblk := u_trunc((n + PLC_PB_BYTES - 1) / PLC_PB_BYTES)
408     if (nblk < 1) { nblk := 1 }
409     # フレーム制御: 種別 / 優先度 / 送受信 ID / ブロック数 / ペイロード長
410     fc := [1, cap % 4,
411           u_trunc(src / 256) % 256, src % 256,
412           u_trunc(dst / 256) % 256, dst % 256,
413           u_trunc(nblk / 256) % 256, nblk % 256]
414     k := 3
415     while (k >= 0) { fc <- (u_trunc(n / POW2[8 * k]) % 256); k := k - 1 }
416     fcc := crc24(fc)
417
418     out := []
419     i := 0
420     while (i < len(PLC_PREAMBLE)) { out <- PLC_PREAMBLE[i]; i := i + 1 }
421     i := 0
422     while (i < len(fc)) { out <- fc[i]; i := i + 1 }
423     out <- u_trunc(fcc / 65536) % 256
424     out <- u_trunc(fcc / 256) % 256
425     out <- fcc % 256
426
427     b := 0
428     while (b < nblk) {
429         pb := []
430         i := 0
431         while (i < PLC_PB_BYTES) {
432             idx := b * PLC_PB_BYTES + i
433             if (idx < n) { pb <- payload[idx] } else { pb <- 0 }
434             i := i + 1
435         }
436         c := crc32(pb)
437         i := 0

```

```

438     while (i < PLC_PB_BYTES) { out <- pb[i]; i := i + 1 }
439     k := 3
440     while (k >= 0) { out <- (u_trunc(c / POW2[8 * k]) % 256); k := k - 1 }
441     b := b + 1
442 }
443 return out
444 }
445
446 # -> [status, payload, 理由] status 200 / 503
447 def plc_deframe(bytes) {
448     i := 0
449     while (i < len(PLC_PREAMBLE)) {
450         if (i >= len(bytes)) { return [503, [], "no-preamble"] }
451         if (bytes[i] != PLC_PREAMBLE[i]) { return [503, [], "no-preamble"] }
452         i := i + 1
453     }
454     p := len(PLC_PREAMBLE)
455     if (len(bytes) < p + 15) { return [503, [], "truncated"] }
456     fc := []
457     i := 0
458     while (i < 12) { fc <- bytes[p + i]; i := i + 1 }
459     got := bytes[p + 12] * 65536 + bytes[p + 13] * 256 + bytes[p + 14]
460     if (got != crc24(fc)) { return [503, [], "fc-crc"] }
461     nblk := fc[6] * 256 + fc[7]
462     paylen := fc[8] * 16777216 + fc[9] * 65536 + fc[10] * 256 + fc[11]
463     p := p + 15
464     acc := []
465     b := 0
466     while (b < nblk) {
467         if (len(bytes) < p + PLC_PB_BYTES + 4) { return [503, [], "truncated"] }
468         pb := []
469         i := 0
470         while (i < PLC_PB_BYTES) { pb <- bytes[p + i]; i := i + 1 }
471         p := p + PLC_PB_BYTES
472         c := bytes[p] * 16777216 + bytes[p + 1] * 65536 + bytes[p + 2] * 256 + bytes[p + 3]
473         p := p + 4
474         if (c != crc32(pb)) { return [503, [], "pb-crc"] }
475         i := 0
476         while (i < PLC_PB_BYTES) { acc <- pb[i]; i := i + 1 }
477         b := b + 1
478     }
479     out := []
480     i := 0
481     while (i < paylen && i < len(acc)) { out <- acc[i]; i := i + 1 }
482     return [200, out, ""]
483 }
484
485 # ---- CSMA/CA (IEEE 1901 の優先度解決スロット) -----
486 # 局は [name, CAP]。CAP 3 が最優先。
487 # -> [maxCap, cw, 勝者, PRS0 の並び, PRS1 の並び, 抽選結果]
488 def plc_csma(stations, seed) {
489     rnd := rng_new("csma:" + seed)
490     prs0 := []
491     prs1 := []
492     maxcap := 0 - 1
493     i := 0
494     while (i < len(stations)) {
495         nm := stations[i][0]
496         cp := stations[i][1]
497         if (cp >= 2) { prs0 <- nm }
498         if (cp == 3) { prs1 <- nm }
499         if (cp == 1) { prs1 <- nm }
500         if (cp > maxcap) { maxcap := cp }
501         i := i + 1
502     }
503     cwtap := [7, 15, 15, 31]
504     cw := cwtap[u_min(3, u_max(0, maxcap))]
505     draws := []
506     winner := ""
507     bestb := 999999
508     i := 0
509     while (i < len(stations)) {
510         if (stations[i][1] == maxcap) {
511             b := u_trunc(rng_next(rnd) * (cw + 1))

```

```
512         draws <- [stations[i][0], b]
513         if (b < bestb) { bestb := b; winner := stations[i][0] }
514     }
515     i := i + 1
516 }
517 return [maxcap, cw, winner, prs0, prs1, draws]
518 }
```

07 bada/07_json.bada

JSON の解析と生成 — Webhook 本文を公式仕様どおりに読むための再帰下降パーサと生成器

321 行

```

1  # =====
2  # 07_json.bada -- BADA ULTRA NETWORK / JSON の解析と生成
3  #
4  # LINE Messaging API と Instagram Graph API の Webhook 本文は JSON で
5  # 届く。署名 (HMAC-SHA256) は「生の本文そのもの」に対して計算するので、
6  # 本文の文字列と、そこから取り出した値の両方が要る。
7  #
8  # Bada に連想配列は無いので、値はタグ付きの配列で表す:
9  #   ["obj", 鍵の並び, 値の並び]
10 #   ["arr", 要素の並び]
11 #   ["str", 文字列]   ["num", 数]   ["bool", 真偽]   ["null"]
12 # =====
13
14 def json_is(v, tag) {
15     if (len(v) == 0) { return false }
16     return v[0] == tag
17 }
18
19 def json_num(v) { if (json_is(v, "num")) { return v[1] } return 0 }
20 def json_str(v) { if (json_is(v, "str")) { return v[1] } return "" }
21 def json_bool(v) { if (json_is(v, "bool")) { return v[1] } return false }
22
23 # オブジェクトから鍵で引く。無ければ ["null"]
24 def json_get(v, key) {
25     if (!json_is(v, "obj")) { return ["null"] }
26     keys := v[1]
27     i := 0
28     while (i < len(keys)) {
29         if (keys[i] == key) { return v[2][i] }
30         i := i + 1
31     }
32     return ["null"]
33 }
34
35 def json_has(v, key) { return !json_is(json_get(v, key), "null") }
36
37 # 配列の i 番目。範囲外なら ["null"]
38 def json_at(v, i) {
39     if (!json_is(v, "arr")) { return ["null"] }
40     if (i < 0) { return ["null"] }
41     if (i >= len(v[1])) { return ["null"] }
42     return v[1][i]
43 }
44
45 def json_len(v) {
46     if (json_is(v, "arr")) { return len(v[1]) }
47     if (json_is(v, "obj")) { return len(v[1]) }
48     return 0
49 }
50
51 # "entry.0.messaging.0.message.text" のような経路で潜る
52 def json_path(v, path) {
53     parts := split(path, ".")
54     cur := v
55     i := 0
56     while (i < len(parts)) {
57         p := parts[i]
58         c0 := ord(p, 0)
59         if (c0 >= 48 && c0 <= 57) {
60             n := 0
61             k := 0
62             while (k < len(p)) { n := n * 10 + (ord(p, k) - 48); k := k + 1 }
63             cur := json_at(cur, n)
64         } else {
65             cur := json_get(cur, p)
66         }
67         i := i + 1
68     }

```

```

69     return cur
70 }
71
72 # ---- 解析 -----
73 # st は [text, pos] 。 pos を進めながら読む。
74 def js_peek(st) {
75     if (st[1] >= len(st[0])) { return 0 }
76     return ord(st[0], st[1])
77 }
78
79 def js_skip_ws(st) {
80     while (st[1] < len(st[0])) {
81         c := ord(st[0], st[1])
82         ws := false
83         if (c == 32) { ws := true }
84         if (c == 9) { ws := true }
85         if (c == 10) { ws := true }
86         if (c == 13) { ws := true }
87         if (ws) { st[1] = st[1] + 1 } else { return 0 }
88     }
89     return 0
90 }
91
92 def js_hexval(c) {
93     if (c >= 48 && c <= 57) { return c - 48 }
94     if (c >= 97 && c <= 102) { return c - 87 }
95     if (c >= 65 && c <= 70) { return c - 55 }
96     return 0
97 }
98
99 def js_parse_string(st) {
100     st[1] = st[1] + 1          # 開きの "
101     out := ""
102     while (st[1] < len(st[0])) {
103         c := ord(st[0], st[1])
104         if (c == 34) { st[1] = st[1] + 1; return out }
105         if (c == 92) {
106             st[1] = st[1] + 1
107             e := ord(st[0], st[1])
108             if (e == 110) { out := out + chr(10) }
109             else { if (e == 116) { out := out + chr(9) }
110                 else { if (e == 114) { out := out + chr(13) }
111                     else { if (e == 98) { out := out + chr(8) }
112                         else { if (e == 102) { out := out + chr(12) }
113                             else { if (e == 117) {
114                                 v := js_hexval(ord(st[0], st[1] + 1)) * 4096 +
115                                     js_hexval(ord(st[0], st[1] + 2)) * 256 +
116                                     js_hexval(ord(st[0], st[1] + 3)) * 16 +
117                                     js_hexval(ord(st[0], st[1] + 4))
118                                 out := out + chr(v)
119                                 st[1] = st[1] + 4
120                             }
121                             else { out := out + chr(e) } } } } } }
122             st[1] = st[1] + 1
123         } else {
124             out := out + chr(c)
125             st[1] = st[1] + 1
126         }
127     }
128     return out
129 }
130
131 # 数の字句を切り出す。JSON では数の切れ目は必ず区切り文字なので、
132 # 数字・小数点・指数・符号が続くがぎり読めばよい。
133 def js_parse_number(st) {
134     text := st[0]
135     n := len(text)
136     start := st[1]
137     e := start
138     scanning := true
139     while (scanning && e < n) {
140         c := ord(text, e)
141         good := false
142         if (c >= 48 && c <= 57) { good := true }

```

```

143     if (c == 46) { good := true }
144     if (c == 101) { good := true }
145     if (c == 69) { good := true }
146     if (c == 43) { good := true }
147     if (c == 45) { good := true }
148     if (good) { e := e + 1 } else { scanning := false }
149 }
150 st[1] = e
151 return str_to_num(substr(text, start, e - start))
152 }
153
154 # 十進文字列 -> 数 (符号・小数・指数に対応)
155 def str_to_num(txt) {
156     n := len(txt)
157     i := 0
158     sign := 1
159     if (i < n) {
160         c := ord(txt, i)
161         if (c == 45) { sign := 0 - 1; i := i + 1 }
162         else { if (c == 43) { i := i + 1 } }
163     }
164     # 整数部
165     ip := 0.0
166     reading := true
167     while (reading && i < n) {
168         c := ord(txt, i)
169         if (c >= 48 && c <= 57) { ip := ip * 10 + (c - 48); i := i + 1 }
170         else { reading := false }
171     }
172     # 小数部
173     frac := 0.0
174     if (i < n) {
175         if (ord(txt, i) == 46) {
176             i := i + 1
177             scale := 0.1
178             reading := true
179             while (reading && i < n) {
180                 c := ord(txt, i)
181                 if (c >= 48 && c <= 57) { frac := frac + (c - 48) * scale; scale := scale / 10; i := i + 1 }
182                 else { reading := false }
183             }
184         }
185     }
186     v := (ip + frac) * sign
187     # 指数部
188     if (i < n) {
189         c := ord(txt, i)
190         isexp := false
191         if (c == 101) { isexp := true }
192         if (c == 69) { isexp := true }
193         if (isexp) {
194             i := i + 1
195             esign := 1
196             if (i < n) {
197                 c2 := ord(txt, i)
198                 if (c2 == 45) { esign := 0 - 1; i := i + 1 }
199                 else { if (c2 == 43) { i := i + 1 } }
200             }
201             ev := 0
202             reading := true
203             while (reading && i < n) {
204                 c3 := ord(txt, i)
205                 if (c3 >= 48 && c3 <= 57) { ev := ev * 10 + (c3 - 48); i := i + 1 }
206                 else { reading := false }
207             }
208             v := v * ipow(10, ev * esign)
209         }
210     }
211     return v
212 }
213
214 def js_parse_value(st) {
215     js_skip_ws(st)
216     c := js_peek(st)

```

```

217     if (c == 123) { return js_parse_object(st) }
218     if (c == 91) { return js_parse_array(st) }
219     if (c == 34) { return ["str", js_parse_string(st)] }
220     if (c == 116) { st[1] = st[1] + 4; return ["bool", true] }
221     if (c == 102) { st[1] = st[1] + 5; return ["bool", false] }
222     if (c == 110) { st[1] = st[1] + 4; return ["null"] }
223     return ["num", js_parse_number(st)]
224 }
225
226 def js_parse_object(st) {
227     st[1] = st[1] + 1
228     keys := []
229     vals := []
230     js_skip_ws(st)
231     if (js_peek(st) == 125) { st[1] = st[1] + 1; return ["obj", keys, vals] }
232     while (st[1] < len(st[0])) {
233         js_skip_ws(st)
234         k := js_parse_string(st)
235         js_skip_ws(st)
236         st[1] = st[1] + 1          # :
237         v := js_parse_value(st)
238         keys <- k
239         vals <- v
240         js_skip_ws(st)
241         c := js_peek(st)
242         if (c == 44) { st[1] = st[1] + 1 }
243         else { st[1] = st[1] + 1; return ["obj", keys, vals] }
244     }
245     return ["obj", keys, vals]
246 }
247
248 def js_parse_array(st) {
249     st[1] = st[1] + 1
250     items := []
251     js_skip_ws(st)
252     if (js_peek(st) == 93) { st[1] = st[1] + 1; return ["arr", items] }
253     while (st[1] < len(st[0])) {
254         v := js_parse_value(st)
255         items <- v
256         js_skip_ws(st)
257         c := js_peek(st)
258         if (c == 44) { st[1] = st[1] + 1 }
259         else { st[1] = st[1] + 1; return ["arr", items] }
260     }
261     return ["arr", items]
262 }
263
264 def json_parse(text) {
265     st := [text, 0]
266     return js_parse_value(st)
267 }
268
269 # ---- 生成 -----
270 def json_escape(s) {
271     out := ""
272     i := 0
273     while (i < len(s)) {
274         c := ord(s, i)
275         if (c == 34) { out := out + chr(92) + chr(34) }
276         else { if (c == 92) { out := out + chr(92) + chr(92) }
277             else { if (c == 10) { out := out + chr(92) + "n" }
278                 else { if (c == 13) { out := out + chr(92) + "r" }
279                     else { if (c == 9) { out := out + chr(92) + "t" }
280                         else { out := out + chr(c) } } } } }
281         i := i + 1
282     }
283     return out
284 }
285
286 def json_q(s) { return chr(34) + json_escape(s) + chr(34) }
287
288 def json_stringify(v) {
289     if (json_is(v, "str")) { return json_q(v[1]) }
290     if (json_is(v, "num")) { return "" + v[1] }

```

```
291     if (json_is(v, "bool")) { if (v[1]) { return "true" } return "false" }
292     if (json_is(v, "null")) { return "null" }
293     if (json_is(v, "arr")) {
294         out := "["
295         i := 0
296         while (i < len(v[1])) {
297             if (i > 0) { out := out + "," }
298             out := out + json_stringify(v[1][i])
299             i := i + 1
300         }
301         return out + "]"
302     }
303     if (json_is(v, "obj")) {
304         out := "{"
305         i := 0
306         while (i < len(v[1])) {
307             if (i > 0) { out := out + "," }
308             out := out + json_q(v[1][i]) + ":" + json_stringify(v[2][i])
309             i := i + 1
310         }
311         return out + "}"
312     }
313     return "null"
314 }
315
316 # 組み立ての補助
317 def jobj(keys, vals) { return ["obj", keys, vals] }
318 def jarr(items)      { return ["arr", items] }
319 def jstr(s)          { return ["str", s] }
320 def jnum(n)          { return ["num", n] }
321 def jbool(b)         { return ["bool", b] }
```

08 bada/08_msgmux.bada

L4 LINE / Instagram 多重化 — 公式 API の Webhook 取り込みと署名検証・送信要求の組立・統合受信箱

308 行

```

1 # =====
2 # 08_msgmux.bada -- BADA ULTRA NETWORK / L4 メッセージ多重化
3 #
4 # LINE と Instagram のメッセージ機能をウルトラネットワークに取り込む層。
5 # 取り込みは各社が公開している「公式 API の仕様どおり」に行う。
6 #
7 # LINE      Messaging API の Webhook イベント
8 #           署名 X-Line-Signature = Base64(HMAC-SHA256(channelSecret, 生の本文))
9 #           送信 /v2/bot/message/reply ・ /v2/bot/message/push
10 # Instagram Instagram Graph API (Messenger Platform) の Webhook
11 #           署名 X-Hub-Signature-256 = "sha256=" + hex(HMAC-SHA256(appSecret, 生の本文))
12 #           送信 /<IG_ID>/messages
13 #
14 # 非公開 API の解析や認証の迂回は一切しない。実運用には各
15 # プラットフォームで発行したチャンネルシークレット / アクセストークンが
16 # 要る。それが無い場合は、公式仕様と同じ形の本文を組み立てて
17 # 署名経路まで本物を通すローカル模擬 (msg_simulate) で動かす。
18 #
19 # 統合メッセージの表現:
20 # ["msg", platform, chatId, userId, displayName, type, text, replyToken, ts]
21 # =====
22
23 LINE_REPLY_EP := "https://api.line.me/v2/bot/message/reply"
24 LINE_PUSH_EP  := "https://api.line.me/v2/bot/message/push"
25 IG_SEND_EP    := "https://graph.instagram.com/v21.0/{IG_ID}/messages"
26 LINE_DOCS     := "https://developers.line.biz/ja/reference/messaging-api/"
27 IG_DOCS       := "https://developers.facebook.com/docs/messenger-platform/instagram/"
28
29 # ---- 署名 ----
30 def line_signature(secret, raw_body) {
31     return base64_of_bytes(hmac_sha256(utf8_of_str(secret), utf8_of_str(raw_body)))
32 }
33
34 def line_verify(secret, raw_body, header) {
35     return const_time_eq_str(line_signature(secret, raw_body), header)
36 }
37
38 def ig_signature(secret, raw_body) {
39     return "sha256=" + hex_of_bytes(hmac_sha256(utf8_of_str(secret), utf8_of_str(raw_body)))
40 }
41
42 def ig_verify(secret, raw_body, header) {
43     return const_time_eq_str(ig_signature(secret, raw_body), header)
44 }
45
46 # ---- 統合メッセージ ----
47 def msg_new(platform, chat, user, name, type, text, reply, ts) {
48     return ["msg", platform, chat, user, name, type, text, reply, ts]
49 }
50
51 def msg_platform(m) { return m[1] }
52 def msg_chat(m)     { return m[2] }
53 def msg_user(m)     { return m[3] }
54 def msg_name(m)     { return m[4] }
55 def msg_type(m)     { return m[5] }
56 def msg_text(m)     { return m[6] }
57 def msg_reply(m)    { return m[7] }
58 def msg_ts(m)       { return m[8] }
59
60 # 統合メッセージが載る zone アドレス (これが NTT 番号へ写像される)
61 def msg_zone_url(m) {
62     chat := m[2]
63     if (chat == "") { chat := "self" }
64     return "zone://url.or.jp/msg/" + m[1] + "/" + chat
65 }
66
67 # 転送用の直列化 (封緘の対象になる)
68 def msg_serialize(m) {

```

```

69     return jsonify(jobj(["v","p","c","u","n","t","x","r","ts"],
70         [jnum(1), jstr(m[1]), jstr(m[2]), jstr(m[3]), jstr(m[4]),
71             jstr(m[5]), jstr(m[6]), jstr(m[7]), jnum(m[8])]))
72 }
73
74 def msg_deserialize(s) {
75     v := json_parse(s)
76     return msg_new(json_str(json_get(v, "p")), json_str(json_get(v, "c")),
77         json_str(json_get(v, "u")), json_str(json_get(v, "n")),
78         json_str(json_get(v, "t")), json_str(json_get(v, "x")),
79         json_str(json_get(v, "r")), json_num(json_get(v, "ts")))
80 }
81
82 # ---- LINE Messaging API の Webhook イベント -> 統合メッセージ -----
83 def msg_from_line_event(ev) {
84     if (json_str(json_get(ev, "type")) != "message") { return [] }
85     msgo := json_get(ev, "message")
86     if (json_is(msgo, "null")) { return [] }
87     src := json_get(ev, "source")
88     chat := json_str(json_get(src, "groupId"))
89     if (chat == "") { chat := json_str(json_get(src, "roomId")) }
90     if (chat == "") { chat := json_str(json_get(src, "userId")) }
91     mtype := json_str(json_get(msgo, "type"))
92     text := ""
93     if (mtype == "text") { text := json_str(json_get(msgo, "text")) }
94     else {
95         if (mtype == "sticker") {
96             text := "[スタンプ " + json_str(json_get(msgo, "packageId")) + "/" +
97                 json_str(json_get(msgo, "stickerId")) + "]"
98         } else { text := "[" + mtype + "]" }
99     }
100     name := json_str(json_path(ev, "__profile.displayName"))
101     if (name == "") { name := "LINE ユーザー" }
102     return [msg_new("line", chat, json_str(json_get(src, "userId")), name, mtype, text,
103         json_str(json_get(ev, "replyToken")), json_num(json_get(ev, "timestamp")))]
104 }
105
106 # -> [status, メッセージの並び, 署名が検証されたか]
107 def msg_ingest_line(raw_body, secret, sig_header) {
108     verified := true
109     if (secret != "") {
110         verified := line_verify(secret, raw_body, sig_header)
111         if (!verified) { return [401, [], false] }
112     }
113     body := json_parse(raw_body)
114     evs := json_get(body, "events")
115     out := []
116     i := 0
117     while (i < json_len(evs)) {
118         m := msg_from_line_event(json_at(evs, i))
119         if (len(m) == 1) { out <- m[0] }
120         i := i + 1
121     }
122     return [200, out, verified]
123 }
124
125 # ---- Instagram Graph API の Webhook -> 統合メッセージ -----
126 def msg_from_ig_messaging(mg, entry_id) {
127     msgo := json_get(mg, "message")
128     if (json_is(msgo, "null")) { return [] }
129     if (json_bool(json_get(msgo, "is_echo"))) { return [] }
130     sender := json_str(json_path(mg, "sender.id"))
131     text := json_str(json_get(msgo, "text"))
132     mtype := "text"
133     atts := json_get(msgo, "attachments")
134     if (json_len(atts) > 0) {
135         mtype := json_str(json_path(msgo, "attachments.0.type"))
136         if (mtype == "") { mtype := "image" }
137         if (text == "") { text := "[" + mtype + "]" }
138     }
139     name := json_str(json_path(mg, "__profile.username"))
140     if (name == "") { name := "IG:" + sender }
141     return [msg_new("instagram", sender, sender, name, mtype, text, "",
142         json_num(json_get(mg, "timestamp")))]

```

```

143 }
144
145 def msg_ingest_instagram(raw_body, secret, sig_header) {
146   verified := true
147   if (secret != "") {
148     verified := ig_verify(secret, raw_body, sig_header)
149     if (!verified) { return [401, [], false] }
150   }
151   body := json_parse(raw_body)
152   entries := json_get(body, "entry")
153   out := []
154   i := 0
155   while (i < json_len(entries)) {
156     entry := json_at(entries, i)
157     eid := json_str(json_get(entry, "id"))
158     ms := json_get(entry, "messaging")
159     k := 0
160     while (k < json_len(ms)) {
161       m := msg_from_ig_messaging(json_at(ms, k), eid)
162       if (len(m) == 1) { out <- m[0] }
163       k := k + 1
164     }
165     i := i + 1
166   }
167   return [200, out, verified]
168 }
169
170 # ---- 公式 API への送信要求 -----
171 # -> [endpoint, method, 認可ヘッダ, 本文 JSON]
172 def msg_to_line_reply(reply_token, text) {
173   body := jobj(["replyToken", "messages"],
174     [jstr(reply_token),
175       jarr([jobj(["type", "text"], [jstr("text"), jstr(text)]))])])
176   return [LINE_REPLY_EP, "POST", "Bearer {CHANNEL_ACCESS_TOKEN}", json_stringify(body)]
177 }
178
179 def msg_to_line_push(to, text) {
180   body := jobj(["to", "messages"],
181     [jstr(to),
182       jarr([jobj(["type", "text"], [jstr("text"), jstr(text)]))])])
183   return [LINE_PUSH_EP, "POST", "Bearer {CHANNEL_ACCESS_TOKEN}", json_stringify(body)]
184 }
185
186 def msg_to_instagram_send(ig_id, recipient, text) {
187   ep := IG_SEND_EP
188   cut := str_find(ep, "{IG_ID}")
189   if (cut >= 0) {
190     ep := substr(ep, 0, cut) + ig_id + substr(ep, cut + 7, len(ep) - cut - 7)
191   }
192   body := jobj(["recipient", "message"],
193     [jobj(["id"], [jstr(recipient)]),
194       jobj(["text"], [jstr(text)])])
195   return [ep, "POST", "Bearer {IG_ACCESS_TOKEN}", json_stringify(body)]
196 }
197
198 def msg_to_outbound(m, ig_id) {
199   if (m[1] == "line") {
200     if (m[7] != "") { return msg_to_line_reply(m[7], m[6]) }
201     return msg_to_line_push(m[2], m[6])
202   }
203   if (m[1] == "instagram") { return msg_to_instagram_send(ig_id, m[2], m[6]) }
204   return [msg_zone_url(m), "PUT", "", json_stringify(jstr(m[6]))]
205 }
206
207 # ---- ローカル模擬（資格情報が無くても署名経路を本物で通す） -----
208 SIM_LINE_ID := ["U4af4980629...", "U9c1f2d7b41...", "Ub2e88a0f53..."]
209 SIM_LINE_NAME := ["田中 さくら", "山口 まさあき", "研究会グループ"]
210 SIM_LINE_TEXT := ["電力線でつながった?", "コンセントから zone:// が見えてる",
211   "今日の実験ログ送るね", "NTT の番号写像うまくいった?",
212   "量子鍵の QBER は 0 だった"]
213 SIM_IG_ID := ["17841400000000001", "17841400000000002"]
214 SIM_IG_NAME := ["sakura.photo", "bada.lang"]
215 SIM_IG_TEXT := ["配線図の写真あげたよ", "ウルトラネットワークの図が綺麗",
216   "この投稿みて!", "DM ありがとう"]

```

```

217
218 # LINE の Webhook 本文を公式仕様の形で組み立てて、生の JSON 文字列を返す
219 def sim_line_body(n, seed) {
220     rnd := rng_new("simline:" + seed)
221     evs := []
222     i := 0
223     while (i < n) {
224         who := u_trunc(rng_next(rnd) * len(SIM_LINE_ID)) % len(SIM_LINE_ID)
225         wht := u_trunc(rng_next(rnd) * len(SIM_LINE_TEXT)) % len(SIM_LINE_TEXT)
226         ev := jobj(["type", "mode", "timestamp", "source", "webhookEventId",
227             "deliveryContext", "replyToken", "message", "__profile"],
228             [jstr("message"), jstr("active"), jnum(1700000000000 + i * 60000),
229             jobj(["type", "userId", [jstr("user"), jstr(SIM_LINE_ID[who])]),
230             jstr("01H" + (1000000 + i * 7)),
231             jobj(["isRedelivery"], [jbool(false)]),
232             jstr(hex_of_word32(mix32("rt:" + seed + ":" + i))),
233             jobj(["id", "type", "text",
234                 [jstr(" " + (10000000000000 + i)), jstr("text"), jstr(SIM_LINE_TEXT[wht])]),
235             jobj(["displayName"], [jstr(SIM_LINE_NAME[who])])])
236         evs <- ev
237         i := i + 1
238     }
239     return json_stringify(jobj(["destination", "events"],
240         [jstr("Ubada00000000000000000000000000000000"), jarr(evs)]))
241 }
242
243 # Instagram の Webhook 本文を公式仕様の形で組み立てる
244 def sim_instagram_body(n, seed) {
245     rnd := rng_new("simig:" + seed)
246     ms := []
247     i := 0
248     while (i < n) {
249         who := u_trunc(rng_next(rnd) * len(SIM_IG_ID)) % len(SIM_IG_ID)
250         wht := u_trunc(rng_next(rnd) * len(SIM_IG_TEXT)) % len(SIM_IG_TEXT)
251         mg := jobj(["sender", "recipient", "timestamp", "message", "__profile"],
252             [jobj(["id"], [jstr(SIM_IG_ID[who])]),
253             jobj(["id"], [jstr("17841499999999999")]),
254             jnum(1700000000000 + i * 60000),
255             jobj(["mid", "text",
256                 [jstr("aWc6" + hex_of_word32(mix32("mid:" + seed + ":" + i))),
257                 jstr(SIM_IG_TEXT[wht])]),
258             jobj(["username"], [jstr(SIM_IG_NAME[who])])])
259         ms <- mg
260         i := i + 1
261     }
262     entry := jobj(["id", "time", "messaging"],
263         [jstr("17841499999999999"), jnum(1700000000), jarr(ms)])
264     return json_stringify(jobj(["object", "entry"], [jstr("instagram"), jarr([entry])]))
265 }
266
267 # 署名まで含めた完全な模擬 Webhook -> [status, メッセージの並び, 検証, 生本文, 署名]
268 def msg_simulate(platform, n, secret, seed) {
269     if (platform == "line") {
270         raw := sim_line_body(n, seed)
271         sig := line_signature(secret, raw)
272         r := msg_ingest_line(raw, secret, sig)
273         return [r[0], r[1], r[2], raw, sig]
274     }
275     raw := sim_instagram_body(n, seed)
276     sig := ig_signature(secret, raw)
277     r := msg_ingest_instagram(raw, secret, sig)
278     return [r[0], r[1], r[2], raw, sig]
279 }
280
281 # ---- 統合受信箱 -----
282 # [キーの並び, スレッドの並び]。各スレッドは [platform, chatId, 表示名, メッセージの並び]
283 def inbox_new() { return [[], []] }
284
285 def inbox_add(box, m) {
286     key := m[1] + ":" + m[2]
287     i := 0
288     while (i < len(box[0])) {
289         if (box[0][i] == key) {
290             box[1][i][3] <- m

```

```
291         box[1][i][2] = m[4]
292         return i
293     }
294     i := i + 1
295 }
296 box[0] <- key
297 box[1] <- [m[1], m[2], m[4], [m]]
298 return len(box[0]) - 1
299 }
300
301 def inbox_count(box) {
302     n := 0
303     i := 0
304     while (i < len(box[1])) { n := n + len(box[1][i][3]); i := i + 1 }
305     return n
306 }
307
308 def inbox_threads(box) { return box[1] }
```

09 bada/09_streams.bada

AT&T ベル研究所式 STREAMS — stream head と driver の間に積む wput / rput の対、全 5 層の往復

364 行

```

1 # =====
2 # 09_streams.bada -- BADA ULTRA NETWORK / AT&T ベル研究所式 STREAMS
3 #
4 # 層の組み方は、ベル研究所が UNIX の装置入出力に使った STREAMS
5 # (Dennis Ritchie の Streams I/O) と同じにしてある。
6 #
7 #      stream head   アプリケーション (統合受信箱)
8 #      | wput        rput |
9 #      +--- msgmux ---+ L4  LINE / Instagram 多重化
10 #      +--- jones ---+ L3  Jones 多項式量子暗号
11 #      +--- zone ---+ L2  zone:// P2P リング DHT
12 #      +--- ntt ---+ L1  NTT 電話回線への写像
13 #      | put         srv |
14 #      driver plc    L0  Panasonic HD-PLC (家のコンセント)
15 #
16 # 各モジュールは wput (下り) と rput (上り) の対だけを持つ。上位は
17 # 下位を知らず、下位は上位を知らない。下りで各層がしたことを、
18 # 上りで対になる層がちょうど元に戻す。
19 #
20 # Bada は関数を配列に入れて呼べないので、モジュールの呼び分けは
21 # 名前による明示的なディスパッチで行う (stream_wput / stream_rput)。
22 #
23 # メッセージブロック: ["blk", 種別, 中身, status, 理由]
24 # 種別は "M_DATA" / "M_PROTO" / "M_CTL"
25 # =====
26
27 def blk_new(kind, payload)      { return ["blk", kind, payload, 200, ""] }
28 def blk_ctl(status, reason)    { return ["blk", "M_CTL", [], status, reason] }
29 def blk_kind(b)                { return b[1] }
30 def blk_payload(b)             { return b[2] }
31 def blk_status(b)              { return b[3] }
32 def blk_is_ctl(b)              { return b[1] == "M_CTL" }
33
34 # ---- 文脈 (層をまたいで共有する状態) -----
35 # 添字で引く。Bada に連想配列は無いので定数で名前をつける。
36 CTX_NET      := 0    # zone のピア環
37 CTX_TONEMAP  := 1    # HD-PLC のトーンマップ
38 CTX_LINE     := 2    # 回線種別 metal / isdn / hikari
39 CTX_CAP      := 3    # CSMA/CA の優先度
40 CTX_MARGIN   := 4    # 電力線の劣化 dB
41 CTX_TAPPED   := 5    # 量子路を盗聴するか
42 CTX_TAMPER   := 6    # 物理層で回線を改竄するか
43 CTX_TAMPERZ  := 7    # 悪意あるピアがレコードを書き換えるか
44 CTX_SEED     := 8
45 CTX_QKDOK    := 9
46 CTX_PHY      := 10   # 最後の物理層の計量
47 CTX_NTT      := 11   # 最後の番号写像
48 CTX_ZONE     := 12   # 最後の DHT 解決
49 CTX_CRYPT0   := 13   # 最後の封緘
50 CTX_TRACE    := 14   # ブロックの流れ
51 CTX_FROM     := 15
52 CTX_TO       := 16
53
54 def ctx_new(from_outlet, to_outlet, line) {
55     ch := plc_channel(from_outlet, to_outlet)
56     tm := plc_tonemap(ch)
57     return [zone_boot(), tm, line, 2, 0, false, false, false, "0", true,
58             [], [], [], [], [], from_outlet, to_outlet]
59 }
60
61 def ctx_relink(ctx, from_outlet, to_outlet, line) {
62     ctx[CTX_FROM] = from_outlet
63     ctx[CTX_TO] = to_outlet
64     ctx[CTX_LINE] = line
65     ctx[CTX_TONEMAP] = plc_tonemap(plc_channel(from_outlet, to_outlet))
66     return ctx
67 }
68

```

```

69 def trace_add(ctx, dir, mod, note, b) {
70   size := 0
71   if (len(b) > 2) {
72     pl := b[2]
73     if (len(pl) > 0) { size := len(pl) }
74   }
75   ctx[CTX_TRACE] <- [dir, mod, note, b[1], size]
76   return 0
77 }
78
79 # =====
80 # L4 msgmux -- 統合メッセージ <-> [url, 本文]
81 # =====
82 def msgmux_wput(b, ctx) {
83   m := blk_payload(b)
84   return blk_new("M_DATA", [msg_zone_url(m), msg_serialize(m)])
85 }
86
87 def msgmux_rput(b, ctx) {
88   p := blk_payload(b)
89   return blk_new("M_PROTO", msg_deserialize(p[1]))
90 }
91
92 # =====
93 # L3 jones -- 結び目鍵で封緘 / 開封
94 # =====
95 def jones_wput(b, ctx) {
96   p := blk_payload(b)
97   url := p[0]
98   z := zone_parse(url)
99   if (len(z) == 0) { return blk_ctl(400, zone_status_text(400)) }
100   q := qkd_session()
101   if (ctx[CTX_TAPPED]) { q := qkd_session_tapped() }
102   ctx[CTX_QKDOK] = q[0]
103   if (!qkd_check(q)) { return blk_ctl(495, zone_status_text(495)) }
104   key := jones_key(zone_diagram(z[0]))
105   sealed := zone_seal(p[1], key, q[1])
106   ctx[CTX_CRYPT0] = [z[0], key, q[1], sealed[2], len(sealed[0])]
107   return blk_new("M_DATA", [url, sealed])
108 }
109
110 def jones_rput(b, ctx) {
111   p := blk_payload(b)
112   url := p[0]
113   z := zone_parse(url)
114   if (len(z) == 0) { return blk_ctl(400, zone_status_text(400)) }
115   key := jones_key(zone_diagram(z[0]))
116   opened := zone_open(p[1], key)
117   if (len(opened) == 0) { return blk_ctl(409, zone_status_text(409)) }
118   return blk_new("M_DATA", [url, opened[0]])
119 }
120
121 # =====
122 # L2 zone -- DHT で担当ピアを決め、台帳に追記する
123 # =====
124 def zone_wput(b, ctx) {
125   p := blk_payload(b)
126   url := p[0]
127   sealed := p[1]
128   net := ctx[CTX_NET]
129   key := zone_id(url)
130   r := zone_route(net, key)
131   peer_store_put(net[r[0]], url, sealed)
132   fact := ["zone-record", url, net[r[0]][0], key, sealed[2]]
133   fact >> tuplespace
134   ctx[CTX_ZONE] = [key, net[r[0]][0], r[1]]
135   return blk_new("M_DATA", [url, sealed])
136 }
137
138 def zone_rput(b, ctx) {
139   p := blk_payload(b)
140   url := p[0]
141   sealed := p[1]
142   net := ctx[CTX_NET]

```

```

143     key := zone_id(url)
144     r := zone_route(net, key)
145     hit := peer_store_get(net[r[0]], url)
146     if (len(hit) == 0) { return blk_ctl(404, zone_status_text(404)) }
147     fact := ["zone-fetch", url, net[r[0]][0]]
148     fact >> tuplespace
149     # 悪意あるピアがレコードを書き換える攻撃。上の jones が弾くはず。
150     if (ctx[CTX_TAMPERZ]) {
151         ct := []
152         i := 0
153         while (i < len(sealed[0])) { ct <- sealed[0][i]; i := i + 1 }
154         ct[0] = (ct[0] + 1) % 65536
155         sealed := [ct, sealed[1], sealed[2]]
156     }
157     return blk_new("M_DATA", [url, sealed])
158 }
159
160 # =====
161 # L1 ntt -- 番号を割り当て、加入者線のペア上に載せる
162 # =====
163 def ntt_wput(b, ctx) {
164     p := blk_payload(b)
165     url := p[0]
166     sealed := p[1]
167     kind := "geo"
168     if (ctx[CTX_LINE] == "hikari") { kind := "ip" }
169     num := ntt_map(url, kind)
170     bytes := envelope_encode(url, sealed)
171     sig := ntt_bearer_send(bytes, ctx[CTX_LINE], mix32(url))
172     ctx[CTX_NTT] = [num[0], num[1], ctx[CTX_LINE], len(bytes)]
173     return blk_new("M_DATA", [sig, num[0]])
174 }
175
176 def ntt_rput(b, ctx) {
177     p := blk_payload(b)
178     sig := p[0]
179     bytes := ntt_bearer_recv(sig)
180     env := envelope_decode(bytes)
181     if (env[0] != 200) { return blk_ctl(400, zone_status_text(400)) }
182     return blk_new("M_DATA", [env[1], env[2]])
183 }
184
185 # =====
186 # L0 plc driver -- ウェーブレット OFDM で電力線へ出し入れする
187 # =====
188 def plc_put(b, ctx) {
189     p := blk_payload(b)
190     sig := p[0]
191     # 回線信号をバイト列に詰める
192     bytes := ntt_signal_to_bytes(sig)
193     frame := plc_frame(bytes, 1, 2, ctx[CTX_CAP])
194     m := plc_modulate(frame, ctx[CTX_TONEMAP])
195     samples := m[0]
196     if (ctx[CTX_MARGIN] > 0) {
197         samples := plc_through_channel(samples, ctx[CTX_TONEMAP], ctx[CTX_MARGIN], ctx[CTX_SEED])
198     }
199     ctx[CTX_PHY] = [m[1], ctx[CTX_TONEMAP][4], m[1] * PLC_TSYM, len(samples), len(frame)]
200     return blk_new("M_DATA", [samples, m[2], sig[0], sig[1], sig[2]])
201 }
202
203 def plc_srv(wire, ctx) {
204     samples := wire[0]
205     bitlen := wire[1]
206     bytes := plc_demodulate(samples, ctx[CTX_TONEMAP], bitlen)
207     df := plc_deframe(bytes)
208     if (df[0] != 200) { return blk_ctl(503, "plc-" + df[2]) }
209     sig := ntt_bytes_to_signal(df[1], wire[2], wire[3], wire[4])
210     return blk_new("M_DATA", [sig, ""])
211 }
212
213 # 回線信号オブジェクト <-> バイト列 (PLC はバイト列しか運ばない)
214 def ntt_signal_to_bytes(sig) {
215     kind := sig[0]
216     if (kind == "digital") { return sig[3] }

```

```

217     if (kind == "rtp") {
218         out := []
219         pkts := sig[3]
220         i := 0
221         while (i < len(pkts)) {
222             hdr := pkts[i][2]
223             pay := pkts[i][3]
224             k := 0
225             while (k < len(hdr)) { out <- hdr[k]; k := k + 1 }
226             k := 0
227             while (k < len(pay)) { out <- pay[k]; k := k + 1 }
228             i := i + 1
229         }
230         return out
231     }
232     # voiceband: 16 ビット符号付きサンプルを 2 バイトに詰める
233     out := []
234     s := sig[3]
235     i := 0
236     while (i < len(s)) {
237         v := u_fix(s[i])
238         if (v < 0) { v := v + 65536 }
239         out <- u_trunc(v / 256) % 256
240         out <- v % 256
241         i := i + 1
242     }
243     return out
244 }
245
246 def ntt_bytes_to_signal(bytes, kind, line, bitlen) {
247     if (kind == "digital") { return ["digital", line, bitlen, bytes] }
248     if (kind == "rtp") {
249         pkts := []
250         p := 0
251         while (p + 12 <= len(bytes)) {
252             hdr := []
253             i := 0
254             while (i < 12) { hdr <- bytes[p + i]; i := i + 1 }
255             paylen := u_min(RTP_PTTIME, len(bytes) - p - 12)
256             pay := []
257             i := 0
258             while (i < paylen) { pay <- bytes[p + 12 + i]; i := i + 1 }
259             pkts <- [hdr[2] * 256 + hdr[3],
260                     hdr[4] * 16777216 + hdr[5] * 65536 + hdr[6] * 256 + hdr[7],
261                     hdr, pay]
262             p := p + 12 + paylen
263         }
264         return ["rtp", line, bitlen, pkts]
265     }
266     s := []
267     i := 0
268     while (i + 1 < len(bytes)) {
269         v := bytes[i] * 256 + bytes[i + 1]
270         if (v >= 32768) { v := v - 65536 }
271         s <- v
272         i := i + 2
273     }
274     return ["voiceband", line, bitlen, s]
275 }
276
277 # =====
278 # ストリームの往復
279 # =====
280 STREAM_MODS := ["ntt", "zone", "jones", "msgmux"] # 下から上
281
282 def stream_wput(mod, b, ctx) {
283     if (mod == "msgmux") { return msgmux_wput(b, ctx) }
284     if (mod == "jones") { return jones_wput(b, ctx) }
285     if (mod == "zone") { return zone_wput(b, ctx) }
286     return ntt_wput(b, ctx)
287 }
288
289 def stream_rput(mod, b, ctx) {
290     if (mod == "msgmux") { return msgmux_rput(b, ctx) }

```

```

291     if (mod == "jones") { return jones_rput(b, ctx) }
292     if (mod == "zone") { return zone_rput(b, ctx) }
293     return ntt_rput(b, ctx)
294 }
295
296 def stream_note_w(mod) {
297     if (mod == "msgmux") { return "統合メッセージを直列化し zone アドレスを付ける" }
298     if (mod == "jones") { return "結び目鍵で封緘" }
299     if (mod == "zone") { return "DHT で担当ピアを決め、台帳に追記" }
300     return "番号を割当て、加入者線のペアラに載せる"
301 }
302
303 def stream_note_r(mod) {
304     if (mod == "msgmux") { return "統合メッセージへ復元" }
305     if (mod == "jones") { return "結び目鍵で開封 - 改竄は 409" }
306     if (mod == "zone") { return "担当ピアの照合と台帳への追記" }
307     return "ペアラから取り出し、 $\phi^{-1}$  で zone に戻す"
308 }
309
310 # 下り: stream head -> modules (上から下) -> driver
311 def stream_write(b, ctx) {
312     ctx[CTX_TRACE] = []
313     trace_add(ctx, "down", "head", "アプリが書き込み", b)
314     i := len(STREAM_MODS) - 1
315     while (i >= 0) {
316         mod := STREAM_MODS[i]
317         b := stream_wput(mod, b, ctx)
318         trace_add(ctx, "down", mod, stream_note_w(mod), b)
319         if (blk_is_ctl(b)) { return b }
320         i := i - 1
321     }
322     b := plc_put(b, ctx)
323     trace_add(ctx, "down", "plc", "ウェーブレット OFDM で電力線へ", b)
324     return b
325 }
326
327 # 上り: driver -> modules (下から上) -> stream head
328 def stream_read(wire, ctx) {
329     b := plc_srv(wire, ctx)
330     trace_add(ctx, "up", "plc", "電力線からウェーブレット OFDM 復調", b)
331     if (blk_is_ctl(b)) { return b }
332     i := 0
333     while (i < len(STREAM_MODS)) {
334         mod := STREAM_MODS[i]
335         b := stream_rput(mod, b, ctx)
336         trace_add(ctx, "up", mod, stream_note_r(mod), b)
337         if (blk_is_ctl(b)) { return b }
338         i := i + 1
339     }
340     trace_add(ctx, "up", "head", "アプリへ到達", b)
341     return b
342 }
343
344 # 1 通をスタックの下まで送り、下から上へ戻す (折返し試験)
345 # -> [status, 理由, 受信した統合メッセージ, 往復一致か]
346 def ultra_send(m, ctx) {
347     down := stream_write(blk_new("M_PROTO", m), ctx)
348     if (blk_is_ctl(down)) { return [down[3], down[4], [], false] }
349     wire := blk_payload(down)
350     # 物理層での改竄試験: 回線上の標本を 1 つ壊す
351     if (ctx[CTX_TAMPER]) {
352         s := []
353         i := 0
354         while (i < len(wire[0])) { s <- wire[0][i]; i := i + 1 }
355         idx := u_min(40, len(s) - 1)
356         s[idx] = s[idx] + 4
357         wire := [s, wire[1], wire[2], wire[3], wire[4]]
358     }
359     up := stream_read(wire, ctx)
360     if (blk_is_ctl(up)) { return [up[3], up[4], [], false] }
361     got := blk_payload(up)
362     same := msg_serialize(got) == msg_serialize(m)
363     return [200, zone_status_text(200), got, same]
364 }

```

10 bada/10_main.bada

デモ本体 — @reviser で SEND / LINK の動詞を足し、全層を通してから 3 種類の攻撃を見せる

188 行

```

1 # =====
2 # 10_main.bada -- BADA ULTRA NETWORK / デモ本体
3 #
4 # ここまでの層を全部つないで、実際に動かして見せる。
5 #
6 # 1. @reviser 文法トランザクションで SEND / LINK の動詞を parser に足す
7 # 2. リングと結び目鍵を表示
8 # 3. コンセント対ごとの HD-PLC リンクを測る (LINK 動詞)
9 # 4. LINE と Instagram の Webhook を公式仕様どおりに受け、署名を
10 # HMAC-SHA256 で検証してから、全 5 層を通して折返す (SEND 動詞)
11 # 5. 3 種類の攻撃が、それぞれ別の層で弾かれることを見せる
12 # 6. 写像表  $\phi$  と  $\phi^{-1}$  の検算、アカシック台帳の件数
13 # =====
14
15 print("=====")
16 print("BADA ULTRA NETWORK -- HD-PLC -> NTT -> zone:// -> LINE/Instagram")
17 print(" 家のコンセントを物理層にして、NTT の加入者線へ写像し、")
18 print("  zone:// を通し、LINE と Instagram のメッセージを運ぶ。")
19 print(" 層の組み方は AT&T ベル研究所の STREAMS と同じ。")
20 print("=====")
21
22 # ---- 文法トランザクション: ウルトラネットワークの動詞を parser に足す
23 @reviser : grammar {
24     rule "ultra-send" stmt [ 'SEND' expr expr ] => ultra_demo_send(_1, _2)
25     rule "ultra-link" stmt [ 'LINK' expr ]      => ultra_demo_link(_1)
26 }
27
28 # 実行の文脈 (全部の SEND がこれを共有する)
29 UCTX := ctx_new("living", "study", "metal")
30 # 集計. def の中の "!=" は局所変数になるので、書き換えたい値は
31 # 配列の要素に置く (要素への代入なら呼び出し側にも届く)。
32 STATS := [0, 0] # [SEND した回数, 往復一致した回数]
33
34 # LINK 動詞: コンセント対を測る
35 def ultra_demo_link(which) {
36     tm := plc_tonemap(plc_channel(which, "study"))
37     print(" LINK", which, "-> study :",
38           u_trunc(tm[4] / 1000000), "Mbps,",
39           tm[2], "本有効,", tm[3], "本ノッチ,", tm[1], "bit/シンボル")
40     return tm[2]
41 }
42
43 # SEND 動詞: 1 通を全 5 層を通して折返す
44 def ultra_demo_send(m, note) {
45     print("")
46     print("-----")
47     print("SEND", note)
48     print(" 平文      :", msg_platform(m), "/", msg_name(m), ":", msg_text(m))
49     r := ultra_send(m, UCTX)
50     STATS[0] = STATS[0] + 1
51     if (r[0] == 200) {
52         c := UCTX[CTX_CRYPTO]
53         n := UCTX[CTX_NTT]
54         z := UCTX[CTX_ZONE]
55         p := UCTX[CTX_PHY]
56         print(" msgmux -> zone アドレス", msg_zone_url(m))
57         print(" jones  -> 結び目鍵", c[1], " ソルト", c[2], " タグ", c[3], " 暗号文", c[4], " ユニット")
58         print(" zone  -> 鍵", z[0], " 担当ピア", z[1])
59         print(" ntt  ->  $\phi$  =", ntt_format(n[0]), "(" + ntt_city(n[1]) + ")",
60               ntt_line_name(n[2]), " 封筒", n[3], " バイト")
61         print(" plc  -> ", p[0], " OFDM シンボル", u_trunc(p[1] / 1000000), "Mbps,",
62               u_trunc(p[2] * 1000000), " $\mu$ s 占有", p[3], "標本")
63         print(" ntt  <-  $\phi^{-1}$  (" + ntt_format(n[0]) + ") =", ntt_unmap(n[0]))
64         print(" msgmux <- ", msg_name(r[2]), ":", msg_text(r[2]))
65         print(" STATUS 200 zone-delivered 往復一致:", r[3])
66         if (r[3]) { STATS[1] = STATS[1] + 1 }
67     } else {
68         print(" STATUS", r[0], r[1])
69     }
70 }

```

```

69         if (r[0] == 409) { print("                タグが合わない。改竄か鍵違いとして排除する。") }
70         if (r[0] == 495) { print("                Bell 対の相関が壊れている。鍵を破棄し送信しない。") }
71         if (r[0] == 503) { print("                物理層の検査に失敗。回線が荒れている。") }
72     }
73     return r[0]
74 }
75
76 # ---- リングと結び目 -----
77 print("")
78 print("ウルトラネットワークのリング (RING =", RING, ", ピアのハッシュだけで作る):")
79 NET := UCTX[CTX_NET]
80 i := 0
81 while (i < len(NET)) {
82     print("    node", i, ":", NET[i][0], " id =", NET[i][1])
83     i := i + 1
84 }
85
86 print("")
87 print("ゾーンの正体である結び目 -> Jones 鍵:")
88 print(" url.or.jp (三葉結び目 / 交点 3) 鍵 =", jones_key(zone_diagram("url.or.jp")))
89 print(" bada.or.jp (8 の字結び目 / 交点 4) 鍵 =", jones_key(zone_diagram("bada.or.jp")))
90
91 # ---- HD-PLC のリンク -----
92 print("")
93 print("HD-PLC (帯域", PLC_FLO / 1000000, "-", PLC_FHI / 1000000, "MHz /", PLC_NCARR, "サブキャリア /",
94       "Δf", u_trunc(PLC_DF), "Hz / シンボル長", f5(PLC_TSYM * 1000000), "μs):")
95 LINK "living"
96 LINK "kitchen"
97
98 # ---- LINE / Instagram の Webhook を受ける -----
99 print("")
100 print("-----")
101 print("LINE Messaging API の Webhook を受ける")
102 LINE_SECRET := "demo-channel-secret"
103 LR := msg_simulate("line", 1, LINE_SECRET, "d1")
104 print(" 署名ヘッダ : X-Line-Signature")
105 print("  算法      : Base64(HMAC-SHA256(channelSecret, 生の本文))")
106 print("  本文       :", len(LR[3]), "バイト")
107 print(" 受信署名    :", substr(LR[4], 0, 32) + "...")
108 print("  検証       :", LR[2], " (status", LR[0], ")")
109 BAD := msg_ingest_line(LR[3], LINE_SECRET, line_signature("wrong-secret", LR[3]))
110 print(" 偽署名      : status", BAD[0], zone_status_text(BAD[0]), " - 正しく拒否")
111
112 print("")
113 print("Instagram Graph API の Webhook を受ける")
114 IG_SECRET := "demo-app-secret"
115 IR := msg_simulate("instagram", 1, IG_SECRET, "d2")
116 print(" 署名ヘッダ : X-Hub-Signature-256")
117 print("  算法      : \"sha256=\" + hex(HMAC-SHA256(appSecret, 生の本文))")
118 print("  本文       :", len(IR[3]), "バイト")
119 print(" 受信署名    :", substr(IR[4], 0, 32) + "...")
120 print("  検証       :", IR[2], " (status", IR[0], ")")
121
122 # ---- 全 5 層を通す -----
123 INBOX := inbox_new()
124
125 if (len(LR[1]) > 0) {
126     SEND LR[1][0] "LINE - メタル回線 (音声帯 16-QAM) 経由"
127     inbox_add(INBOX, LR[1][0])
128 }
129
130 UCTX[CTX_LINE] = "hikari"
131 if (len(IR[1]) > 0) {
132     SEND IR[1][0] "Instagram - ひかり電話 (G.711 μ-law + RTP) 経由"
133     inbox_add(INBOX, IR[1][0])
134 }
135
136 UCTX[CTX_LINE] = "isdn"
137 SEND msg_new("zone", "self", "me", "山口 まさあき", "text", "INS ネット 64 の透過", "", 0) "zone:// - INS ネット 64 (ディジタル透過) 経由"
138
139 # ---- 攻撃 -----
140 print("")
141 print("=====")

```

```

142 print("攻撃 - それぞれ別の層が弾く")
143 print("=====")
144
145 ATK := msg_new("line", "U4af4980629", "U4af4980629", "田中 さくら", "text", "改竄試験", "", 0)
146
147 UCTX[CTX_TAMPER] = true
148 SEND ATK "物理層で回線の標本を 1 つ壊す"
149 UCTX[CTX_TAMPER] = false
150
151 UCTX[CTX_TAMPERZ] = true
152 SEND ATK "悪意あるピアが保管中のレコードを書き換える"
153 UCTX[CTX_TAMPERZ] = false
154
155 UCTX[CTX_TAPPED] = true
156 SEND ATK "量子路を盗聴する (intercept-resend)"
157 UCTX[CTX_TAPPED] = false
158
159 # ---- 画像表と台帳 -----
160 print("")
161 print("=====")
162 print("画像 φ : zone:// |-> NTT 番号 (φ-1 を割当表で検算)")
163 print("=====")
164 PHI_OK := true
165 i := 0
166 while (i < len(NTT_REG_URL)) {
167     back := ntt_unmap(NTT_REG_NUM[i])
168     if (back != NTT_REG_URL[i]) { PHI_OK := false }
169     print(" ", ntt_format(NTT_REG_NUM[i]), " <- ", NTT_REG_URL[i])
170     i := i + 1
171 }
172 print(" φ-1 が厳密な逆写像 : inverse ok:", PHI_OK)
173
174 print("")
175 print("統合受信箱 :", inbox_count(INBOX), "通 /", len(inbox_threads(INBOX)), "スレッド")
176 i := 0
177 while (i < len(inbox_threads(INBOX))) {
178     th := inbox_threads(INBOX)[i]
179     print(" [" + th[0] + "]", th[2], "-", len(th[3]), "件")
180     i := i + 1
181 }
182
183 print("")
184 print("アカシック台帳 :", len(tuplespace), "件 (追記専用)")
185 print("SEND 実行", STATS[0], "回 / うち往復一致", STATS[1], "回")
186 print("=====")
187 print("BADA ULTRA NETWORK -- done")
188 print("=====")

```

11 bada/11_selftest.bada

自己診断 — 既知答えから全層の折返しまで 107 項目を実際に計算して検算する

387 行

```

1  # =====
2  # 11_selftest.bada -- BADA ULTRA NETWORK / 自己診断
3  #
4  # 01-09 のモジュールを読み込んだ上で走らせる検証プログラム。
5  # 既知答えのあるもの (SHA-256 / RFC 4231 の HMAC テストベクタ /
6  # CRC-32 の検査値) から、ウェーブレット変換の逆変換、音声帯モデムの
7  # 往復、 $\phi^{-1}$  の逆写像、全 5 層の折返し、攻撃の排除まで実際に計算する。
8  #
9  #   node ultra_network/tools/build-bada.js   でこの版が走る
10 # =====
11
12 TEST := [0, 0]      # [項目数, 合格数]
13
14 def check(name, cond, detail) {
15     TEST[0] = TEST[0] + 1
16     mark := "FAIL"
17     if (cond) { TEST[1] = TEST[1] + 1; mark := "ok  " }
18     tail := ""
19     if (detail != "") { tail := " - " + detail }
20     print(" " + mark + " " + name + tail)
21     return cond
22 }
23
24 def section(t) { print(""); print(t) }
25
26 print("=====")
27 print("BADA ULTRA NETWORK -- 自己診断 (全層を実際に計算して検算)")
28 print("=====")
29
30 # ---- 1. 基本演算 -----
31 section("1. 基本演算とビット演算")
32 check("切り捨てと符号つき切り捨て", u_trunc(7.6) == 7 && u_fix(0 - 7.6) == 0 - 7, "")
33 check("XOR / AND / OR / NOT32",
34     u_xor(61680, 4080) == 65280 && u_and(65280, 4080) == 3840 &&
35     u_or(61440, 15) == 61455 && u_not32(0) == MASK32, "")
36 check("3 項 XOR が 2 段 XOR と一致", u_xor3(123456, 654321, 999999) == u_xor(u_xor(123456, 654321), 999999), "")
37 check("バイト単位 XOR が 32 ビット版と一致", u_xor8(170, 255) == u_xor(170, 255) && u_xor8(240, 15) == 255, "")
38 check("巡回右シフト", u_rotr32(1, 1) == 2147483648 && u_rotr32(2147483648, 31) == 1, "")
39 check("32 ビット乗算 (2^53 を超えても正確)",
40     u_mul32(MASK32, MASK32) == 1 && u_mul32(305419896, 2596069104) == 606937216, "Math.imul と同値")
41 check("三角関数 (Taylor + 範囲縮約)", f5(u_sin(0.5)) == "0.47943" && f5(u_cos(0.0)) == "1.00000", "")
42 check("16 進表記", hex_of_word32(3735928559) == "deadbeef", "")
43 check("Base64 (詰め物つき)",
44     base64_of_bytes(utf8_of_str("hello")) == "aGVsbG8=" && base64_of_bytes(utf8_of_str("hi")) == "aGk=", "")
45 check("UTF-8 の往復 (CJK + 絵文字)",
46     str_of_utf8(utf8_of_str("電力線 🌩 zone:// ✓")) == "電力線 🌩 zone:// ✓", "サロゲート対を含む")
47 check("壊れた UTF-8 を U+FFFD に替えて落ちない",
48     str_of_utf8([65, 255, 66]) == "A" + chr(65533) + "B", "回線の列は攻撃者に触られる")
49 FUZZ := true
50 i := 0
51 while (i < 150) {
52     arr := []
53     k := 0
54     while (k < 1 + i % 9) { arr <- ((i * 37 + k * 101) % 256); k := k + 1 }
55     r := str_of_utf8(arr)
56     if (len(r) < 0) { FUZZ := false }
57     i := i + 1
58 }
59 check("無作為バイト列 150 通りでも投げない", FUZZ, "")
60
61 # ---- 2. SHA-256 / HMAC -----
62 section("2. SHA-256 と HMAC-SHA256 (既知答え)")
63 check("SHA-256(\\\"\\\")", sha256_hex([]) == "e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855", "")
64 check("SHA-256(\\\"abc\\\")", sha256_hex(utf8_of_str("abc")) ==
65     "ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad", "")
66 K1 := []
67 i := 0
68 while (i < 20) { K1 <- 11; i := i + 1 }

```

```

68 check("RFC 4231 テストベクタ #1",
69       hmac_sha256_hex(K1, utf8_of_str("Hi There")) ==
       "b0344c61d8db38535ca8afceaf0bf12b881dc200c9833da726e9376c2e32cff7", "")
70 check("RFC 4231 テストベクタ #2",
71       hmac_sha256_hex(utf8_of_str("Jefe"), utf8_of_str("what do ya want for nothing?")) ==
72       "5bdcc146bf60754e6a042426089575c75a003f089d2739839dec58b964ec3843", "")
73 K6 := []
74 i := 0
75 while (i < 131) { K6 <- 170; i := i + 1 }
76 check("RFC 4231 テストベクタ #6 (ブロック長超の鍵)",
77       hmac_sha256_hex(K6, utf8_of_str("Test Using Larger Than Block-Size Key - Hash Key First")) ==
78       "60e431591ee0b67f0d8a26aacbf5b77f8e0bc6213728c5140546040f0ee37f54", "")
79 check("定数時間比較", const_time_eq([1,2,3],[1,2,3]) && !const_time_eq([1,2,3],[1,2,4]) && !const_time_eq([1],[1,2]),
      "")
80
81 # ---- 3. Jones 多項式量子暗号 -----
82 section("3. Jones 多項式量子暗号")
83 KA := jones_key(zone_diagram("url.or.jp"))
84 KB := jones_key(zone_diagram("bada.or.jp"))
85 check("交点の無い図のブラケットは 1", kauffman([], 1.0) == 1.0, "")
86 check("結び目ごとに異なる鍵", KA != KB, "三葉 " + KA + " / 8 の字 " + KB)
87 check("既存 zone.bada と同じ鍵", KA == 919492 && KB == 400638, "ZoneBrowser と相互運用できる")
88 check("鍵導出は決定的", jones_key(zone_diagram("url.or.jp")) == KA, "")
89 TXT := "コンセントから zone:// が見えてる 🐼 - 日本語と絵文字"
90 SEALED := zone_seal(TXT, KA, 7)
91 check("AEAD の往復 (16 ビット符号単位)", zone_open(SEALED, KA)[0] == TXT, "")
92 TAMP := []
93 i := 0
94 while (i < len(SEALED[0])) { TAMP <- SEALED[0][i]; i := i + 1 }
95 TAMP[0] = (TAMP[0] + 1) % 65536
96 check("暗号文の改竄は 409", len(zone_open([TAMP, SEALED[1], SEALED[2]], KA)) == 0, "zone-guard-reject")
97 check("タグの偽造は 409", len(zone_open([SEALED[0], SEALED[1], SEALED[2] + 1], KA)) == 0, "")
98 check("結び目が違えば開封できない", len(zone_open(SEALED, KB)) == 0, "")
99 check("ソルトが変われば タグも変わる", zone_seal(TXT, KA, 7)[2] != zone_seal(TXT, KA, 8)[2], "")
100 Q := qkd_session()
101 QT := qkd_session_tapped()
102 check("Bell 対: 禁止状態が厳密に 0", Q[0] && Q[3] == 0, "|01>+|10> = " + Q[3])
103 check("Bell 対: |00> と |11> が各 1/2", Q[2][0] == 0.5 && Q[2][3] == 0.5, "")
104 check("盗聴すると相関が壊れ検出される", !qkd_check(QT) && QT[3] > 0, "|01>+|10> = " + f5(QT[3]))
105
106 # ---- 4. zone:// -----
107 section("4. zone:// ウルトラネットワーク WWW")
108 NET := zone_boot()
109 Z := zone_parse("zone://url.or.jp/msg/line/U1")
110 check("zone:// の文法", Z[0] == "url.or.jp" && Z[1] == "/msg/line/U1", "")
111 check("パス省略時は /", zone_parse("zone://url.or.jp")[1] == "/", "")
112 check("zone:// 以外は拒否", len(zone_parse("https://url.or.jp/")) == 0 && len(zone_parse("x zone://y")) == 0, "400 bad-zone")
113 check("リングは node-id 順", NET[0][1] < NET[1][1] && NET[4][1] < NET[5][1], "")
114 check("リング距離は巡回的", ring_dist(10, RING - 10) == 20, "")
115 REACH := true
116 URLS := ["zone://url.or.jp/", "zone://bada.or.jp/x", "zone://url.or.jp/msg/line/U9"]
117 i := 0
118 while (i < len(URLS)) {
119   r := zone_route(NET, zone_id(URLS[i]))
120   if (r[0] != dht_nearest(NET, zone_id(URLS[i]))) { REACH := false }
121   i := i + 1
122 }
123 check("貪欲ルーティングが担当ピアへ到達", REACH, "")
124 PUB := zone_publish(NET, "zone://url.or.jp/", "# home | ウルトラネットワーク")
125 GOT := zone_get(NET, "zone://url.or.jp/")
126 check("公開と取得", PUB[0] == 200 && GOT[0] == 200 && GOT[1] == "# home | ウルトラネットワーク", "")
127 check("未保管のアドレスは 404", zone_get(NET, "zone://ghost.or.jp/")[0] == 404, "zone-not-found")
128 ENV := envelope_decode(envelope_encode("zone://url.or.jp/x", [[1, 65535, 42], 1234, 3735928559]))
129 check("封筒の符号化と復号が往復",
130       ENV[0] == 200 && ENV[1] == "zone://url.or.jp/x" && ENV[2][1] == 1234 && ENV[2][2] == 3735928559, "")
131 check("壊れた封筒は 400", envelope_decode([1,2,3])[0] == 400, "")
132 check("状態符号が既存実装と同じ",
133       zone_status_text(409) == "zone-guard-reject" && zone_status_text(495) == "quantum-channel-compromised", "")
134
135 # ---- 5. HD-PLC -----
136 section("5. HD-PLC 物理層 (IEEE 1901 ウェーブレット OFDM)")
137 XS := []
138 i := 0

```

```

139 while (i < 512) { XS <- (u_sin(i / 7) * 3 + u_cos(i / 3) - i / 500); i := i + 1 }
140 YS := dwt_synthesize(dwt_analyze(XS))
141 MX := 0.0
142 i := 0
143 while (i < 512) {
144     d := abs(YS[i] - XS[i])
145     if (d > MX) { MX := d }
146     i := i + 1
147 }
148 check("ウェーブレット変換が厳密な逆変換", MX < 0.000000001, "最大誤差 " + sci(MX))
149 check("帯域 2-28 MHz を 512 分割", PLC_NCARR == 512 && u_trunc(PLC_DF) == 50781, "Δf = 50.78 kHz")
150 check("シンボル長 = 1/Δf (ガードインターバル無し)", f5(PLC_TSYM * 1000000) == "19.69231", "19.69 μs")
151 check("アマチュア無線帯の判定", in_ham_band(7100000) && in_ham_band(14200000) && !in_ham_band(5000000), "")
152 check("Gray 符号の対応表", gray_decode(gray_encode(23)) == 23 && GRAY_ENC[2] == 3 && GRAY_ENC[3] == 2, "")
153 check("CRC-32 の検査値", crc32(utf8_of_str("123456789")) == 3421780262 && crc32([]) == 0, "0xCBf43926")
154 CH := plc_channel("living", "kitchen")
155 TM := plc_tonemap(CH)
156 NOTCH_OK := true
157 CAP_OK := true
158 i := 0
159 while (i < PLC_NCARR) {
160     if (in_ham_band(CH[0][i]) && TM[0][i] != 0) { NOTCH_OK := false }
161     if (TM[0][i] > PLC_MAXBITS) { CAP_OK := false }
162     i := i + 1
163 }
164 check("ノッチにビットを割り当てない", NOTCH_OK, TM[3] + " 本をノッチ")
165 check("割当ビットは 32-PAM (5 bit) が上限", CAP_OK, "")
166 check("適応ビットローディング", TM[2] > 50 && TM[4] > 1000000,
167     u_trunc(TM[4] / 1000000) + " Mbps / 有効 " + TM[2] + " 本")
168 HIST := [0, 0, 0, 0, 0, 0]
169 i := 0
170 while (i < PLC_NCARR) { HIST[TM[0][i]] = HIST[TM[0][i]] + 1; i := i + 1 }
171 NLEVELS := 0
172 i := 0
173 while (i < 6) {
174     if (HIST[i] > 0) { NLEVELS := NLEVELS + 1 }
175     i := i + 1
176 }
177 check("SNR に応じてビット数が分布", NLEVELS >= 3, "0-5 本数 " + HIST[0] + "/" + HIST[1] + "/" + HIST[2] + "/" + HIST[3]
178     + "/" + HIST[4] + "/" + HIST[5])
179 CH2 := plc_channel("router", "study")
180 TM2 := plc_tonemap(CH2)
181 PAY := utf8_of_str("ウルトラネットワーク payload 0123456789 ✓ 電力線")
182 MOD := plc_modulate(PAY, TM2)
183 check("ウェーブレット OFDM の完全往復",
184     str_of_utf8(plc_demodulate(MOD[0], TM2, MOD[2])) == str_of_utf8(PAY),
185     MOD[1] + " シンボル / " + len(MOD[0]) + " 標本")
186 NOISY := plc_through_channel(MOD[0], TM2, 0, "n1")
187 check("設計 SNR では回線雑音を通しても誤らない",
188     str_of_utf8(plc_demodulate(NOISY, TM2, MOD[2])) == str_of_utf8(PAY), "")
189 BADCH := plc_through_channel(MOD[0], TM2, 24, "n2")
190 check("24 dB 劣化させると誤りが現れる",
191     str_of_utf8(plc_demodulate(BADCH, TM2, MOD[2])) != str_of_utf8(PAY), "")
192 FR := plc_frame(PAY, 1, 2, 2)
193 DF := plc_deframe(FR)
194 check("IEEE 1901 フレームの組立と分解", DF[0] == 200 && str_of_utf8(DF[1]) == str_of_utf8(PAY), len(FR) + " バイト")
195 def corrupt_at(fr, idx) {
196     c := []
197     i := 0
198     while (i < len(fr)) { c <- fr[i]; i := i + 1 }
199     c[idx] = u_xor8(c[idx], 255)
200     return c
201 }
202 check("ペイロード破損を CRC-32 が検出", plc_deframe(corrupt_at(FR, 40))[2] == "pb-crc", "")
203 check("フレーム制御の破損を CRC-24 が検出", plc_deframe(corrupt_at(FR, 10))[2] == "fc-crc", "")
204 check("プリアンブル欠落を検出", plc_deframe(corrupt_at(FR, 0))[2] == "no-preamble", "")
205 CS := plc_csma(["A", 1], ["B", 3], ["C", 3], ["D", 0]), "x")
206 CS_WIN := false
207 if (CS[2] == "B") { CS_WIN := true }
208 if (CS[2] == "C") { CS_WIN := true }
209 check("CSMA/CA: 最優先 CAP が送信権", CS[0] == 3 && CS_WIN, "勝者 " + CS[2])
210 # ---- 6. NTT 写像 -----
211 section("6. NTT 電話回線への写像")

```

```

212 SURLS := ["zone://url.or.jp/", "zone://url.or.jp/news", "zone://bada.or.jp/",
213           "zone://url.or.jp/msg/line/U1", "zone://url.or.jp/msg/instagram/17841"]
214 PHI_OK := true
215 FMT_OK := true
216 i := 0
217 while (i < len(SURLS)) {
218     m := ntt_map(SURLS[i], "geo")
219     if (ntt_unmap(m[0]) != SURLS[i]) { PHI_OK := false }
220     if (len(m[0]) != 10) { FMT_OK := false }
221     if (ntt_e164(m[0]) != "+81" + substr(m[0], 1, 9)) { FMT_OK := false }
222     i := i + 1
223 }
224 check("写像  $\phi^{-1}$  が厳密な逆写像", PHI_OK, ntt_format(ntt_map(SURLS[0], "geo")[0]) + " ⇔ " + SURLS[0])
225 check("0AB-J は 10 桁、E.164 は +81", FMT_OK, "")
226 check("写像  $\phi$  は安定", ntt_map(SURLS[0], "geo")[0] == ntt_map(SURLS[0], "geo")[0], "")
227 check("050 / 090 の帯へも写せる",
228       substr(ntt_map(SURLS[0], "ip")[0], 0, 3) == "050" && substr(ntt_map(SURLS[0], "mobile")[0], 0, 3) == "090",
229       "")
229 check("未割当の番号は逆引きできない", ntt_unmap("03-9999-9999") == "", "")
230 check("番号の整形",
231       ntt_format("0312345678") == "03-1234-5678" && ntt_format("0112345678") == "011-234-5678" &&
232       ntt_format("05012345678") == "050-1234-5678", "")
233 BTXT := "音声帯 300-3400 Hz を通す封筒 #42 ✓"
234 BPAY := utf8_of_str(BTXT)
235 MSIG := ntt_bearer_send(BPAY, "metal", 287454020)
236 check("メタル: 音声帯 16-QAM の往復", str_of_utf8(ntt_bearer_recv(MSIG)) == BTXT,
237       "2000 baud / 8000 bps / " + u_trunc(len(MSIG[3]) / 4) + " シンボル")
238 PTS := ntt_constellation(MSIG[3], 60)
239 GRID := true
240 i := 0
241 while (i < len(PTS)) {
242     ai := abs(PTS[i][0])
243     aq := abs(PTS[i][1])
244     if (ai != 1 && ai != 3) { GRID := false }
245     if (aq != 1 && aq != 3) { GRID := false }
246     i := i + 1
247 }
248 check("受信点が 16-QAM の格子上に乗る", GRID, "")
249 ISIG := ntt_bearer_send(BPAY, "isdn", 0)
250 check("INS ネット 64: デジタル透過", str_of_utf8(ntt_bearer_recv(ISIG)) == BTXT, len(ISIG[3]) + " オクテット")
251 HSIG := ntt_bearer_send(BPAY, "hikari", 287454020)
252 check("ひかり電話: G.711  $\mu$ -law + RTP の往復", str_of_utf8(ntt_bearer_recv(HSIG)) == BTXT,
253       len(HSIG[3]) + " パケット (PCMU 20 ms)")
254 check("RTP ヘッダが PT=0 / 20 ms", HSIG[3][0][2][1] == 0 && len(HSIG[3][0][3]) == RTP_PTIME, "")
255 REV := []
256 i := len(HSIG[3]) - 1
257 while (i >= 0) { REV <- HSIG[3][i]; i := i - 1 }
258 check("RTP の順序が乱れても並べ直して復元", str_of_utf8(ntt_bearer_recv(["rtp", "hikari", HSIG[2], REV])) == BTXT, "")
259 WORST := 0.0
260 v := 0 - 8000
261 while (v <= 8000) {
262     e := abs(mulaw_decode(mulaw_encode(v)) - v) / (abs(v) + 200)
263     if (e > WORST) { WORST := e }
264     v := v + 137
265 }
266 check("G.711  $\mu$ -law の量子化誤差が判定余裕の内側", WORST < 0.09, f5(WORST * 100) + "%")
267 CALL := ntt_call("zone://url.or.jp/", "zone://bada.or.jp/", "hikari")
268 check("呼制御の状態機械", len(CALL[2]) == 6 && CALL[2][0][0] == "on-hook" && CALL[2][5][0] == "data", "")
269
270 # ---- 7. JSON -----
271 section("7. JSON (Webhook 本文の解析と生成)")
272 JS1 := json_parse("{\"a\":-12.5,\"b\":true,\"c\":null,\"d\":[1,2,3],\"e\":\"q\\\"r\\\",\\\"f\\\":1.5e3}")
273 check("数 / 真偽 / null / 配列",
274       json_num(json_get(JS1, "a")) == 0 - 12.5 && json_bool(json_get(JS1, "b")) &&
275       json_is(json_get(JS1, "c"), "null") && json_num(json_at(json_get(JS1, "d"), 2)) == 3, "")
276 check("エスケープと指数", json_str(json_get(JS1, "e")) == "q" + chr(34) + "r" && json_num(json_get(JS1, "f")) == 1500,
277       "")
278 JS2 := json_parse("{\"entry\":{\"messaging\":{\"message\":{\"text\":\"配線図\"}}}}")
279 check("経路で潜る", json_str(json_path(JS2, "entry.0.messaging.0.message.text")) == "配線図", "")
280 check("無い鍵は null", json_is(json_get(JS2, "nope"), "null"), "")
281 check("生成して読み直せる",
282       json_str(json_get(json_parse(json_stringify(jobj(["k"], [jstr("値 \" と改行")]))), "k")) == "値 " + chr(34) + "
282       と改行", "")

```

```

283 # ---- 8. LINE / Instagram -----
284 section("8. LINE / Instagram (公式 API の仕様どおり)")
285 LRAW := sim_line_body(1, "t1")
286 LSIG := line_signature("channel-secret", LRAW)
287 check("LINE: X-Line-Signature が一致", line_verify("channel-secret", LRAW, LSIG), "")
288 check("LINE: 誤った秘密は拒否", !line_verify("channel-secret", LRAW, line_signature("wrong", LRAW)), "")
289 check("LINE: 本文が 1 文字違えば拒否", !line_verify("channel-secret", LRAW + " ", LSIG), "")
290 LING := msg_ingest_line(LRAW, "channel-secret", LSIG)
291 check("LINE: Webhook から取り込む", LING[0] == 200 && len(LING[1]) == 1 && msg_platform(LING[1][0]) == "line", "")
292 check("LINE: replyToken を保持", msg_reply(LING[1][0]) != "", "")
293 check("LINE: 署名不一致は 401", msg_ingest_line(LRAW, "channel-secret", "bogus")[0] == 401, "")
294 IRAW := sim_instagram_body(1, "t2")
295 ISIG2 := ig_signature("app-secret", IRAW)
296 check("Instagram: sha256= 接頭辞つき hex", substr(ISIG2, 0, 7) == "sha256=" && len(ISIG2) == 71, "")
297 check("Instagram: X-Hub-Signature-256 が一致", ig_verify("app-secret", IRAW, ISIG2), "")
298 check("Instagram: 誤った秘密は拒否", !ig_verify("app-secret", IRAW, ig_signature("wrong", IRAW)), "")
299 IING := msg_ingest_instagram(IRAW, "app-secret", ISIG2)
300 check("Instagram: Webhook から取り込む", IING[0] == 200 && len(IING[1]) == 1 && msg_platform(IING[1][0]) ==
    "instagram", "")
301 check("Instagram: 署名不一致は 401", msg_ingest_instagram(IRAW, "app-secret", "sha256=00")[0] == 401, "")
302 OUT1 := msg_to_outbound(msg_new("line", "U1", "U1", "x", "text", "こんにちは", "rt1", 0), "")
303 check("LINE: reply の送信要求が公式仕様の形",
304     OUT1[0] == LINE_REPLY_EP && str_find(OUT1[3], "\"replyToken\": \"rt1\"") >= 0 &&
305     str_find(OUT1[3], "\"type\": \"text\"") >= 0, "")
306 OUT2 := msg_to_outbound(msg_new("line", "U2", "U2", "x", "text", "push", "", 0), "")
307 check("LINE: replyToken が無ければ push", OUT2[0] == LINE_PUSH_EP && str_find(OUT2[3], "\"to\": \"U2\"") >= 0, "")
308 OUT3 := msg_to_outbound(msg_new("instagram", "1784140001", "1784140001", "x", "text", "hi", "", 0), "1784149999")
309 check("Instagram: /<IG_ID>/messages の形",
310     OUT3[0] == "https://graph.instagram.com/v21.0/1784149999/messages" &&
311     str_find(OUT3[3], "\"recipient\": {\"id\": \"1784140001\"}") >= 0, "")
312 MM := msg_new("line", "U1", "U1", "田中 さくら", "text", "直列化 🐼 ✓", "rt", 1700000000000)
313 MB := msg_deserialize(msg_serialize(MM))
314 check("統合メッセージの直列化が往復",
315     msg_text(MB) == msg_text(MM) && msg_name(MB) == msg_name(MM) && msg_reply(MB) == "rt", "")
316 check("メッセージの zone アドレス", msg_zone_url(MM) == "zone://url.or.jp/msg/line/U1", "")
317 BOX := inbox_new()
318 inbox_add(BOX, LING[1][0])
319 inbox_add(BOX, IING[1][0])
320 inbox_add(BOX, LING[1][0])
321 check("統合受信箱に LINE と Instagram が同居", inbox_count(BOX) == 3 && len(inbox_threads(BOX)) == 2, "")
322
323 # ---- 9. STREAMS 全層 -----
324 section("9. AT&T ベル研究所式 STREAMS - 全 5 層の往復")
325 check("driver の上に 4 モジュールが積まれている",
326     len(STREAM_MODS) == 4 && STREAM_MODS[0] == "ntt" && STREAM_MODS[3] == "msgmux",
327     "plc <- ntt <- zone <- jones <- msgmux <- head")
328 SCTX := ctx_new("living", "study", "metal")
329 MSG := msg_new("line", "U4af", "U4af", "田中 さくら", "text", "電力線から NTT を経て zone:// 🐼 🌐", "rt",
330     1700000000000)
331 SR := ultra_send(MSG, SCTX)
332 check("全 5 層を通した往復が完全一致", SR[0] == 200 && SR[3], SR[0] + " " + SR[1])
333 check("本文と表示名が保たれる", msg_text(SR[2]) == msg_text(MSG) && msg_name(SR[2]) == msg_name(MSG), "")
334 TRACE := SCTX[CTX_TRACE]
335 TSEQ := ""
336 i := 0
337 while (i < len(TRACE)) {
338     d := "v"
339     if (TRACE[i][0] == "up") { d := "^" }
340     TSEQ := TSEQ + d + TRACE[i][1] + " "
341     i := i + 1
342 }
343 check("下りと上りが各層を 1 回ずつ通る",
344     TSEQ == "vhead vmsgmux vjones vzone vntt vplc ^plc ^ntt ^zone ^jones ^msgmux ^head ", TSEQ)
345 check("各層が自分の計量を残す",
346     len(SCTX[CTX_NTT][0]) >= 10 && SCTX[CTX_ZONE][1] != "" && SCTX[CTX_CRYPTO][1] > 0 && SCTX[CTX_PHY][0] > 0,
347     "φ=" + ntt_format(SCTX[CTX_NTT][0]) + " / ピア=" + SCTX[CTX_ZONE][1] + " / " +
348     SCTX[CTX_PHY][0] + " OFDM シンボル")
349 ICTX := ctx_new("living", "study", "isdn")
350 IRES := ultra_send(MSG, ICTX)
351 check("回線種別: INS ネット 64 でも往復", IRES[0] == 200 && IRES[3], "")
352 HCTX := ctx_new("living", "study", "hikari")
353 HRES := ultra_send(MSG, HCTX)
354 check("回線種別: ひかり電話でも往復", HRES[0] == 200 && HRES[3], "")
355 OCTX := ctx_new("bedroom", "router", "isdn")

```

```

355 ORES := ultra_send(SMSG, OCTX)
356 check("別のコンセント対でも往復", ORES[0] == 200 && ORES[3],
357       u_trunc(OCTX[CTX_TONEMAP][4] / 1000000) + " Mbps")
358
359 section("  攻撃 - それぞれ正しい状態符号で排除される")
360 ATKM := msg_new("line", "U1", "U1", "試験", "text", "改竄試験", "", 0)
361 ACTX := ctx_new("living", "study", "isdn")
362 ACTX[CTX_TAMPER] = true
363 AR := ultra_send(ATKM, ACTX)
364 check("物理層で回線を改竄 -> PLC の CRC が弾く", AR[0] == 503, AR[0] + " " + AR[1])
365 BCTX := ctx_new("living", "study", "isdn")
366 BCTX[CTX_TAMPERZ] = true
367 BR := ultra_send(ATKM, BCTX)
368 check("悪意あるピアの書換え -> AEAD が弾く", BR[0] == 409, BR[0] + " " + BR[1])
369 CCTX := ctx_new("living", "study", "isdn")
370 CCTX[CTX_TAPPED] = true
371 CR := ultra_send(ATKM, CCTX)
372 check("量子路の盗聴 -> 送信そのものを止める", CR[0] == 495, CR[0] + " " + CR[1])
373 DCTX := ctx_new("living", "study", "isdn")
374 DCTX[CTX_MARGIN] = 26
375 DR := ultra_send(ATKM, DCTX)
376 check("電力線を 26 dB 劣化 -> 復号できない", DR[0] != 200, DR[0] + " " + DR[1])
377
378 # ---- 総括 -----
379 print("")
380 print("=====")
381 if (TEST[0] == TEST[1]) {
382     print("合格 " + TEST[1] + " / " + TEST[0] + " - すべて合格")
383 } else {
384     print("合格 " + TEST[1] + " / " + TEST[0] + " 不合格 " + (TEST[0] - TEST[1]))
385 }
386 print("SELFTEST-RESULT " + TEST[1] + " " + TEST[0])
387 print("=====")

```

12 ultra.bada

単一ファイル版リファレンス — bada-cli の既定ステップ上限で走る、64 サブキャリアの縮小版。全層を 1 本にまとめている

1171 行

```

1 # =====
2 # ultra.bada -- the BADA ULTRA NETWORK
3 #
4 # Panasonic HD-PLC turns the house wiring into the physical
5 # layer; the frames are mapped onto an NTT subscriber line;
6 # they carry the zone://url.or.jp ultra-network WWW; the
7 # payload is sealed with the Jones-polynomial quantum cipher;
8 # and the thing being carried is a LINE / Instagram message.
9 #
10 # The whole stack is organised the way AT&T Bell Laboratories
11 # organised device I/O -- STREAMS (Dennis Ritchie's Streams
12 # I/O system): a driver at the bottom, a stream head at the
13 # top, and modules PUSHed in between, each one nothing but a
14 # pair of queues, wput (downstream) and rput (upstream).
15 #
16 #         stream head   (the application: unified inbox)
17 #         | wput         rput |
18 #     +--- msgmux ---+   LINE / Instagram multiplexing
19 #     +--- jones  ---+   Jones-polynomial AEAD + Bell QKD
20 #     +--- zone   ---+   zone:// P2P ring DHT
21 #     +--- ntt    ---+   mapping onto the NTT phone line
22 #         | put         srv |
23 #     driver plc   (Panasonic HD-PLC = the wall socket)
24 #     ~~~~~ the house wiring, 2-28 MHz ~~~~~
25 #
26 # Every layer is written here in Bada itself. The algorithms
27 # are the same ones the single-file app (ultra_network/
28 # index.html) runs, and the naming, the key schedule and the
29 # ring are identical to examples/zone.bada, so records made
30 # here are readable by the ZoneBrowser.
31 #
32 # NOTE ON SIZE. The app uses 512 subcarriers; this reference
33 # uses 64 so the interpreter finishes quickly. The wavelet
34 # transform, the bit loading and the framing are otherwise
35 # the same construction.
36 #
37 # NOTE ON OPERATORS. Bada has no bitwise operators, so where
38 # a wire format would normally use XOR (CRC-32, Gray coding)
39 # this reference uses the modular rolling hash / plain binary
40 # indexing instead. Everything is arithmetic.
41 # =====
42
43 print("=====")
44 print("BADA ULTRA NETWORK -- HD-PLC -> NTT -> zone:// -> LINE/Instagram")
45 print("=====")
46
47 RING    := 4096      # zone:// ring order (same as zone.bada)
48 NCARR   := 64       # subcarriers in this reference
49 FLO     := 2000000   # 2 MHz
50 FHI     := 28000000  # 28 MHz
51 TXPSD   := 0 - 50   # transmit PSD, dBm/Hz
52 MAXBITS := 5        # 32-PAM ceiling
53
54 # ---- grammar transaction: teach the parser the ultra verbs -----
55 @reviser : grammar {
56     rule "ultra-send" stmt [ 'SEND' expr expr ] => ultra_send(_1, _2)
57     rule "ultra-link" stmt [ 'LINK' expr ]      => ultra_link(_1)
58 }
59 print("ultra verbs committed to the grammar ledger:", len(tuplespace), "facts")
60
61 # =====
62 # 0. arithmetic helpers (Bada has sqrt/log/exp/abs but no floor/trig)
63 # =====
64
65 # truncation toward zero; we only ever use it on non-negatives
66 def u_trunc(x) { return x - (x % 1) }
67
68 def ipow(base, e) {

```

```

69     if (e < 0) {
70         r := 1.0
71         k := 0
72         m := 0 - e
73         while (k < m) { r := r * base; k := k + 1 }
74         return 1.0 / r
75     }
76     r := 1.0
77     k := 0
78     while (k < e) { r := r * base; k := k + 1 }
79     return r
80 }
81
82 TWO_PI := 6.283185307179586
83
84 # sine by range reduction + Taylor series (7 terms is ample here)
85 def u_sin(x) {
86     t := x % TWO_PI
87     if (t > 3.141592653589793) { t := t - TWO_PI }
88     if (t < 0 - 3.141592653589793) { t := t + TWO_PI }
89     term := t
90     sum := t
91     k := 1
92     while (k < 9) {
93         term := 0 - term * t * t / ((2 * k) * (2 * k + 1))
94         sum := sum + term
95         k := k + 1
96     }
97     return sum
98 }
99
100 def u_cos(x) { return u_sin(x + 1.5707963267948966) }
101
102 def u_log2(x) { return log(x) / 0.6931471805599453 }
103
104 # integer division and the k-th binary digit, without bit operators.
105 # POW2 is precomputed: ipow() in an inner loop is what makes an
106 # interpreted physical layer slow.
107 POW2 := [1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096,
108          8192, 16384, 32768, 65536]
109
110 def u_div(a, b) { return u_trunc(a / b) }
111 def u_bit(v, k) { return u_trunc(v / POW2[k]) % 2 }
112
113 # =====
114 # L3 (key schedule): the Jones-polynomial quantum cipher
115 # -----
116 # A zone's long-term key is DERIVED FROM A KNOT. The Kauffman bracket
117 # state sum <D>(A) = sum over A/B smoothings of A^(a-b) d^(loops-1),
118 # d = -A^2 - 1/A^2, is sampled at several A and hashed. The knot is
119 # the zone's secret; the polynomial is its key schedule.
120 # =====
121
122 def str_hash(s) {
123     h := 7
124     i := 0
125     n := len(s)
126     while (i < n) {
127         h := (h * 31 + ord(s, i)) % 999983
128         i := i + 1
129     }
130     return h
131 }
132
133 def zone_id(s) { return str_hash(s) % RING }
134
135 def uf_find(parent, x) {
136     r := x
137     while (parent[r] >= 0) { r := parent[r] }
138     return r
139 }
140
141 # a crossing is [id, e0, e1, e2, e3, sign]; the bracket ignores sign
142 def kauffman(cross, A) {

```

```

143     n := len(cross)
144     if (n == 0) { return 1.0 }
145     maxlbl := 0
146     i := 0
147     while (i < n) {
148         k := 1
149         while (k <= 4) {
150             if (cross[i][k] > maxlbl) { maxlbl := cross[i][k] }
151             k := k + 1
152         }
153         i := i + 1
154     }
155     U := maxlbl + 1
156     d := 0 - A * A - 1.0 / (A * A)
157     states := ipow(2, n)
158     sum := 0.0
159     bits := []
160     q := 0
161     while (q < n) { bits <- 0; q := q + 1 }
162
163     st := 0
164     while (st < states) {
165         parent := []
166         q := 0
167         while (q < U) { parent <- (0 - 1); q := q + 1 }
168         a_cnt := 0
169         b_cnt := 0
170         i := 0
171         while (i < n) {
172             e0 := cross[i][1]
173             e1 := cross[i][2]
174             e2 := cross[i][3]
175             e3 := cross[i][4]
176             if (bits[i] == 0) {
177                 a_cnt := a_cnt + 1
178                 p := e0; qq := e1; r := e2; s := e3
179             } else {
180                 b_cnt := b_cnt + 1
181                 p := e0; qq := e3; r := e1; s := e2
182             }
183             ra := uf_find(parent, p)
184             rb := uf_find(parent, qq)
185             if (ra != rb) { parent[rb] = ra }
186             rc := uf_find(parent, r)
187             rd := uf_find(parent, s)
188             if (rc != rd) { parent[rd] = rc }
189             i := i + 1
190         }
191         seen := []
192         q := 0
193         while (q < U) { seen <- 0; q := q + 1 }
194         loops := 0
195         lbl := 0
196         while (lbl < U) {
197             r2 := uf_find(parent, lbl)
198             if (seen[r2] == 0) { seen[r2] = 1; loops := loops + 1 }
199             lbl := lbl + 1
200         }
201         sum := sum + ipow(A, a_cnt - b_cnt) * ipow(d, loops - 1)
202         c := 0
203         carry := 1
204         while (c < n && carry == 1) {
205             if (bits[c] == 0) { bits[c] = 1; carry := 0 }
206             else { bits[c] = 0 }
207             c := c + 1
208         }
209         st := st + 1
210     }
211     return sum
212 }
213
214 def jones_key(diagram) {
215     As := [0.8, 1.0, 1.2, 1.5, 2.0]
216     acc := "jones|"

```

```

217     i := 0
218     while (i < len(As)) {
219         acc := acc + f5(kauffman(diagram, As[i])) + "|"
220         i := i + 1
221     }
222     return str_hash(acc)
223 }
224
225 # the zone's identity knots (its secret) -- same as zone.bada
226 def zone_diagram(host) {
227     if (host == "bada.or.jp") {
228         return [ [0,1,2,3,4,1], [1,3,4,5,6,1], [2,5,6,7,8,1], [3,7,8,1,2,1] ]
229     }
230     return [ [0,1,2,3,4,1], [1,3,4,5,6,1], [2,5,6,1,2,1] ]
231 }
232
233 def lcg(state) { return (state * 48271) % 2147483647 }
234
235 def key_seed(key, salt) {
236     state := (key + salt * 2654435761) % 2147483647
237     if (state <= 0) { state := state + 2147483646 }
238     return state
239 }
240
241 # the keystream works on 16-bit code units so CJK round-trips
242 def zone_encrypt(plain, key, salt) {
243     state := key_seed(key, salt)
244     out := []
245     i := 0
246     while (i < len(plain)) {
247         state := lcg(state)
248         out <- (ord(plain, i) + (state % 65536)) % 65536
249         i := i + 1
250     }
251     return out
252 }
253
254 def zone_decrypt(ct, key, salt) {
255     state := key_seed(key, salt)
256     out := ""
257     i := 0
258     while (i < len(ct)) {
259         state := lcg(state)
260         out := out + chr((ct[i] - (state % 65536) + 65536) % 65536)
261         i := i + 1
262     }
263     return out
264 }
265
266 def zone_tag(ct, key, salt) {
267     h := (key % 999983) + salt + 1
268     i := 0
269     while (i < len(ct)) {
270         h := (h * 31 + ct[i] + 7) % 2147483647
271         i := i + 1
272     }
273     return h
274 }
275
276 def zone_seal(plain, key, salt) {
277     ct := zone_encrypt(plain, key, salt)
278     return [ct, salt, zone_tag(ct, key, salt)]
279 }
280
281 # open -> [plaintext] ; [] means 409 zone-guard-reject
282 def zone_open(sealed, key) {
283     ct := sealed[0]
284     salt := sealed[1]
285     tag := sealed[2]
286     if (zone_tag(ct, key, salt) != tag) { return [] }
287     return [zone_decrypt(ct, key, salt)]
288 }
289
290 # ---- Bell-pair QKD: session salt + channel-tamper evidence -----

```

```

291 # |Phi> = (|00> + |11>)/sqrt(2). The forbidden outcomes |01>,|10>
292 # stay EXACTLY 0; an intercept-resend tap breaks that correlation.
293 def qkd_session() {
294     bell := qubit(2)
295     bell := H(bell, 0)
296     bell := CNOT(bell, 0, 1)
297     outcome := Measure(bell)
298     ok := (outcome[1] == 0) && (outcome[2] == 0)
299     salt := (len(tuplespace) * 131 + 17) % 4096
300     return [ok, salt, outcome]
301 }
302
303 # =====
304 # L2: zone:// naming and the P2P ring DHT (no DNS, no origin server)
305 # =====
306
307 # zone://host/path -> [host, path, labels] ([] if not a zone URL)
308 def zone_parse(url) {
309     if (str_find(url, "zone://") != 0) { return [] }
310     rest := substr(url, 7, len(url) - 7)
311     cut := str_find(rest, "/")
312     if (cut < 0) { return [rest, "/", split(rest, ".")] }
313     host := substr(rest, 0, cut)
314     path := substr(rest, cut, len(rest) - cut)
315     return [host, path, split(host, ".")]
316 }
317
318 def ring_dist(a, b) {
319     d := a - b
320     if (d < 0) { d := 0 - d }
321     e := RING - d
322     if (e < d) { return e }
323     return d
324 }
325
326 # a peer is [name, node-id, ring-index]
327 def peer_new(name) { return [name, zone_id("peer:" + name), 0] }
328
329 def zone_boot() {
330     names := ["tokyo.zone.jp", "osaka.zone.jp", "nagoya.zone.jp",
331             "sapporo.zone.jp", "fukuoka.zone.jp", "naha.zone.jp"]
332     net := []
333     i := 0
334     while (i < len(names)) { net <- peer_new(names[i]); i := i + 1 }
335     # insertion sort by node-id
336     sorted := []
337     i := 0
338     while (i < len(net)) {
339         p := net[i]
340         pos := len(sorted)
341         found := false
342         j := 0
343         while (j < len(sorted)) {
344             if (!found) {
345                 if (p[1] < sorted[j][1]) { pos := j; found := true }
346             }
347             j := j + 1
348         }
349         out := []
350         k := 0
351         while (k < len(sorted)) {
352             if (k == pos) { out <- p }
353             out <- sorted[k]
354             k := k + 1
355         }
356         if (pos == len(sorted)) { out <- p }
357         sorted := out
358         i := i + 1
359     }
360     m := 0
361     while (m < len(sorted)) { sorted[m][2] = m; m := m + 1 }
362     return sorted
363 }
364

```

```

365 def dht_nearest(net, key) {
366     best := 0
367     bd := ring_dist(net[0][1], key)
368     i := 1
369     while (i < len(net)) {
370         d := ring_dist(net[i][1], key)
371         if (d < bd) { bd := d; best := i }
372         i := i + 1
373     }
374     return best
375 }
376
377 # greedy routing over the +1 / -1 / +2 fingers
378 def zone_route(net, key) {
379     n := len(net)
380     target := dht_nearest(net, key)
381     cur := 0
382     hops := [net[0][0]]
383     guard := 0
384     while (cur != target && guard < 2 * n) {
385         nb := [(cur + 1) % n, (cur + n - 1) % n, (cur + 2) % n]
386         best := cur
387         bd := ring_dist(net[cur][1], key)
388         i := 0
389         while (i < len(nb)) {
390             d := ring_dist(net[nb[i]][1], key)
391             if (d < bd) { bd := d; best := nb[i] }
392             i := i + 1
393         }
394         if (best == cur) { cur := target } else { cur := best }
395         hops <- net[cur][0]
396         guard := guard + 1
397     }
398     return [target, hops]
399 }
400
401 # =====
402 # L1: the mapping onto an NTT subscriber line
403 # -----
404 # phi : zone URL |-> NTT number. The ring position picks the area
405 # code (so neighbouring zones land in neighbouring exchanges) and the
406 # subscriber digits come from the URL hash. A registry -- the same
407 # device number portability uses -- makes phi^-1 an exact inverse.
408 # =====
409
410 AREA_CODE := ["011", "022", "03", "045", "052", "06", "075", "082", "092", "098"]
411 AREA_CITY := ["札幌", "仙台", "東京", "横浜", "名古屋", "大阪", "京都", "広島", "福岡", "那覇"]
412 AREA_SUB := [ 7, 7, 8, 7, 7, 8, 7, 7, 7, 7]
413
414 REG_URL := [] # the allocation ledger: parallel arrays
415 REG_NUM := []
416 REG_SEC := []
417
418 def pad_digits(v, width) {
419     s := "" + (v % ipow(10, width))
420     while (len(s) < width) { s := "0" + s }
421     return s
422 }
423
424 def reg_find_num(number) {
425     i := 0
426     while (i < len(REG_NUM)) {
427         if (REG_NUM[i] == number) { return i }
428         i := i + 1
429     }
430     return 0 - 1
431 }
432
433 def reg_find_url(url) {
434     i := 0
435     while (i < len(REG_URL)) {
436         if (REG_URL[i] == url) { return i }
437         i := i + 1
438     }

```

```

439     return 0 - 1
440 }
441
442 # phi
443 def ntt_map(url) {
444     hit := reg_find_url(url)
445     if (hit >= 0) { return [REG_NUM[hit], REG_SEC[hit]] }
446     z := zone_id(url)
447     sector := u_trunc(z * len(AREA_CODE) / RING)
448     if (sector >= len(AREA_CODE)) { sector := len(AREA_CODE) - 1 }
449     area := AREA_CODE[sector]
450     width := AREA_SUB[sector]
451     base := str_hash("ntt:" + url) * 7919 % ipow(10, width)
452     probe := 0
453     number := area + pad_digits(base, width)
454     while (reg_find_num(number) >= 0 && probe < 1000) {
455         probe := probe + 1
456         number := area + pad_digits(base + probe, width)
457     }
458     REG_URL <- url
459     REG_SEC <- sector
460     REG_NUM <- number;
461     ["ntt-assign", url, number] >> tuplespace
462     return [number, sector]
463 }
464
465 # display form: 03-1234-5678 / 082-199-4512
466 def ntt_format(number) {
467     n := len(number)
468     if (n != 10) { return number }
469     a3 := substr(number, 0, 3)
470     if (substr(number, 1, 1) == "3") { a3 := substr(number, 0, 2) }
471     if (substr(number, 1, 1) == "6") { a3 := substr(number, 0, 2) }
472     head := len(a3)
473     mid := 10 - head - 4
474     return a3 + "-" + substr(number, head, mid) + "-" + substr(number, head + mid, 4)
475 }
476
477 # phi^-1 -- exact, because the registry recorded the allocation
478 def ntt_unmap(number) {
479     hit := reg_find_num(number)
480     if (hit < 0) { return "" }
481     return REG_URL[hit]
482 }
483
484 # ---- the bearer: a voiceband 16-QAM soft modem -----
485 # A metallic subscriber line passes only 300-3400 Hz, so the envelope
486 # is modulated into it. fs = 8000 Hz and fc = 2000 Hz, so fc/fs = 1/4
487 # and the carrier samples are exactly {1,0,-1,0} and {0,1,0,-1}: the
488 # correlator is then an exact orthogonal pair and demodulation is
489 # error-free. 2000 baud x 4 bits = 8000 bps.
490 QAM_LVL := [0 - 3, 0 - 1, 1, 3]
491 QAM_AMP := 2600
492
493 def qam_index(level) {
494     best := 0
495     bd := 9999
496     i := 0
497     while (i < 4) {
498         d := abs(QAM_LVL[i] - level)
499         if (d < bd) { bd := d; best := i }
500         i := i + 1
501     }
502     return best
503 }
504
505 # byte array -> voiceband samples
506 def ntt_modulate(bytes) {
507     out := []
508     i := 0
509     while (i < len(bytes)) {
510         b := bytes[i]
511         # one byte = two 16-QAM symbols (4 bits each)
512         s := 0

```

```

513     while (s < 2) {
514         nib := b % 16
515         if (s == 0) { nib := u_trunc(b / 16) }
516         I := QAM_LVL[u_trunc(nib / 4)]
517         Q := QAM_LVL[nib % 4]
518         out <- QAM_AMP * I
519         out <- 0 - QAM_AMP * Q
520         out <- 0 - QAM_AMP * I
521         out <- QAM_AMP * Q
522         s := s + 1
523     }
524     i := i + 1
525 }
526 return out
527 }
528
529 # voiceband samples -> byte array
530 def ntt_demodulate(samples) {
531     out := []
532     nsym := u_trunc(len(samples) / 4)
533     k := 0
534     acc := 0
535     half := 0
536     while (k < nsym) {
537         a := samples[k * 4]
538         b := samples[k * 4 + 1]
539         c := samples[k * 4 + 2]
540         d := samples[k * 4 + 3]
541         I := 0.5 * (a - c) / QAM_AMP
542         Q := 0 - 0.5 * (b - d) / QAM_AMP
543         nib := qam_index(I) * 4 + qam_index(Q)
544         if (half == 0) { acc := nib * 16; half := 1 }
545         else { out <- (acc + nib); half := 0 }
546         k := k + 1
547     }
548     return out
549 }
550
551 # =====
552 # L0: Panasonic HD-PLC -- the house wiring as the physical layer
553 # -----
554 # Zimmermann-Dostert multipath transfer function, colored background
555 # noise, amateur-band notches, adaptive PAM bit loading, and a
556 # wavelet (Haar) filter bank standing in for HD-PLC's DWT.
557 # =====
558
559 HAM_LO := [1.8, 3.5, 7.0, 10.1, 14.0, 18.068, 21.0, 24.89, 28.0]
560 HAM_HI := [2.0, 3.8, 7.2, 10.15, 14.35, 18.168, 21.45, 24.99, 29.7]
561
562 def in_ham_band(f_hz) {
563     f := f_hz / 1000000
564     i := 0
565     while (i < len(HAM_LO)) {
566         if (f >= HAM_LO[i] && f <= HAM_HI[i]) { return true }
567         i := i + 1
568     }
569     return false
570 }
571
572 # outlets: [name, distance m, branches]
573 def outlet(which) {
574     if (which == "living") { return ["居間 コンセント", 12, 3] }
575     if (which == "kitchen") { return ["台所 コンセント", 17, 5] }
576     if (which == "bedroom") { return ["寝室 コンセント", 28, 4] }
577     if (which == "router") { return ["ONU 脇 コンセント", 5, 2] }
578     return ["書斎 コンセント (PC)", 21, 4]
579 }
580
581 # the multipath set for a pair of outlets: [gain, path length]
582 def plc_paths(a, b) {
583     oa := outlet(a)
584     ob := outlet(b)
585     base := oa[1] + ob[1]
586     nb := oa[2]

```

```

587     if (ob[2] > nb) { nb := ob[2] }
588     npath := 3 + nb - 1
589     if (npath > 6) { npath := 6 }
590     paths := []
591     i := 0
592     while (i < npath) {
593         g := 1.0
594         if (i > 0) { g := 0.75 - 0.12 * i }
595         if (i % 2 == 1) { g := 0 - g }
596         d := base
597         if (i > 0) { d := base + i * 11 }
598         paths <- [g, d]
599         i := i + 1
600     }
601     return paths
602 }
603
604 # |H(f)| : attenuation exp(-a1 f d) and comb fading from path delays
605 def plc_transfer(paths, f_hz) {
606     a1 := 0.00000000078
607     vp := 150000000
608     re := 0.0
609     im := 0.0
610     i := 0
611     while (i < len(paths)) {
612         att := exp(0 - a1 * f_hz * paths[i][1])
613         ph := 0 - TWO_PI * f_hz * paths[i][1] / vp
614         re := re + paths[i][0] * att * u_cos(ph)
615         im := im + paths[i][0] * att * u_sin(ph)
616         i := i + 1
617     }
618     return sqrt(re * re + im * im)
619 }
620
621 # impedance mismatch at every branch; the loss rises with frequency
622 def plc_coupling_db(a, b, f_hz) {
623     oa := outlet(a)
624     ob := outlet(b)
625     j := (oa[2] - 1) + (ob[2] - 1)
626     db := 3.0 * j + 5.0
627     t := (f_hz - FLO) / (FHI - FLO)
628     return db + (14 + 2.6 * j) * t
629 }
630
631 # colored background noise, dBm/Hz
632 def plc_noise_db(f_hz) {
633     fm := f_hz / 1000000
634     return 0 - 145 + 53.23 * exp(0 - 0.337 * log(fm))
635 }
636
637 # the tone map: bits for each subcarrier, 0 where notched or too weak
638 def plc_tonemap(a, b) {
639     paths := plc_paths(a, b)
640     df := (FHI - FLO) / NCARR
641     gap := 9.55 # SNR gap for 1e-7 BER
642     bits := []
643     snrs := []
644     total := 0
645     active := 0
646     notches := 0
647     n := 0
648     while (n < NCARR) {
649         f := FLO + (n + 0.5) * df
650         if (in_ham_band(f)) {
651             bits <- 0
652             snrs <- 0
653             notches := notches + 1
654         } else {
655             h := plc_transfer(paths, f)
656             if (h < 0.000000000001) { h := 0.000000000001 }
657             hdb := 20 * log(h) / 2.302585092994046 - plc_coupling_db(a, b, f)
658             snr := TXPSD + hdb - plc_noise_db(f)
659             lin := exp(snr * 0.23025850929940458)
660             bq := u_trunc(u_log2(1 + lin / gap))

```

```

661         if (bq < 1) { bq := 0 }
662         if (bq > MAXBITS) { bq := MAXBITS }
663         bits <- bq
664         snrs <- snr
665         if (bq > 0) { active := active + 1; total := total + bq }
666     }
667     n := n + 1
668 }
669 sym_sec := 1.0 / df
670 return [bits, total, active, notches, total / sym_sec, snrs]
671 }
672
673 # ---- the wavelet filter bank (normalised Haar; exact inverse) -----
674 SQ1_2 := 0.7071067811865476
675
676 # time waveform -> wavelet coefficients (subbands = subcarriers)
677 def dwt_analyze(x) {
678     a := []
679     i := 0
680     while (i < len(x)) { a <- x[i]; i := i + 1 }
681     n := len(a)
682     while (n > 1) {
683         half := u_trunc(n / 2)
684         out := []
685         i := 0
686         while (i < half) { out <- (a[2 * i] + a[2 * i + 1]) * SQ1_2; i := i + 1 }
687         i := 0
688         while (i < half) { out <- (a[2 * i] - a[2 * i + 1]) * SQ1_2; i := i + 1 }
689         i := 0
690         while (i < n) { a[i] = out[i]; i := i + 1 }
691         n := half
692     }
693     return a
694 }
695
696 # wavelet coefficients -> time waveform (the exact inverse)
697 def dwt_synthesize(c) {
698     a := []
699     i := 0
700     while (i < len(c)) { a <- c[i]; i := i + 1 }
701     N := len(a)
702     n := 1
703     while (n < N) {
704         half := n
705         n := n * 2
706         out := []
707         i := 0
708         while (i < n) { out <- 0; i := i + 1 }
709         i := 0
710         while (i < half) {
711             out[2 * i] = (a[i] + a[half + i]) * SQ1_2
712             out[2 * i + 1] = (a[i] - a[half + i]) * SQ1_2
713             i := i + 1
714         }
715         i := 0
716         while (i < n) { a[i] = out[i]; i := i + 1 }
717     }
718     return a
719 }
720
721 # ---- bytes <-> bits -----
722 def bytes_to_bits(bytes) {
723     bits := []
724     i := 0
725     while (i < len(bytes)) {
726         v := bytes[i]
727         k := 7
728         while (k >= 0) {
729             p := POW2[k]
730             if (v >= p) { bits <- 1; v := v - p } else { bits <- 0 }
731             k := k - 1
732         }
733         i := i + 1
734     }

```

```

735     return bits
736 }
737
738 def bits_to_bytes(bits) {
739     out := []
740     i := 0
741     while (i + 7 < len(bits)) {
742         v := 0
743         k := 0
744         while (k < 8) { v := v * 2 + bits[i + k]; k := k + 1 }
745         out <- v
746         i := i + 8
747     }
748     return out
749 }
750
751 # ---- wavelet OFDM -----
752 def plc_modulate(bytes, tm) {
753     tone := tm[0]
754     per := tm[1]
755     bits := bytes_to_bits(bytes)
756     nsym := u_trunc((len(bits) + per - 1) / per)
757     pad := nsym * per - len(bits)
758     i := 0
759     while (i < pad) { bits <- 0; i := i + 1 }
760
761     samples := []
762     p := 0
763     s := 0
764     while (s < nsym) {
765         coeffs := []
766         n := 0
767         while (n < NCARR) { coeffs <- 0; n := n + 1 }
768         n := 0
769         while (n < NCARR) {
770             bq := tone[n]
771             if (bq > 0) {
772                 v := 0
773                 k := 0
774                 while (k < bq) { v := v * 2 + bits[p]; p := p + 1; k := k + 1 }
775                 coeffs[n] = 2 * v - (POW2[bq] - 1) # PAM level
776             }
777             n := n + 1
778         }
779         wave := dwt_synthesize(coeffs)
780         i := 0
781         while (i < NCARR) { samples <- wave[i]; i := i + 1 }
782         s := s + 1
783     }
784     return [samples, nsym, len(bits) - pad]
785 }
786
787 def plc_demodulate(samples, tm, bitlen) {
788     tone := tm[0]
789     nsym := u_trunc(len(samples) / NCARR)
790     bits := []
791     s := 0
792     while (s < nsym) {
793         chunk := []
794         i := 0
795         while (i < NCARR) { chunk <- samples[s * NCARR + i]; i := i + 1 }
796         coeffs := dwt_analyze(chunk)
797         n := 0
798         while (n < NCARR) {
799             bq := tone[n]
800             if (bq > 0) {
801                 M := POW2[bq]
802                 idx := u_trunc((coeffs[n] + (M - 1) + 1) / 2) # hard slicer
803                 if (idx < 0) { idx := 0 }
804                 if (idx > M - 1) { idx := M - 1 }
805                 k := bq - 1
806                 while (k >= 0) {
807                     p := POW2[k]
808                     if (idx >= p) { bits <- 1; idx := idx - p } else { bits <- 0 }

```

```

809         k := k - 1
810     }
811 }
812     n := n + 1
813 }
814     s := s + 1
815 }
816 trimmed := []
817 i := 0
818 while (i < bitlen && i < len(bits)) { trimmed <- bits[i]; i := i + 1 }
819 return bits_to_bytes(trimmed)
820 }
821
822 # ---- framing (IEEE 1901 style; modular hash stands in for CRC-32) ---
823 PREAMBLED := [26, 207, 252, 29]
824
825 def plc_check(bytes) {
826     h := 216
827     i := 0
828     while (i < len(bytes)) {
829         h := (h * 33 + bytes[i] + 11) % 16777213
830         i := i + 1
831     }
832     return h
833 }
834
835 def plc_frame(payload) {
836     out := []
837     i := 0
838     while (i < len(PREAMBLE)) { out <- PREAMBLE[i]; i := i + 1 }
839     n := len(payload)
840     out <- u_trunc(n / 256)
841     out <- n % 256
842     i := 0
843     while (i < n) { out <- payload[i]; i := i + 1 }
844     c := plc_check(payload)
845     out <- u_trunc(c / 65536) % 256
846     out <- u_trunc(c / 256) % 256
847     out <- c % 256
848     return out
849 }
850
851 # -> [status, payload]; status 200 delivered / 503 physical error
852 def plc_deframe(bytes) {
853     i := 0
854     while (i < len(PREAMBLE)) {
855         if (bytes[i] != PREAMBLE[i]) { return [503, []] }
856         i := i + 1
857     }
858     n := bytes[4] * 256 + bytes[5]
859     payload := []
860     i := 0
861     while (i < n) { payload <- bytes[6 + i]; i := i + 1 }
862     got := bytes[6 + n] * 65536 + bytes[7 + n] * 256 + bytes[8 + n]
863     if (got != plc_check(payload)) { return [503, []] }
864     return [200, payload]
865 }
866
867 # =====
868 # L4: the unified message (what LINE and Instagram both become)
869 # -----
870 # The app ingests real LINE Messaging API and Instagram Graph API
871 # webhook events and verifies their HMAC-SHA256 signatures. Here we
872 # carry the normalised result: platform, chat id, display name, text.
873 # =====
874
875 def msg_new(platform, chat, who, text) { return [platform, chat, who, text] }
876
877 def msg_serialize(m) { return m[0] + "~" + m[1] + "~" + m[2] + "~" + m[3] }
878
879 def msg_deserialize(s) {
880     f := split(s, "~")
881     if (len(f) < 4) { return ["zone", "", "", s] }
882     return [f[0], f[1], f[2], f[3]]

```

```

883 }
884
885 def msg_zone_url(m) { return "zone://url.or.jp/msg/" + m[0] + "/" + m[1] }
886
887 # =====
888 # The STREAMS stack itself
889 # -----
890 # Each module is a pair wput (downstream) / rput (upstream). The
891 # stream head hands a message down; the driver puts it on the wire;
892 # coming back up every module undoes exactly what its twin did.
893 # =====
894
895 NET := zone_boot()
896 PLC_LINK := ["living", "study"]
897 TONEMAP := plc_tonemap("living", "study")
898
899 # --- msgmux -----
900 def msgmux_wput(m) { return [msg_zone_url(m), msg_serialize(m)] }
901 def msgmux_rput(pair) { return msg_deserialize(pair[1]) }
902
903 # --- jones -----
904 # -> [status, url, sealed] ; 495 if the quantum channel was tapped
905 def jones_wput(pair) {
906     url := pair[0]
907     z := zone_parse(url)
908     if (len(z) == 0) { return [400, url, []] }
909     q := qkd_session()
910     if (!q[0]) { return [495, url, []] }
911     key := jones_key(zone_diagram(z[0]))
912     return [200, url, zone_seal(pair[1], key, q[1])]
913 }
914
915 # -> [status, url, plaintext] ; 409 if the seal does not verify
916 def jones_rput(env) {
917     url := env[1]
918     z := zone_parse(url)
919     if (len(z) == 0) { return [400, url, ""] }
920     key := jones_key(zone_diagram(z[0]))
921     opened := zone_open(env[2], key)
922     if (len(opened) == 0) { return [409, url, ""] }
923     return [200, url, opened[0]]
924 }
925
926 # --- zone -----
927 def zone_wput(env) {
928     url := env[1]
929     key := zone_id(url)
930     r := zone_route(NET, key);
931     ["zone-record", url, NET[r[0]][0], key, env[2][2]] >> tuplespace
932     return r
933 }
934
935 def zone_rput(url) {
936     key := zone_id(url)
937     r := zone_route(NET, key);
938     ["zone-fetch", url, NET[r[0]][0]] >> tuplespace
939     return r
940 }
941
942 # --- ntt -----
943 # the sealed record becomes a byte envelope, then a voiceband signal
944 def envelope_encode(url, sealed) {
945     out := []
946     ub := []
947     i := 0
948     while (i < len(url)) { ub <- ord(url, i); i := i + 1 }
949     out <- 90 # 'Z'
950     out <- 78 # 'N'
951     out <- len(ub)
952     i := 0
953     while (i < len(ub)) { out <- ub[i]; i := i + 1 }
954     out <- u_trunc(sealed[1] / 256)
955     out <- sealed[1] % 256
956     t := sealed[2]

```

```

957     out <- u_trunc(t / 16777216) % 256
958     out <- u_trunc(t / 65536) % 256
959     out <- u_trunc(t / 256) % 256
960     out <- t % 256
961     ct := sealed[0]
962     out <- u_trunc(len(ct) / 256)
963     out <- len(ct) % 256
964     i := 0
965     while (i < len(ct)) {
966         out <- u_trunc(ct[i] / 256)
967         out <- ct[i] % 256
968         i := i + 1
969     }
970     return out
971 }
972
973 def envelope_decode(bytes) {
974     if (bytes[0] != 90) { return [400, "", []] }
975     if (bytes[1] != 78) { return [400, "", []] }
976     ulen := bytes[2]
977     url := ""
978     i := 0
979     while (i < ulen) { url := url + chr(bytes[3 + i]); i := i + 1 }
980     p := 3 + ulen
981     salt := bytes[p] * 256 + bytes[p + 1]
982     tag := bytes[p + 2] * 16777216 + bytes[p + 3] * 65536 + bytes[p + 4] * 256 + bytes[p + 5]
983     clen := bytes[p + 6] * 256 + bytes[p + 7]
984     p := p + 8
985     ct := []
986     i := 0
987     while (i < clen) { ct <- (bytes[p + 2 * i] * 256 + bytes[p + 2 * i + 1]); i := i + 1 }
988     return [200, url, [ct, salt, tag]]
989 }
990
991 # =====
992 # ultra_send -- one message all the way down and all the way back
993 # =====
994
995 def ultra_send(m, note) {
996     print("")
997     print("-----")
998     print("SEND", note)
999     print(" message  :", m[0], "/", m[2], ":", m[3])
1000
1001     # ---- down the stream ----
1002     pair := msgmux_wput(m)
1003     print(" msgmux -> zone address", pair[0])
1004
1005     env := jones_wput(pair)
1006     if (env[0] != 200) {
1007         print(" jones -> STATUS", env[0], "quantum-channel-compromised")
1008         print(" the Bell pair lost its correlation: do not transmit")
1009         return env[0]
1010     }
1011     sealed := env[2]
1012     z := zone_parse(pair[0])
1013     print(" jones -> knot key", jones_key(zone_diagram(z[0])),
1014           " salt", sealed[1], " tag", sealed[2], " ct", len(sealed[0]), "units")
1015
1016     r := zone_wput(env)
1017     print(" zone -> key", zone_id(pair[0]), " owner", NET[r[0]][0])
1018
1019     num := ntt_map(pair[0])
1020     bytes := envelope_encode(pair[0], sealed)
1021     vb := ntt_modulate(bytes)
1022     print(" ntt -> phi =", ntt_format(num[0]), "(" + AREA_CITY[num[1]] + ")",
1023           " envelope", len(bytes), "bytes ->", len(vb), "voiceband samples")
1024
1025     frame := plc_frame(vb_to_bytes(vb))
1026     mod := plc_modulate(frame, TONEMAP)
1027     print(" plc ->", mod[1], "wavelet OFDM symbols",
1028           "PHY", u_trunc(TONEMAP[4] / 1000000), "Mbps",
1029           TONEMAP[2], "active /", TONEMAP[3], "notched")
1030

```

```

1031 # ---- the wire ----
1032 wire := mod[0]
1033 if (TAMPER_WIRE) { wire[10] = wire[10] + 4000 }
1034
1035 # ---- up the stream ----
1036 back := plc_demodulate(wire, TONEMAP, mod[2])
1037 df := plc_deframe(back)
1038 if (df[0] != 200) {
1039     print(" plc    <- STATUS 503 plc-check-failed (the wire was altered)")
1040     return 503
1041 }
1042 vb2 := bytes_to_vb(df[1])
1043 bytes2 := ntt_demodulate(vb2)
1044 dec := envelope_decode(bytes2)
1045 if (dec[0] != 200) { print(" ntt    <- STATUS 400 bad-zone"); return 400 }
1046 print(" ntt    <- phi^-1(", ntt_format(num[0]), ") =", ntt_unmap(num[0]))
1047
1048 r2 := zone_rput(dec[1])
1049 print(" zone    <- owner", NET[r2[0]][0], " route", r2[1])
1050
1051 sealed2 := dec[2]
1052 if (TAMPER_PEER) { sealed2[0][0] = (sealed2[0][0] + 1) % 65536 }
1053
1054 op := jones_rput([200, dec[1], sealed2])
1055 if (op[0] != 200) {
1056     print(" jones   <- STATUS 409 zone-guard-reject")
1057     print("          the tag does not verify: tampered, or the wrong knot")
1058     return 409
1059 }
1060 got := msgmux_rput([dec[1], op[2]])
1061 print(" msgmux <- message", got[0], "/", got[2], ":", got[3])
1062
1063 same := (got[3] == m[3]) && (got[2] == m[2]) && (got[0] == m[0])
1064 print(" STATUS 200 zone-delivered round-trip identical:", same)
1065 return 200
1066 }
1067
1068 # the PLC driver carries bytes, so pack the voiceband samples into
1069 # byte pairs and back (16-bit two's complement, big endian)
1070 def vb_to_bytes(vb) {
1071     out := []
1072     i := 0
1073     while (i < len(vb)) {
1074         v := vb[i]
1075         if (v < 0) { v := v + 65536 }
1076         out <- u_trunc(v / 256) % 256
1077         out <- v % 256
1078         i := i + 1
1079     }
1080     return out
1081 }
1082
1083 def bytes_to_vb(bytes) {
1084     out := []
1085     i := 0
1086     while (i + 1 < len(bytes)) {
1087         v := bytes[i] * 256 + bytes[i + 1]
1088         if (v >= 32768) { v := v - 65536 }
1089         out <- v
1090         i := i + 2
1091     }
1092     return out
1093 }
1094
1095 # measure a candidate outlet pair (":" inside a def is local, so this
1096 # reports the link rather than rebinding the running one)
1097 def ultra_link(which) {
1098     tm := plc_tonemap(which, "study")
1099     print("LINK", which, "-> study :", tm[2], "active subcarriers,",
1100         tm[3], "notched,", u_trunc(tm[4] / 1000000), "Mbps")
1101     return tm[2]
1102 }
1103
1104 # =====

```

```

1105 # the demonstration
1106 # =====
1107
1108 TAMPER_WIRE := false
1109 TAMPER_PEER := false
1110
1111 print("")
1112 print("ultra-network ring (RING =", RING, ", built from peer hashes):")
1113 i := 0
1114 while (i < len(NET)) {
1115     print("  node", i, ":", NET[i][0], " id =", NET[i][1])
1116     i := i + 1
1117 }
1118
1119 print("")
1120 print("zone identity knots -> Jones keys:")
1121 print(" url.or.jp (trefoil)      key =", jones_key(zone_diagram("url.or.jp")))
1122 print(" bada.or.jp (figure-eight) key =", jones_key(zone_diagram("bada.or.jp")))
1123
1124 print("")
1125 print("HD-PLC link", PLC_LINK[0], "->", PLC_LINK[1], ":")
1126 print("  subcarriers  :", NCARR, "over 2-28 MHz")
1127 print("  active       :", TONEMAP[2])
1128 print("  notched (ham) :", TONEMAP[3])
1129 print("  bits / symbol :", TONEMAP[1])
1130 print("  PHY rate     :", u_trunc(TONEMAP[4] / 1000000), "Mbps")
1131
1132 # --- the grammar verbs the reviser transaction installed -----
1133 print("")
1134 print("LINK -- measuring candidate outlets (the reviser verb):")
1135 LINK "router"
1136 LINK "living"
1137 LINK "kitchen"
1138
1139 # --- the happy path, once per messaging platform -----
1140 SEND msg_new("line", "U4af4980629", "田中 さくら", "コンセントから zone:// が見えてる") "LINE over the power line"
1141 SEND msg_new("instagram", "17841400000000001", "sakura.photo", "配線図の写真あげたよ") "Instagram over the power line"
1142
1143 # --- the attacks -----
1144 print("")
1145 print("=====")
1146 print("ATTACKS -- each one is rejected by a different layer")
1147 print("=====")
1148
1149 TAMPER_WIRE := true
1150 SEND msg_new("line", "U4af4980629", "田中 さくら", "改竄試験") "someone alters the wire"
1151 TAMPER_WIRE := false
1152
1153 TAMPER_PEER := true
1154 SEND msg_new("line", "U4af4980629", "田中 さくら", "改竄試験") "a malicious peer rewrites the record"
1155 TAMPER_PEER := false
1156
1157 # --- the mapping table -----
1158 print("")
1159 print("phi : zone:// |-> NTT number    (phi^-1 checked against the registry)")
1160 i := 0
1161 while (i < len(REG_URL)) {
1162     print("  ", ntt_format(REG_NUM[i]), " <-  ", REG_URL[i],
1163           "    inverse ok:", ntt_unmap(REG_NUM[i]) == REG_URL[i])
1164     i := i + 1
1165 }
1166
1167 print("")
1168 print("akashic ledger:", len(tuplespace), "facts (append-only)")
1169 print("=====")
1170 print("BADA ULTRA NETWORK -- done")
1171 print("=====")

```